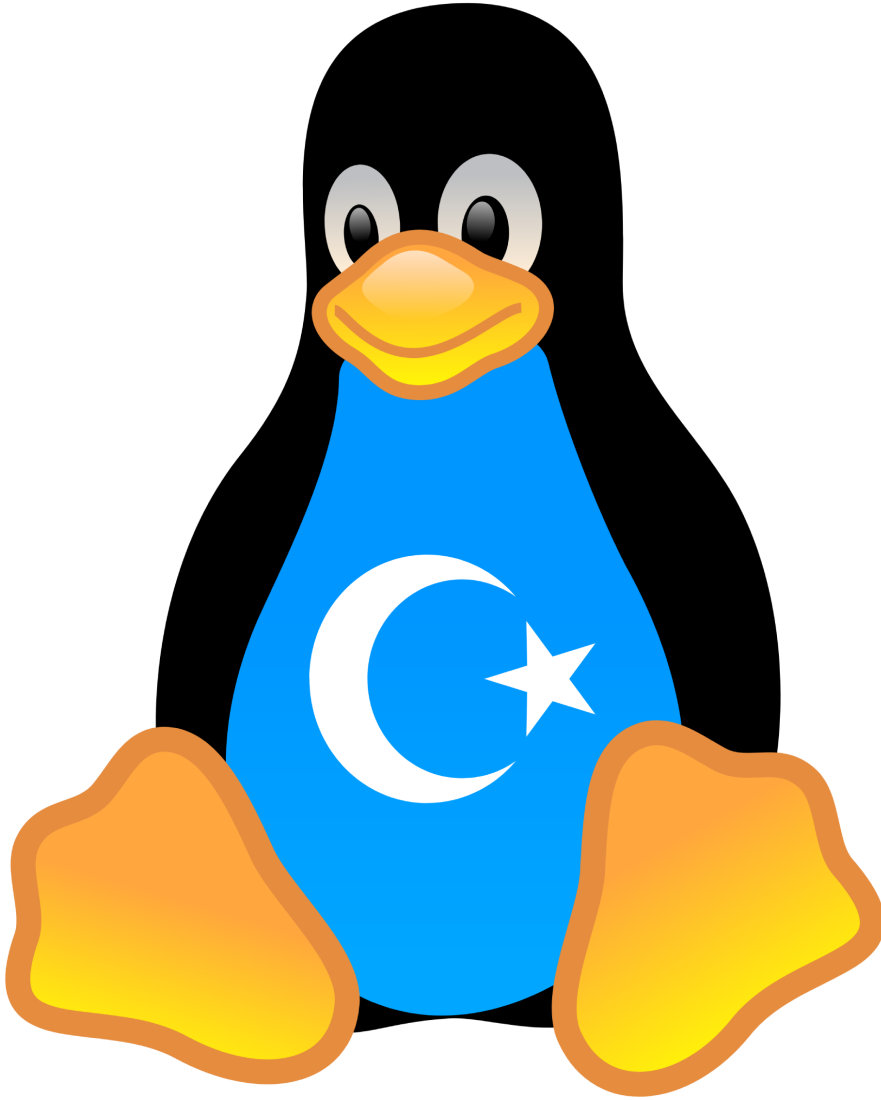


KENDİ LİNUX'UNU YAP



Yazar

- Bayram KARAHAN
- Celalettin AKARSU

Paket Derleme

- Bayram KARAHAN
- Celalettin AKARSU

İletişim

- <https://github.com/kendilinuxunuyap>
- <https://kendilinuxunuyap.github.io>
- kendilinuxunuyap@gmail.com

Ön Söz

Bu kitap, açık kaynak ve özgür yazılım dünyasına ilgi duyan herkes için hazırlanmıştır. Amacı, Türkçe kaynak eksikliğini gidermek ve kendi Linux dağıtımını oluşturmak isteyenlere yol göstermektir.

Bu serinin ilk kitabında, **Temel Linux Sistemi**'nin nasıl derlenip kullanılabilir hale getirileceği anlatılmıştı. Bu kitabımızda ise, oluşturduğumuz **Temel Linux Sistemi** üzerinde **X11 (grafik ortamının)** nasıl derlenip çalıştırılacağı adım adım açıklanacaktır.

Bu kitap, Xorg sunucusunu kaynaktan derlemek isteyen Linux kullanıcıları için kapsamlı bir rehber sunmayı amaçlamaktadır. Xorg, modern Linux sistemlerinde grafiksel kullanıcı arayüzlerinin temelini oluşturan bir bileşendir. Bu rehber, hem yeni başlayanlar hem de deneyimli sistem yöneticileri için tasarlanmıştır ve Debian/Ubuntu tabanlı sistemlere odaklanmaktadır.

Xorg'u kaynaktan derlemek, en son özellikleri kullanmak, özelleştirme yapmak veya belirli bir donanım için optimizasyon sağlamak isteyen kullanıcılar için idealdir. Ayrıca, bu süreç Linux sistemlerinin nasıl çalıştığını anlamak için harika bir öğrenme fırsatı sunar.

İçindekiler

1- Ön Hazırlık	5
3- temelsistem	6
4- GNU Araçlarıyla xorg ve x11 Derleme	9
6- ISO Hazırlama	94
7- Sistem Kurulumu	101
8- sistemcalistirma-inceleme	104
5- paketsistemi	109
9- Yardımcı Konular	120
10- Kaynaklar	140
11- Geliştiricilere Mesajımız	141

Temel Kavramlar

X Pencere Sistemi

X Pencere Sistemi, (X), daha çok GNU/Linux ve Unix benzeri işletim sistemlerinde kullanılan grafik arayüz altyapısıdır. Genel olarak ekrana çizdirme ve kullanıcıdan geribesleme alma işlerini yapan X sunucusunu ve uygulamaların sunucu ile haberleşme için kullanabileceği X kütüphanesini kapsar. Bu sunucu-istemci yapısı sistemin ağ üzerinden çalışmasına da olanak verir. Sunucu doğrudan donanımı yönetebildiği gibi, başka bir altyapı üzerinde de çalışabilir. Bu şekilde diğer işletim sistemlerinde de X sunucusu çalıştırılabilmektedir.

1984 yılında MIT tarafından geliştirilen bu sistem, ağ üzerinden grafiksel uygulamaların çalıştırılmasını mümkün kılar.

İstemci-Sunucu Mimarisi

X, istemci-sunucu mimarisiyle çalışır:

- *İstemciler*, grafik uygulamaları oluşturur.
- *Sunucu*, ekran, klavye ve fare gibi giriş/çıkış aygıtlarını kontrol eder.

Özellikler

X Pencere Sistemi şu özellikleri sunar:

- Çoklu pencere yönetimi
- Ekran paylaşımı
- Uzaktan erişim imkânı

Örneğin, bir X istemcisi, farklı bir bilgisayarda çalışan bir X sunucusuna bağlanarak çalışabilir. Bu sayede kullanıcılar, başka sistemlerdeki uygulamalara kolayca erişebilirler.

X11 Protokolü

Sistemin temelini **X11 protokolü** oluşturur ve bu protokol, geniş bir uygulama yelpazesine destek verir.

Kaynak

[X Pencere Sistemi](#)

Xorg Nedir?

Xorg (veya **X.Org Server**), Unix ve Linux tabanlı işletim sistemlerinde grafiksel kullanıcı arayüzünü (GUI) sağlayan **X Pencere Sistemi (X Window System)**'in en yaygın açık kaynaklı uygulamasıdır. Xorg, ekran, klavye, fare gibi giriş/çıkış aygıtları ile kullanıcı uygulamaları arasında aracı görevi görerek pencere yönetimini ve grafik çıktısı sağlar.

Kısaca Tanımı

Xorg, **X11 protokolünü** temel alarak çalışan bir **görüntü sunucusudur (display server)**. Masaüstü ortamlarının (GNOME, KDE, XFCE vb.) çalışmasını sağlar ama kendisi bir masaüstü ortamı değildir.

Xorg Ne Yapar?

- Grafik kartıyla iletişim kurarak pikselleri ekrana yerleştirir.
- Klavye ve fare gibi giriş aygıtlarını işler.
- Pencereyi yönetmez ama pencere yöneticilerinin çalışmasını sağlar.
- Uzaktaki makinelerden grafik uygulamalarını çalıştırabilir (X forwarding).

Temel Bileşenler

Xorg, aşağıdaki bileşenleri içerebilir veya bunlarla birlikte çalışır:

- **xorg.conf**: Yapılandırma dosyası (modern sistemlerde genellikle otomatik yapılandırma kullanılır).
- **DRI/DRM**: *Direct Rendering Infrastructure / Direct Rendering Manager*; doğrudan donanım erişimi.
- **Mesa**: Açık kaynak OpenGL implementasyonu.
- **Xlib, XCB**: X protokolünü kullanan programlama arabirimleri.

Yapılandırma Dosyaları

- `/etc/X11/xorg.conf` (eski sistemlerde)
- `/etc/X11/xorg.conf.d/` dizini altındaki parçalı yapılandırmalar
- Modern sistemlerde otomatik olarak `/var/log/Xorg.0.log` dosyasına günlük yazar

Kaynaklar

- [X.Org Foundation – Resmi Site](#)
- [X.Org Server GitHub](#)
- [Arch Wiki – Xorg](#)
- [Gentoo Wiki – Xorg Guide](#)

X11 Nedir?

X11 (veya **X Window System, version 11**), Unix benzeri işletim sistemlerinde grafiksel kullanıcı arayüzlerini oluşturmak için kullanılan istemci-sunucu tabanlı bir pencereleme sistemidir. İlk olarak 1984 yılında MIT tarafından geliştirilen X Window System'in 11. versiyonudur ve günümüzde birçok Linux ve Unix sistemde temel grafik altyapısını oluşturur.

X11 Ne İşe Yarar?

- Ekran, klavye, fare gibi aygıtlar ile çalışan uygulamalar arasında aracı olur.
- GUI (grafik kullanıcı arayüzü) oluşturmaz, ama masaüstü ortamlarının GUI'yi oluşturmaya olanak tanır.
- İstemci-sunucu modeli sayesinde uzaktan masaüstü kullanımına izin verir (örneğin ssh -X ile).

Temel Özellikleri

- **Ağ şeffaflığı:** X11, ağ üzerinden çalışan istemcilerin sunucuya bağlanmasını destekler.
- **Donanımdan bağımsızlık:** Grafik donanım soyutlaması sağlar.
- **Uzun ömür:** 1980'lerden beri gelişmiş ve kullanılmaktadır.
- **Modüler yapı:** X sunucusu, pencere yöneticisi ve istemciler ayrı bileşenlerdir.

X11 ile İlgili Bileşenler

- **X Server:** X protokolünü uygulayan grafik sunucusu (örnek: Xorg)
- **X Clients:** Grafik uygulamaları (örnek: terminal, dosya yöneticisi)
- **Window Manager:** Pencere davranışlarını kontrol eden yazılım (örnek: i3, Openbox)
- **Display Manager:** Oturum açmayı yöneten grafik arayüz (örnek: GDM, LightDM)

X11 ile Xorg Arasındaki Fark

- **X11**, bir protokoldür. Grafik uygulamaların nasıl davranması gerektiğini tanımlar.
- **Xorg**, X11 protokolünü uygulayan yazılımdır. Yani X11'in çalışan ve somut hale getirilmiş halidir.

Kaynaklar

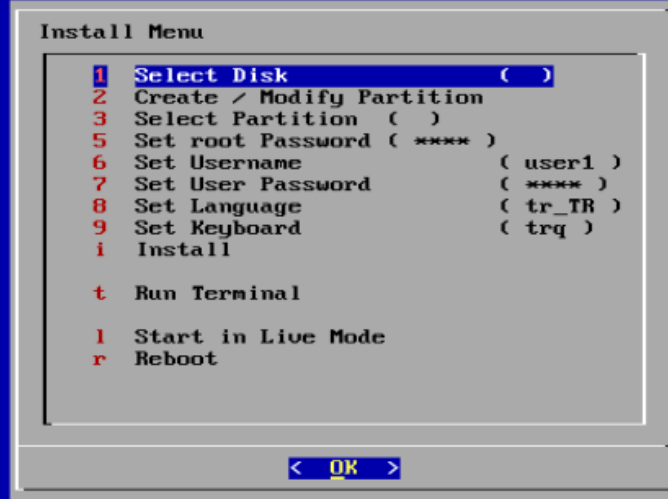
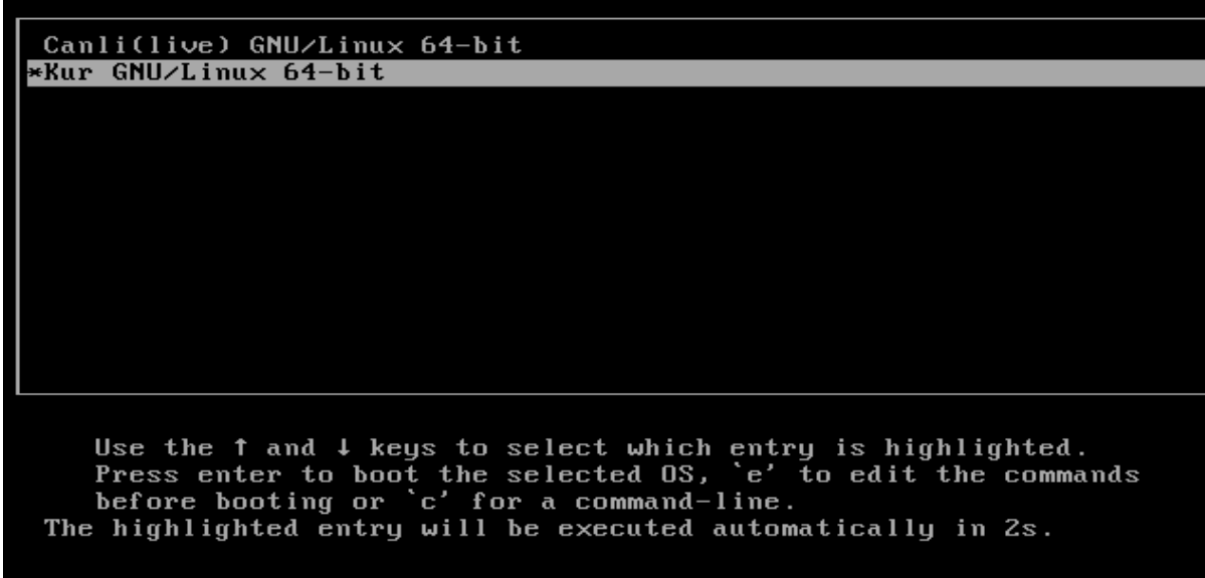
- [X Window System - Wikipedia](#)
- [X.Org Foundation](#)
- [man X \(X11 manual\)](#)
- [Arch Wiki - Xorg](#)

Temel Sistemin Hazırlanması

Bir önceki kitabımızda Temel Linux sistemini oluşturmayı ve iso halinde kurulumu anlatmıştık. Şimdi ise bu temel sistemi kurarak bu sistem üzerine **xorg** derleyerek **x11** pencere sisteminin nasıl çalışacağı anlatılacaktır.

Temel Sistem Kurulumu

Sistemin kurulumu için resimlerde görünen sıraya göre seçimler yapmalıyız.



Kurulum menüsünde kullanıcı adları ve parolaları, klavye varsayılan olarak;

- Kullanıcı: root Parola: 1
- Kullanıcı: user1 Parola: 1
- Dil : tr_TR
- Klavye : trq

menüden değişiklik yapabilirsiniz. Değişiklik yapmadan sadece kurulum diskini ve disk bölümünü seçip Install(Yükle) işlemi yapabilirsiniz.



```
kurulumu geçildi.....
legacy kurulum .....
Kurulum Yapılacak Disk sda
mount: /kaynak: WARNING: source write-protected, mounted read-only.
*****disk bölümleri biçimlendiriliyor *****
e2fsck 1.47.0 (5-Feb-2023)
e2fsck: need terminal for interactive repairs
tune2fs 1.47.0 (5-Feb-2023)
Recovering journal.

This operation requires a freshly checked filesystem.

Please run e2fsck -f on the filesystem.

mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 2096896 4k blocks and 524288 inodes
Filesystem UUID: 9ea082d0-a034-4dab-8e2f-564e68c99c9c
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

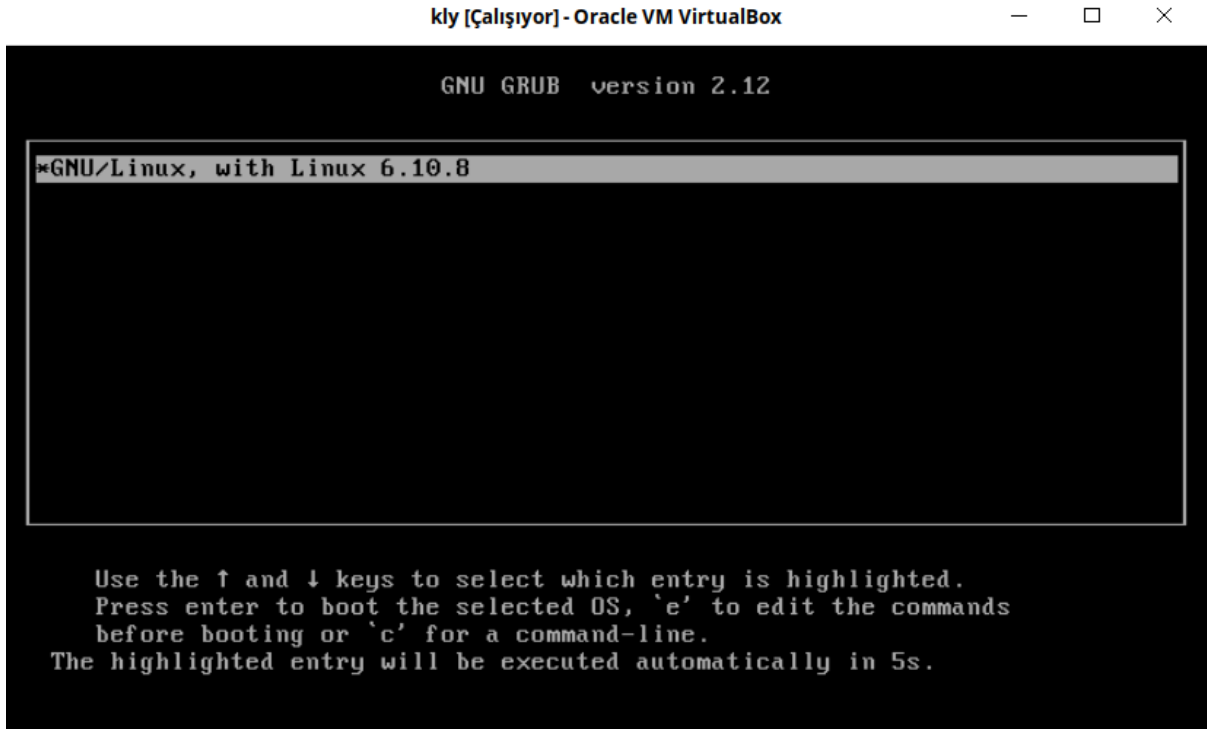
Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

Information: You may need to update /etc/fstab.

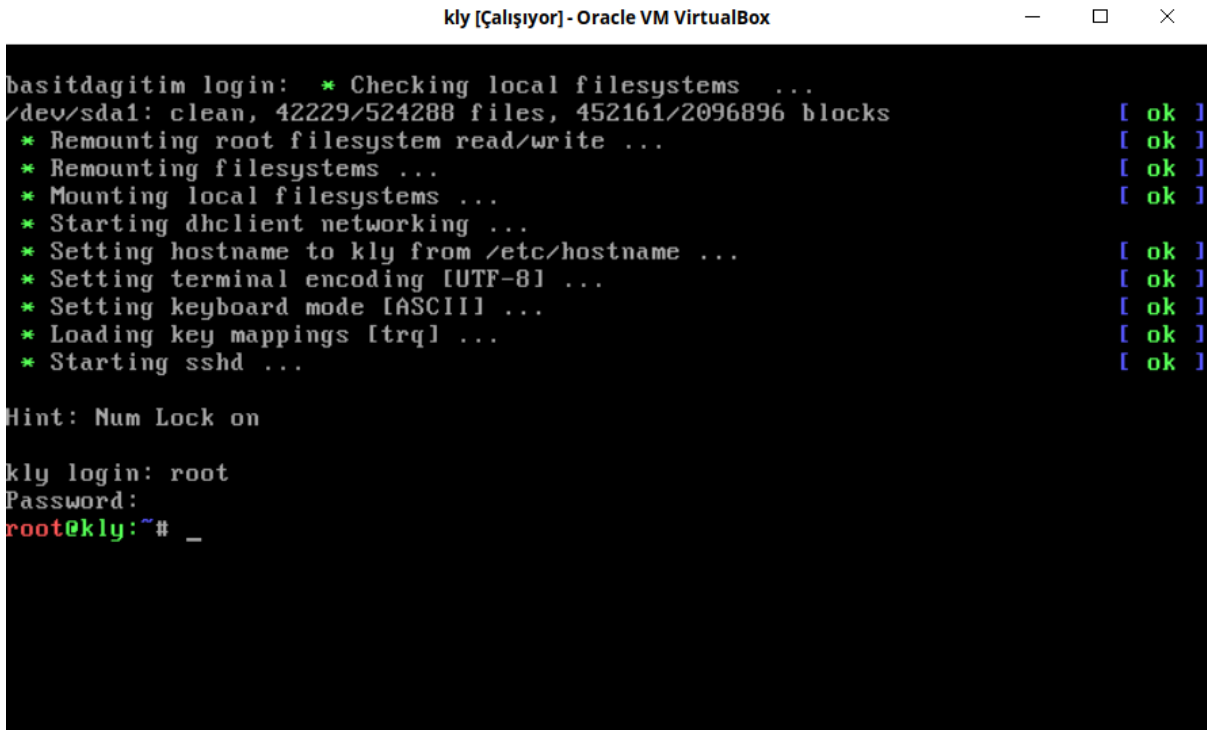
***** kurulum başladı*****
Sistem yüklenmeye başlandı. Tahmini 3-5 dakika sürecektir.. Lütfen bekleyiniz.....
.....
```

Sistemin Çalışması

Sistem kurulumu gerçekleştiğinde sistem resimde görüldüğü gibi açılmalıdır.



Sisteme **root** kullanıcısı olarak giriş yapıldığı görülmektedir.



Temel Sisteme Internet Bağlantısı

Temel Sistem içerisinde bulunan **iproute2**, **dhclient**, **net-tools** paketleri derlendi ve açılışta **dhclient** aktif hale gelmektedir. Bu paket ile ağa dahil olunmaktadır.

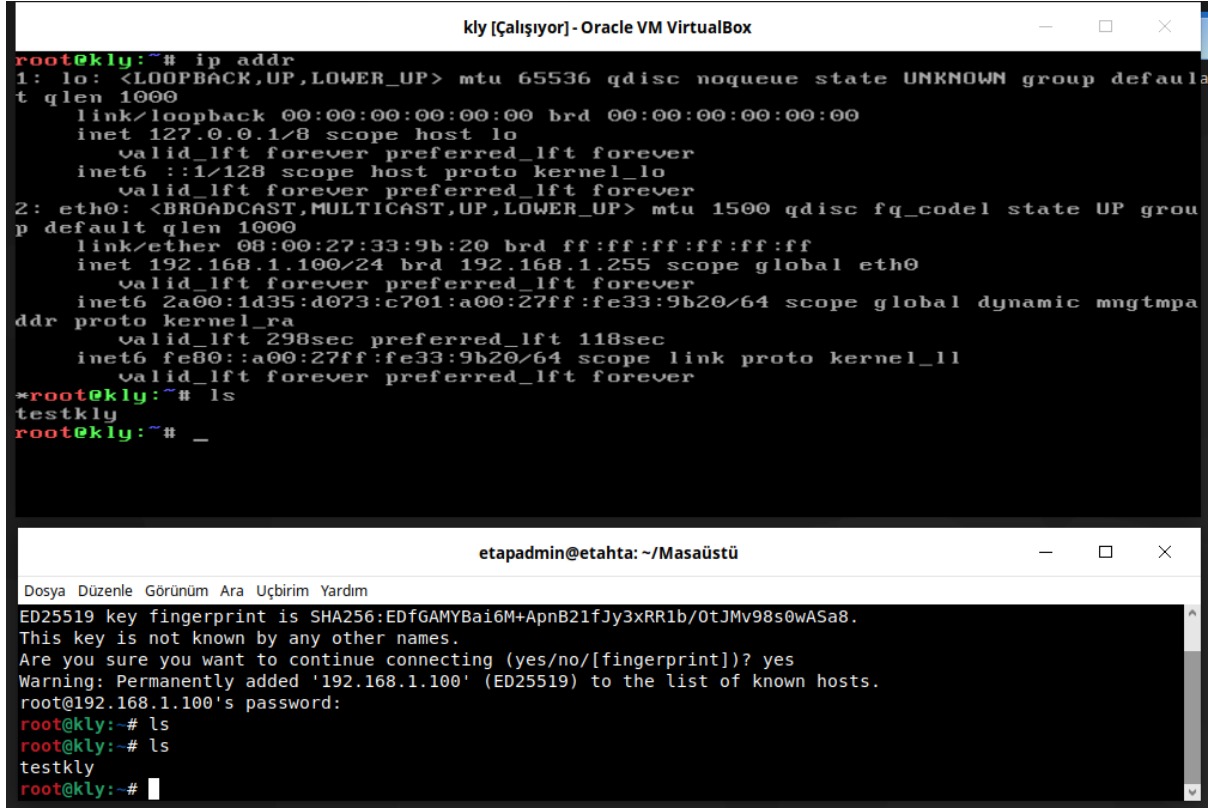
Aşağıda **kly Temel Sistem** ile atanan ip adresini görmekteyiz. Eğer ip adresi almamışsa terminalde **dhclient** komutunu çalıştırınız.

```
kly [Çalışıyor] - Oracle VM VirtualBox
root@kly:~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:33:9b:20 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 2a00:1d35:d073:c701:a00:27ff:fe33:9b20/64 scope global dynamic mngtmpa
        valid_lft 298sec preferred_lft 118sec
    inet6 fe80::a00:27ff:fe33:9b20/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@kly:~# uname -a
Linux kly 6.10.8 #1 SMP PREEMPT_DYNAMIC Sat Sep  7 18:49:23 +03 2024 x86_64 GNU/Linux
root@kly:~#
```

Temel Sisteme openssh ile Bağlanma

ssh terminal üzerinden uzak bilgisayarlara erişim yapan bir uygulamadır. **Temel Sistem** içerisinde ssh paketi derlendi ve açılıştan aktif hale gelmektedir. ssh kullanımı **Yardımcı Konular** bölümünde anlatılmıştır.

Aşağıda **kly Temel Sisteme ssh** ile erişimin nasıl yapıldığı görülmektedir.



The image shows two terminal windows. The top window is titled 'kly [Çalışıyor] - Oracle VM VirtualBox' and shows the output of the 'ip addr' command. It displays details for the loopback interface 'lo' (127.0.0.1) and the ethernet interface 'eth0' (192.168.1.100). The bottom window is titled 'etapadmin@etahta: ~/Masaüstü' and shows the output of the 'ls' command. It displays the output of the 'ls' command, showing the directory structure of the user's home directory.

```
root@kly:~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:33:9b:20 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 2a00:1d35:d073:c701:a00:27ff:fe33:9b20/64 scope global dynamic mngtmpa
    inet6 fe80::a00:27ff:fe33:9b20/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
*root@kly:~# ls
testkly
root@kly:~# _
```

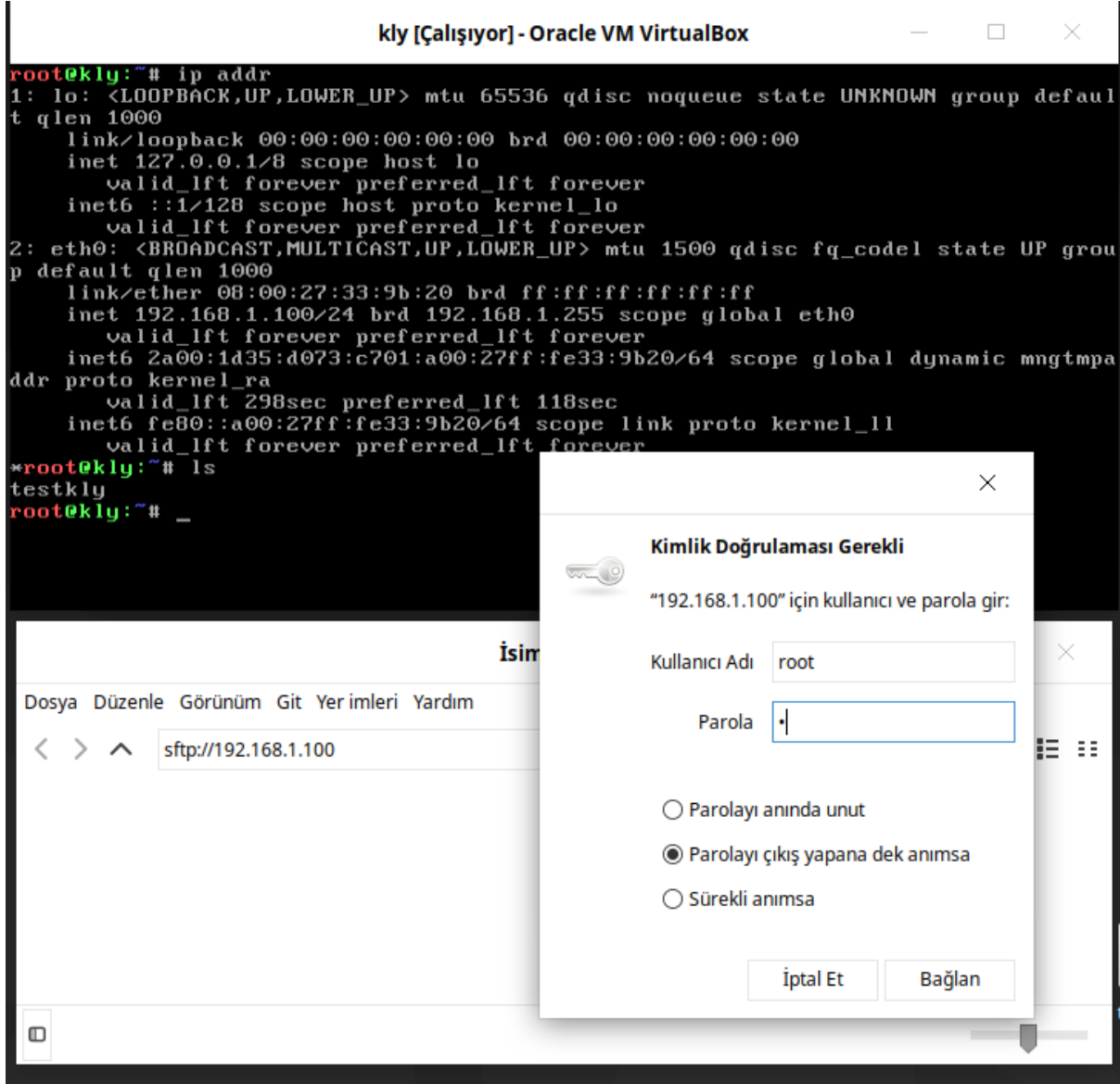
```
etapadmin@etahta: ~/Masaüstü
Dosya Düzenle Görünüm Ara Uçbirim Yardım
ED25519 key fingerprint is SHA256:EDfGAMyBai6M+ApnB21fJy3xRR1b/OtJMv98s0wASa8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.100' (ED25519) to the list of known hosts.
root@192.168.1.100's password:
root@kly:~# ls
testkly
root@kly:~#
```

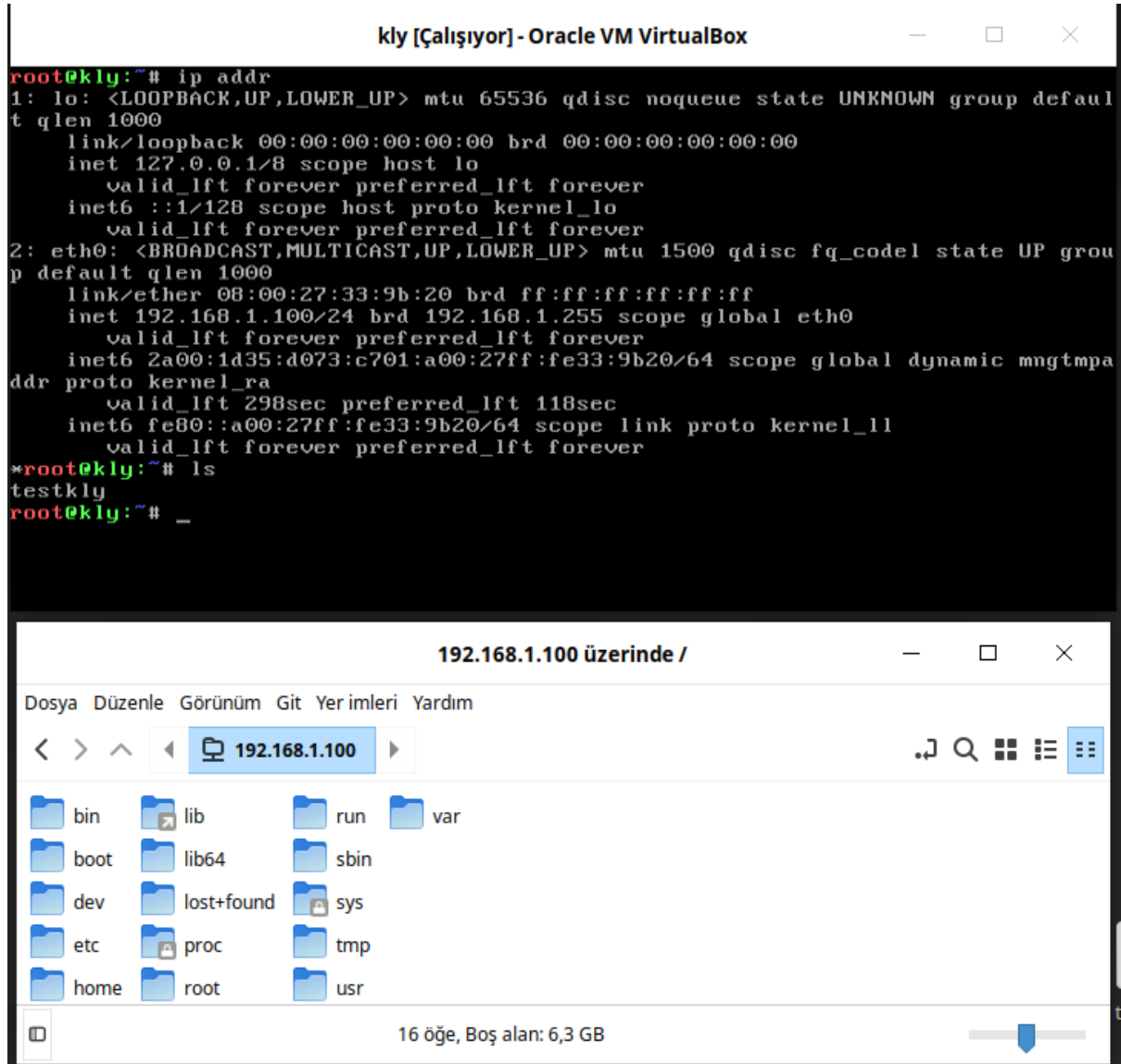
Bu doküman boyunca paketleri **Debian** ortamında derleyeceğiz. Derlediğimiz paketleri **Temel Sistem'e** kopyalacağız. Kopyalama işlemini **ssh** paketi içinde gelen **sftp** veya **scp** ile yapacağız. Kopyalanan paketlerin **kurulması ve kaldırılması** gibi işlemleri **ssh** bağlantısı üzerinden gerçekleştireceğiz.

Temel Sisteme sftp ile Bağlanma

sftp terminal veya pencere yöneticisi üzerinden uzak bilgisayarlara erişim yapan bir uygulamadır. **Temel Sistem** içerisinde **ssh** paketinin bir parçası olarak gelmektedir. sftp kullanımı **Yardımcı Konular** bölümünde anlatılmıştır.

Aşağıda **kly Temel Sisteme sftp** ile erişimin nasıl yapıldığı görülmektedir.

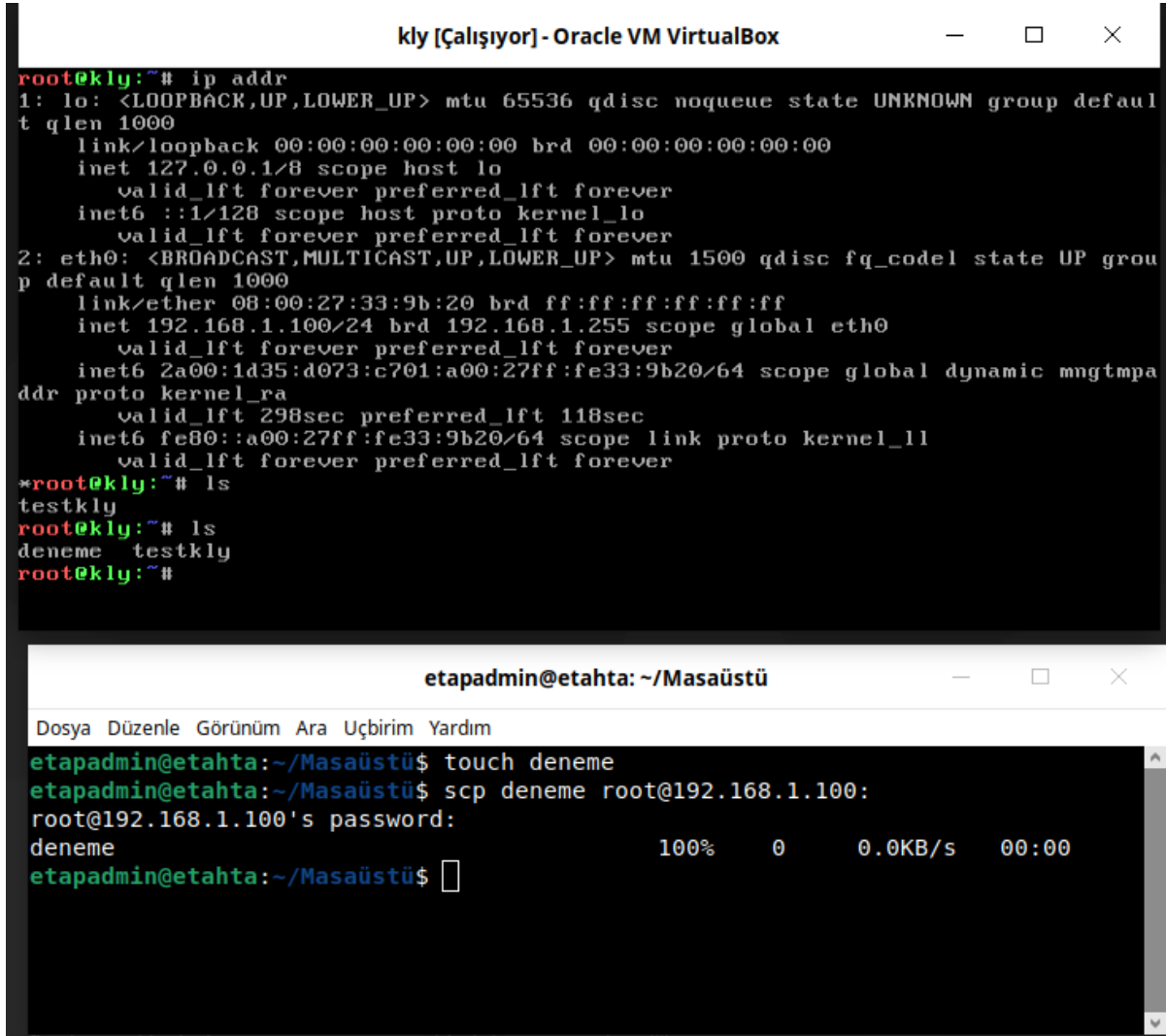




Temel Sisteme scp ile Dosya Kopyalama

scp terminal üzerinden uzak bilgisayarlara dosya kopyalaması yapan bir uygulamadır. **Temel Sistem** içerisinde **ssh** paketinin bir parçası olarak gelmektedir. scp kullanımı **Yardımcı Konular** bölümünde anlatılmıştır.

Aşağıda **kly Temel Sisteme scp** ile **deneme** dosyasının nasıl kopyalandığı görülmektedir.



The image shows two terminal windows. The top window, titled 'kly [Çalışıyor] - Oracle VM VirtualBox', shows the output of the 'ip addr' command for the 'lo' and 'eth0' interfaces. The bottom window, titled 'etapadmin@etahta: ~/Masaüstü', shows the execution of 'touch deneme' and 'scp deneme root@192.168.1.100:' commands, followed by a password prompt and a progress bar for the file transfer.

```
kly [Çalışıyor] - Oracle VM VirtualBox
root@kly:~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:33:9b:20 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 2a00:1d35:d073:c701:a00:27ff:fe33:9b20/64 scope global dynamic mngtmpa
        addr proto kernel_ra
        valid_lft 298sec preferred_lft 118sec
    inet6 fe80::a00:27ff:fe33:9b20/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
*root@kly:~# ls
testkly
root@kly:~# ls
deneme testkly
root@kly:~#

etapadmin@etahta: ~/Masaüstü
Dosya Düzenle Görünüm Ara Uçbirim Yardım
etapadmin@etahta:~/Masaüstü$ touch deneme
etapadmin@etahta:~/Masaüstü$ scp deneme root@192.168.1.100:
root@192.168.1.100's password:
deneme                               100%   0   0.0KB/s   00:00
etapadmin@etahta:~/Masaüstü$
```


kly Paket Sistemi Kullanımı

Paket sistemi, sisteme paket (**kurma, kaldırma, index güncelleme**) gibi temel işlemlerin yapılmasını sağlar. **Temel Sistem** kitabımızda **Paket Sistemi(kly)** tasarımı anlatılmıştı. Bu kitapta ise paketleri derlerken kly paket sistemi kullanılarak derleyeceğiz. Paket sistemi paketleri tekrar tekrar derlemek yerine paket olarak hazırlamakta ve istediğimiz zaman oluşturulan sisteme yüklenmekte veya kaldırılmaktadır. Bir sorun olduğu farkedildiğinde ise tekrar derlenerek sadece bu paket güncelenebilmektedir. Bu tür avantajlardan dolayı paket sistemini kullanacağız.

Burada paket sisteminin nasıl kullanılacağı anlatılacaktır. Paket sistemimiz **Temel Sistem** üzerinde **base-file** paketiyle sisteme yüklendi. Aşağıda **kly Temel Sistem** üzerinde hazır gelen paket sistemi uygulamalarımız görülmektedir.

kly [Çalışıyor] - Oracle VM VirtualBox

```
root@kly:~# ls -l /bin/kly*
-rwxr-xr-x 1 root root 22588 Jul 21 08:15 /bin/kly
-rwxr-xr-x 1 root root 1224 Nov 17 2024 /bin/klykaldir
-rwxr-xr-x 1 root root 1249 Nov 17 2024 /bin/klykur
-rwxr-xr-x 1 root root 2490 Nov 17 2024 /bin/klypaketle
-rwxr-xr-x 1 root root 252 Nov 17 2024 /bin/klyupdate
root@kly:~# kly --help
Usage: kly <options>
-u, --update           : package index update. use: kly -u
-c, --create           : create kly package. use: kly -c      packagedirectory target(default=/)
-i, --install          : package install. use: kly -i        packagename target(default=/)
-ri, --reinstall       : package install. use: kly -ri       packagename target(default=/)
-pi, --packagefile     : packagefile install. use: kly -pi    packagefile target
-r, --remove           : package remove. use: kly -r         packagename target(default=/)
-h, --help             : kly help
```

kly: *klypaketle, klykur, klykadir, klyupdate* scripplerimizin hepsini tek scriptte toplanmış halidir. Ayrıca paketlerin kurulumu ve kaldırılması sırasında bağımlılıklarını da paketle birlikte kurmakta veya kaldırmakta. Bu dokümanda paketlerin derlenmesi, kurulması, kaldırılması vb. işlemler **kly** scripti kullanılarak yapılacaktır. **klypaketle, klykur, klykaldir, klyupdate** scripleri bir öğretici olması nedeniyle bulunmaktadır. Daha kapsamlı işlemler için yetersiz kalacaktır.

kly Paket Sisteminin Debian'a Kurulumu

kly paket sistemi **Temel Sistem** üzerinde kurulu olarak gelmektedir. Üstte verilen resimde görülmektedir. Kullanımı ve parametrelerini unutmanız durumunda **kly --help** komutuyla kullanımı öğrenebilirsiniz.

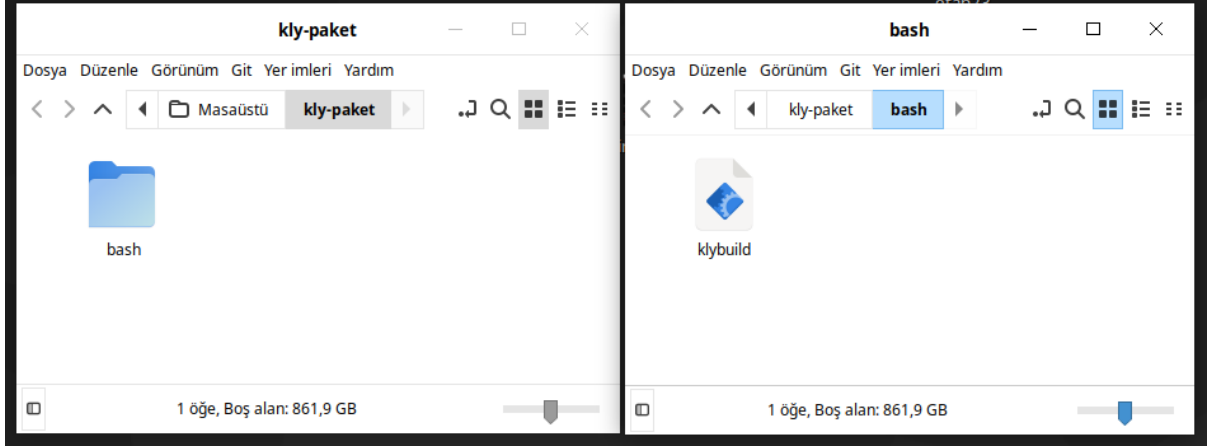
Debian üzerinde kullanmak için kly scriptimizi indirip **/bin** konumuna kopyalayalım. Aşağıda verilen komutları sırasıyla çalıştırınız. **kly** paket sisteminin sorunsuz çalışması için **busybox** paketinin kurulu olması gerekmektedir. Sorunsuz çalışmışsa **Debian** sistemimize **kly** paket sistemimiz kurulmuştur.

```
wget https://kendilinuxunuyap.github.io/_static/files/base-file/kly
sudo mv kly /bin/kly
sudo chown root:root /bin/kly
sudo chmod 755 /bin/kly
sudo apt update
sudo apt install busybox-static -y
```

kly ile Paket Yapma

kly paket sistemi ile paket yapma işlemini Debian ortamında yapacağız. Debian üzerinde paket sistemimizi oluşturan scriptimiz **/bin/kly** konumunda olması gerekmektedir.

Şimdi **bash** paketinin **kly Paket Sistemi**'ni kullanarak derlemesini yapalım. Paket için aşağıda görüldüğü gibi masaüstüne **kly-paket** dizini oluşturuldu. **kly-paket** dizini içine **bash** dizini oluşturuldu. **bash** dizini içine **klybuild** dosyası oluşturuldu.

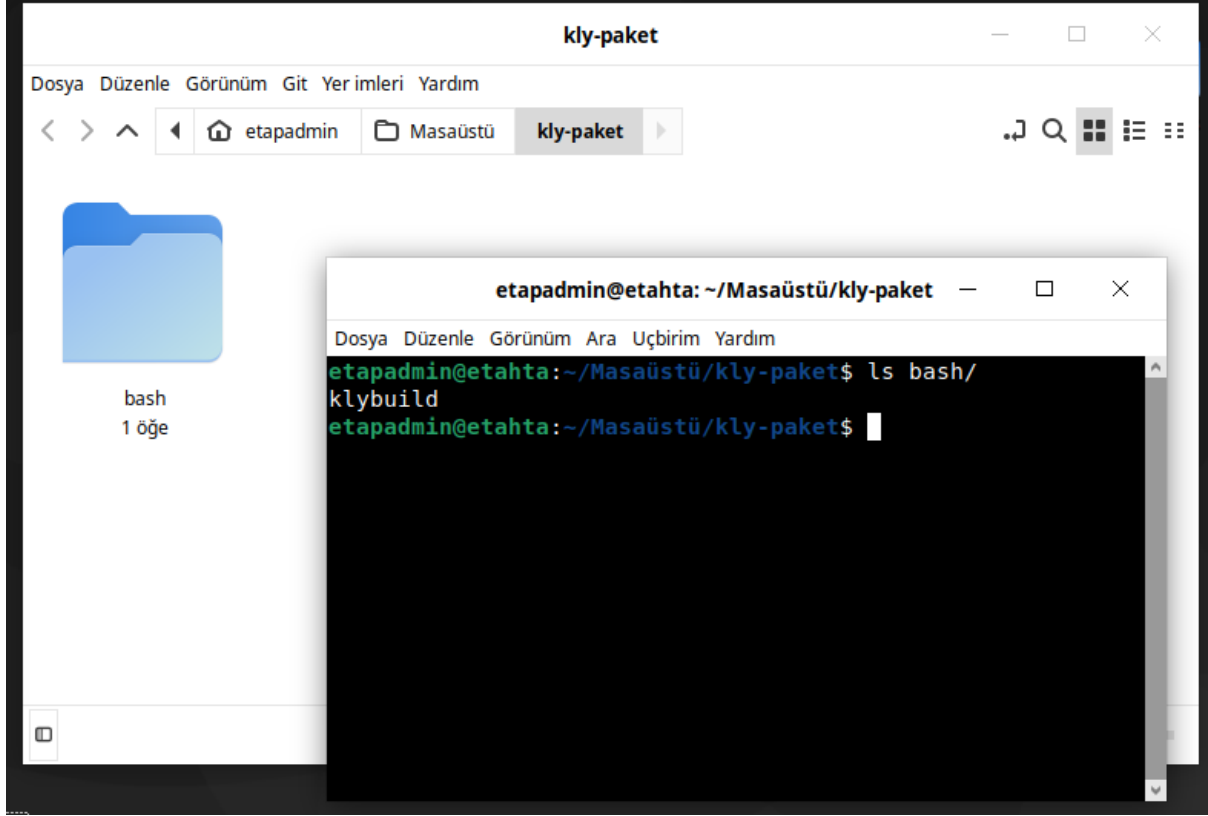


klybuild dosyasının içerisine aşağıdaki kodu ekleyiniz.

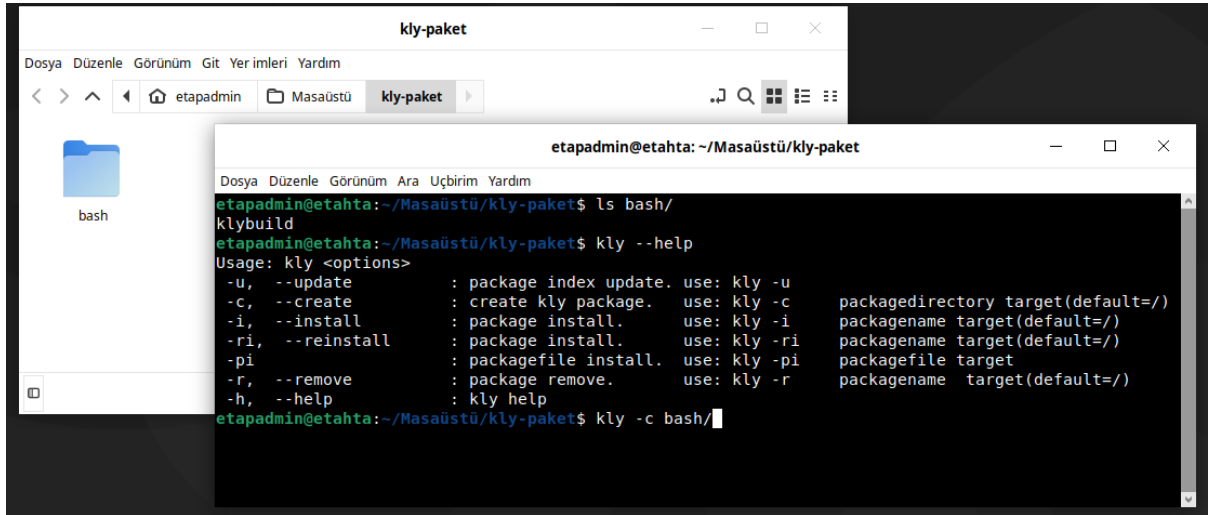
```
#!/usr/bin/env bash
version="5.2.21"
name="bash"
depends="glibc, readline, ncurses"
description="GNU/Linux dağıtımında ön tanımlı kabuk"
source="https://ftp.gnu.org/pub/gnu/bash/${name}-${version}.tar.gz"
groups="app.shell"
setup() {
    cd $SOURCEDIR
    ./configure --prefix=/usr --libdir=/usr/lib64 --bindir=/bin \
        --with-curses --enable-readline --without-bash-malloc
}
build() {
    make
}
package() {
    make install DESTDIR=$DESTDIR
    cd $DESTDIR/bin
    ln -s bash sh
}
```

Not: Bazı paketlerde ek dosyalar olacaktır. Bu paketlerin derleme scriplerinin altında **Ek dosya için tıklayınız** bağlantı(link) göreceksiniz. Bu bağlantıdan ek dosyaları indirin ve paketin dizini içine sıkıştırılmış dosyayı arşivde çıkartınız. Burada örneğimiz **bash** paketi olduğu için eğer ek dosyaları olsaydı **bash** dizini içine indiğimiz dosyayı arşivde çıkartacaktık(klybuil dosyasıyla aynı konumda olacak).

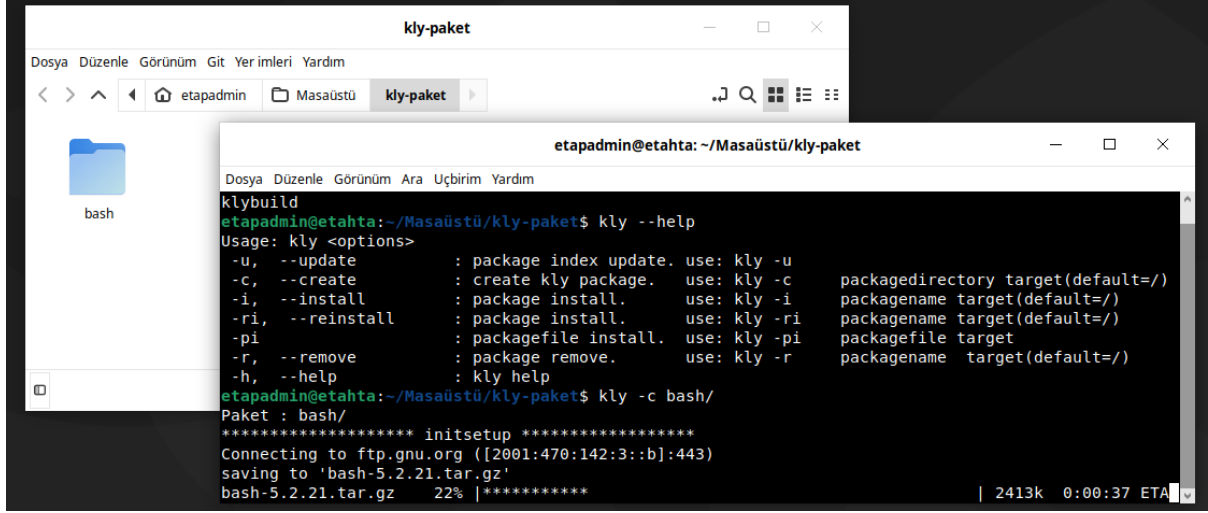
kly-paket dizini konumunda aşağıdaki gibi terminal açınız.



Açılan terminalde aşağıda görüldüğü gibi komutu çalıştırınız. komut hangi konumda çalıştırılmışsa **.kly** uzantılı pakeyimiz orada oluşacaktır. Siz istediğiniz yerde çalıştırabilirsiniz. Önemli olan paket için oluşturduğunuz **klybuild** dosyanızın konumunu doğru vermeniz.



Aşağıda **bash** dizinini parametre olarak vererek **kly-paket** paketimizin derlemesini başlatıyoruz.



The image shows a file manager window titled 'kly-paket' with a sidebar showing a folder named 'bash'. Overlaid on this is a terminal window titled 'etapadmin@etahta: ~/Masaüstü/kly-paket'. The terminal shows the command 'kly --help' and its output, which lists various options for the 'kly' command. Below this, the command 'kly -c bash/' is entered, and the terminal shows the start of the build process, including 'initsetup' and 'Connecting to ftp.gnu.org'.

```
klybuild
etapadmin@etahta:~/Masaüstü/kly-paket$ kly --help
Usage: kly <options>
-u, --update           : package index update. use: kly -u
-c, --create           : create kly package. use: kly -c
-i, --install          : package install. use: kly -i
-ri, --reinstall       : package install. use: kly -ri
-pi, --packagefile install. use: kly -pi
-r, --remove           : package remove. use: kly -r
-h, --help             : kly help
etapadmin@etahta:~/Masaüstü/kly-paket$ kly -c bash/
Paket : bash/
***** initsetup *****
Connecting to ftp.gnu.org ([2001:470:142:3::b]:443)
saving to 'bash-5.2.21.tar.gz'
bash-5.2.21.tar.gz 22% |*****
```

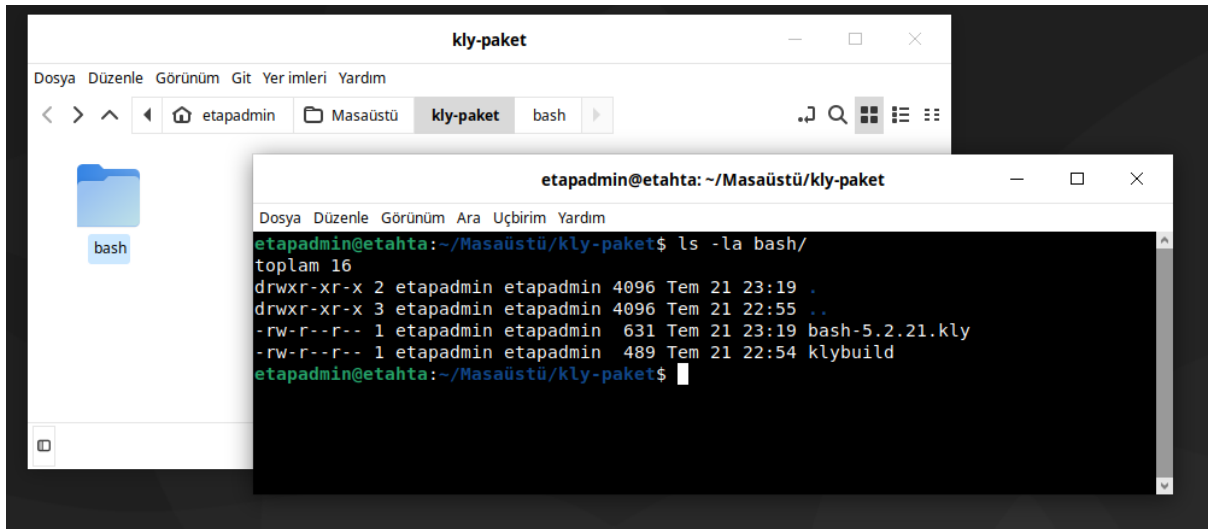
Derleme işlemi paketin büyüklüğüne bağlı olarak zaman alacaktır. Paket derlemesi bittikten sonra aşağıda görüldüğü gibi terminale çıktısı almalısınız. Sorun çıkması durumunda terminalde hata mesajları alırsınız.



The image shows a terminal window titled 'etapadmin@etahta: ~/Masaüstü/kly-paket'. The terminal displays the output of the build process, including 'getconf', 'make[1]: Leaving directory', 'packageindex', 'packagecompress', 'rootfs.tar.xz', 'file.index', 'klybuild', 'postinstall', 'postremove', and various directory paths.

```
etapadmin@etahta: ~/Masaüstü/kly-paket
Dosya Düzenle Görünüm Ara Uçbirim Yardım
getconf
make[1]: Leaving directory '/tmp/kly/build/bash-5.2.21/examples/loadables'
***** packageindex*****
*****packagecompress*****
rootfs.tar.xz
file.index
klybuild
postinstall
postremove
PACKAGEDIR: ///home/etapadmin/Masaüstü/kly-paket/bash/
DESTDIR: ///tmp/kly/build/rootfs-bash-5.2.21
SOURCEDIR: ///tmp/kly/build/bash-5.2.21
BUILDDIR: ///tmp/kly/build/build-bash-5.2.21
etapadmin@etahta:~/Masaüstü/kly-paket$
```

Derleme işlemi bittikten sonra **kly-paket/bash** dizini komusunda aşağıda görüldüğü gibi **bash-5.2.21.kly** paketimizi oluşturacaktır.



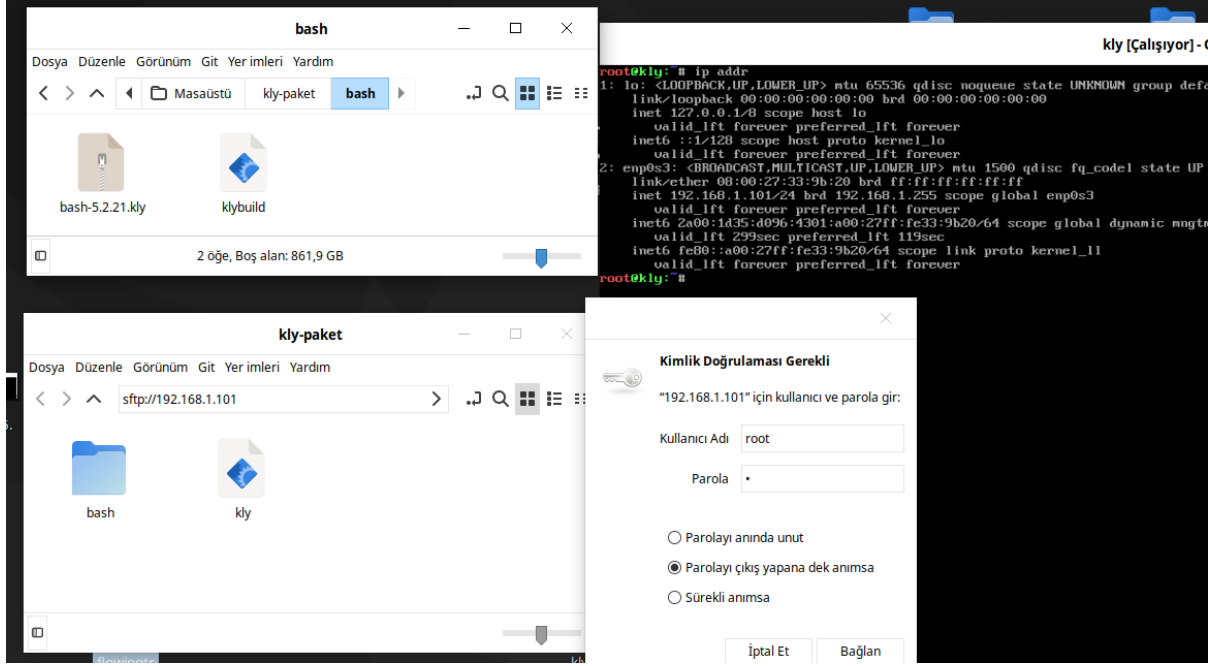
The image shows a file manager window titled 'kly-paket' with a sidebar showing a folder named 'bash'. Overlaid on this is a terminal window titled 'etapadmin@etahta: ~/Masaüstü/kly-paket'. The terminal displays the output of the 'ls -la bash/' command, showing the contents of the 'bash' directory, including 'bash-5.2.21.kly' and 'klybuild'.

```
etapadmin@etahta:~/Masaüstü/kly-paket$ ls -la bash/
toplam 16
drwxr-xr-x 2 etapadmin etapadmin 4096 Tem 21 23:19 .
drwxr-xr-x 3 etapadmin etapadmin 4096 Tem 21 22:55 ..
-rw-r--r-- 1 etapadmin etapadmin 631 Tem 21 23:19 bash-5.2.21.kly
-rw-r--r-- 1 etapadmin etapadmin 489 Tem 21 22:54 klybuild
etapadmin@etahta:~/Masaüstü/kly-paket$
```

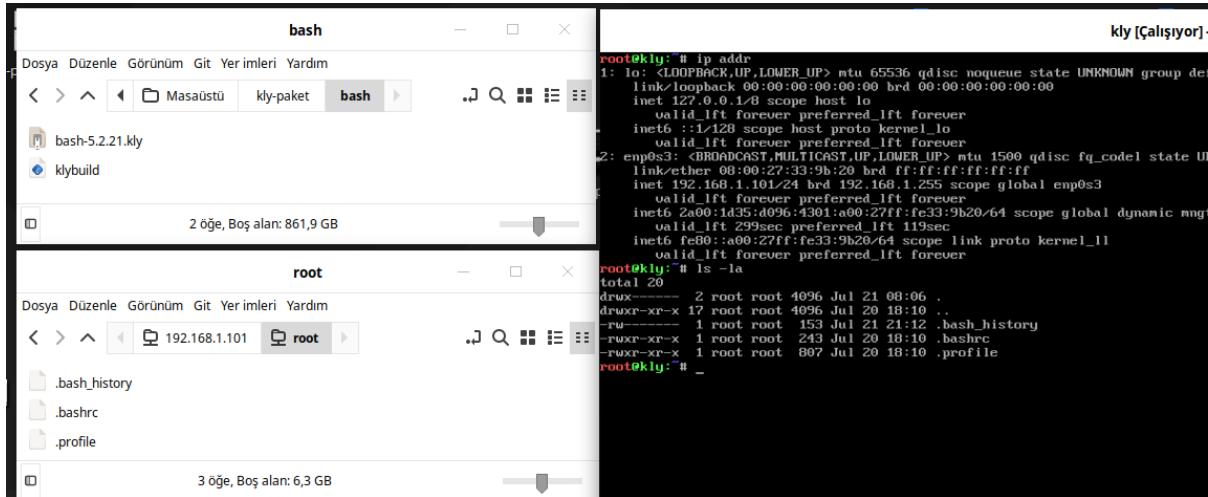
kly ile Paket Kur

kly Paket Sistemi ile paket oluşturma konusunda **bash** paketi derlenmişti. Şimdi ise bu paketi **Temel Sistem** üzerine kopyalamayı ve kurma işlemini yapalım. **bash** paketimiz masaüstümüzde **kly-paket/bash/bash-5.2.21.kly** konumunda bulunmaktadır. Bu konumdaki dosyamızı **Temel Sistem** üzerine **scp** veya **sftp** kullanarak kopyalayabiliriz. **scp** terminal üzerinden kopyalar. **sftp** terminal veya pencere yöneticisi üzerinden kopyalar. Burada tercih size kalmıştır. **sftp** ile pencere yöneticisini kullanmak daha kolay olabilir. **scp ve sftp** kullanımı **Yardımcı Konular** bölümünde detaylıca anlatılmıştır. Ayrıca **Temel Sistemin Hazırlanması** bölümünde de **scp** ve **sftp** **Temel Sistem** ile nasıl kullanılacağı anlatıldı.

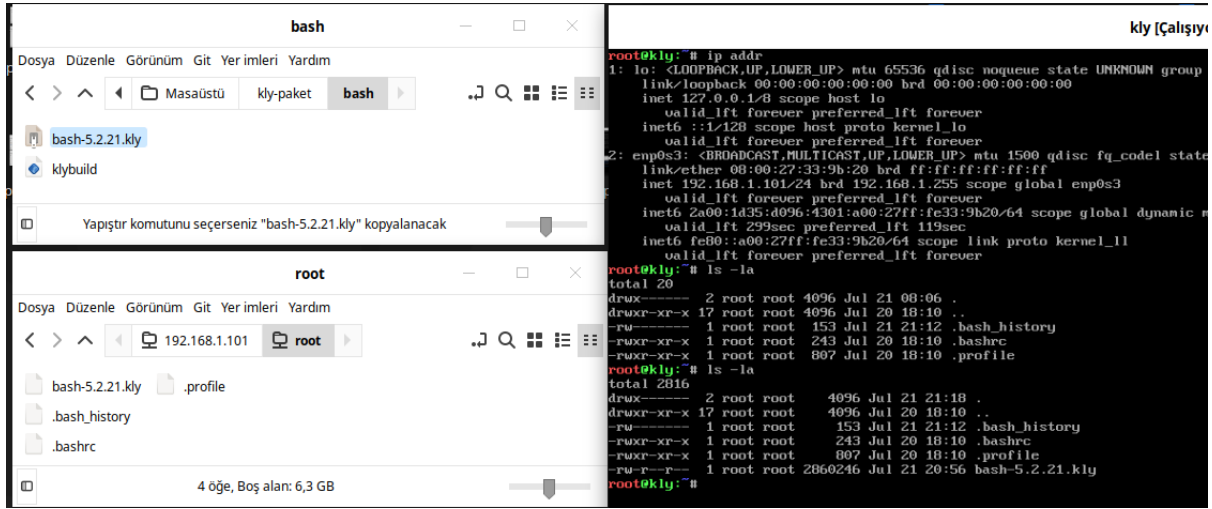
Aşağıda **sftp** ile nasıl bağlantı yapılacağı görülmektedir.



Aşağıda **sftp** ile **bash-5.2.21.kly** paketini kopyalanma öncesi **Temel Sistemde** olup olmadığını **ls -la** komutuyla kontrol ediyoruz.



Kopyalama işleminden sonra **bash-5.2.21.kly** paketinin **Temel Sistemde** olup olmadığını **ls -la** komutuyla görüyoruz.

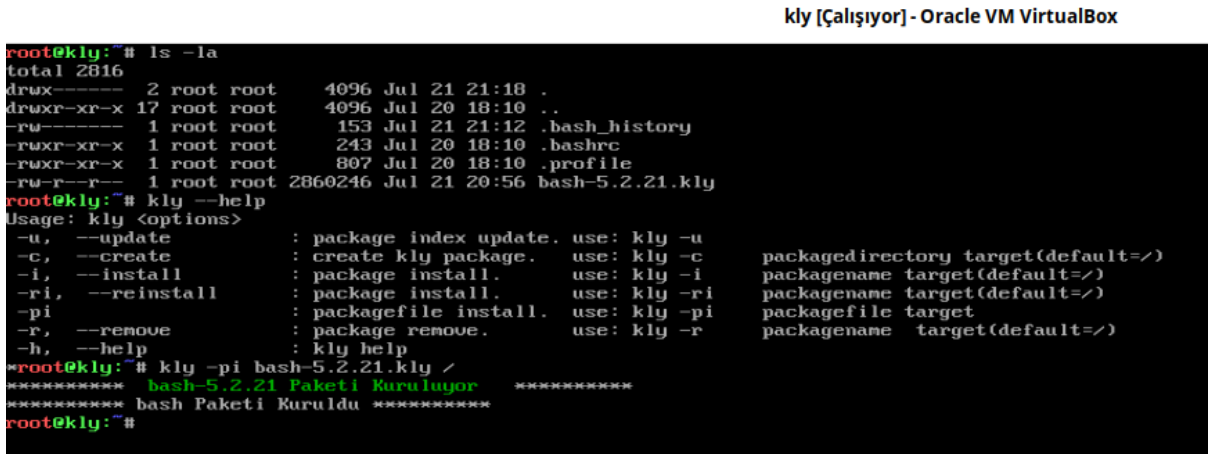


```
root@kly:~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state
    link/ether 08:00:27:33:9b:20 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.101/24 brd 192.168.1.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 2a00:1d35:d096:4301:a00:27ff:fe33:9b20/64 scope global dynamic m
        valid_lft 299sec preferred_lft 119sec
    inet6 fe80::a00:27ff:fe33:9b20/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever

root@kly:~# ls -la
total 20
drwxr-xr-x 2 root root 4096 Jul 21 08:06 .
drwxr-xr-x 17 root root 4096 Jul 20 18:10 ..
-rw-r--r-- 1 root root 153 Jul 21 21:12 .bash_history
-rwxr-xr-x 1 root root 243 Jul 20 18:10 .bashrc
-rwxr-xr-x 1 root root 807 Jul 20 18:10 .profile
root@kly:~# ls -la
total 2816
drwxr-xr-x 2 root root 4096 Jul 21 21:18 .
drwxr-xr-x 17 root root 4096 Jul 20 18:10 ..
-rw-r--r-- 1 root root 153 Jul 21 21:12 .bash_history
-rwxr-xr-x 1 root root 243 Jul 20 18:10 .bashrc
-rwxr-xr-x 1 root root 807 Jul 20 18:10 .profile
-rw-r--r-- 1 root root 2860246 Jul 21 20:56 bash-5.2.21.kly
root@kly:~#
```

kly paket sistemini kullanarak yerelde bulunan paketin kurulumunu **kly -pi bash-5.2.21.kly /** komutuyla yapıyoruz.

kly [Çalışıyor] - Oracle VM VirtualBox



```
root@kly:~# ls -la
total 2816
drwxr-xr-x 2 root root 4096 Jul 21 21:18 .
drwxr-xr-x 17 root root 4096 Jul 20 18:10 ..
-rw-r--r-- 1 root root 153 Jul 21 21:12 .bash_history
-rwxr-xr-x 1 root root 243 Jul 20 18:10 .bashrc
-rwxr-xr-x 1 root root 807 Jul 20 18:10 .profile
-rw-r--r-- 1 root root 2860246 Jul 21 20:56 bash-5.2.21.kly

root@kly:~# kly --help
Usage: kly <options>
-u, --update           : package index update. use: kly -u
-c, --create           : create kly package. use: kly -c
-i, --install          : package install. use: kly -i
-ri, --reinstall       : package install. use: kly -ri
-pi, --packagefile install. use: kly -pi
-r, --remove           : package remove. use: kly -r
-h, --help             : kly help
                        packagedirectory target(default=/)
                        packagename target(default=/)
                        packagename target(default=/)
                        packagefile target
                        packagename target(default=/)

root@kly:~# kly -pi bash-5.2.21.kly /
***** bash-5.2.21 Paketi Kuruluyor *****
***** bash Paketi Kuruldu *****
root@kly:~#
```

kly ile Paket Kaldır

kly Paket Sistemiyle sistemde yüklü olan bir paketi **kly -r paketadı** şeklinde komut kullanılarak kaldırılabilir. Daha önce paket oluşturma ve paket kurulum işlemlerinde **bash** paketini kullanmıştık. Şimdi **bash** paketini kaldırmamız durumunda sistemde terminali kullanamaz duruma getirecektir. Bazı temel paketleri kaldırmak sistemimizin bozulmasına sebep olabilir. Paket kaldırırken dikkatli olmak gerekir.

Aşağıda **bash** paketinin nasıl kaldırılacağı görülmektedir.

```
root@kly:~# kly --help
Usage: kly <options>
-u, --update           : package index update. use: kly -u
-c, --create           : create kly package. use: kly -c    packagedirectory target(default=/)
-i, --install          : package install. use: kly -i      packagename target(default=/)
-ri, --reinstall       : package install. use: kly -ri     packagename target(default=/)
-pi, --packagefile     : packagefile install. use: kly -pi  packagefile target
-r, --remove           : package remove. use: kly -r      packagename target(default=/)
-h, --help             : kly help
```

```
root@kly:~# kly -r bash
```

Bazı paketleri (**glibc**, **ncurses**, **readline**, **bash**) tasarladığımız **kly** paket sisteminde kaldırılmasını engelledik. Bu paketler kalması durumunda sistem kullanılamaz hale gelir. Eğer değişiklik gerekiyorsa sadece yeniden kurulum yapılabilir. Aşağıda paketin kaldırılmadığı mesajı görülmektedir.

```
root@kly:~# kly --help
Usage: kly <options>
-u, --update           : package index update. use: kly -u
-c, --create           : create kly package. use: kly -c    packagedirectory target(default=/)
-i, --install          : package install. use: kly -i      packagename target(default=/)
-ri, --reinstall       : package install. use: kly -ri     packagename target(default=/)
-pi, --packagefile     : packagefile install. use: kly -pi  packagefile target
-r, --remove           : package remove. use: kly -r      packagename target(default=/)
-h, --help             : kly help
```

```
root@kly:~# kly -r bash
```

```
bash Paketi Sistemin Temel Paketidir. Silinemez.
```

```
root@kly:~# _
```

glibc, **ncurses**, **readline**, **bash** dışında başka paketler olsaydı paket kaldırılması gerçekleşecekti.

kly Paketlerini Githuba Yükleme ve İndexleme

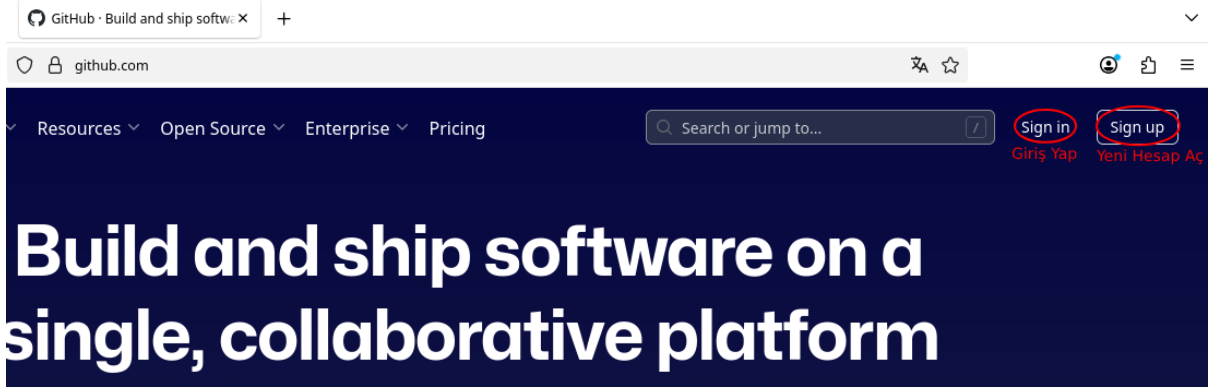
Depo, paketlerimizin olduğu alandır. Depoda ne kadar paket varsa bunların isimleri sürüm numaraları gibi bilgiler ile adreslerini liste halinde oluşturma işlemine **depo indexleme** denir. Depo indexlenirken genellikle bilgiler **paket derleme talimatı(klybuild)** dosyasından alınır. Paketlerin listesi, paketler kurulurken, silinirken ve güncellenirken kullanılmaktadır.

kly github Depo Yapma

Bu doküman kullanılarak hazırlanan paketleri bilgisayarınızda bir dizinde tutabiliriz. Fakat bu çok kısıtlı bir sistem olmasına sebep olacaktır. Paketleri bir internet ortamında bir yerde saklayarak, kurmak istediğimizde internet(uzak) üzerinden kurulması daha doğru bir yöntemdir. Bu dağıtımda paketlerimizi github.com üzerinde oluşturulan bir repository üzerinden çekilmektedir. İnternetteki paketlerimizin listesi her yeni paketi yükleme sırasında güncellenmektedir. Bu işlem github hesabı üzerinden yapılmaktadır. github hakkında temel işlemler için github konusunu okuyunuz.

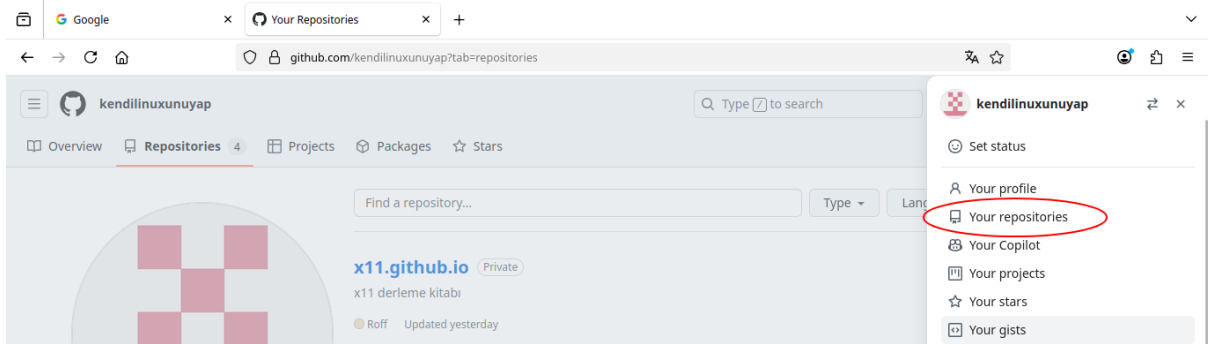
github Sitesi Açılır

Singup seçeneği seçilerek bir hesap oluşturulur. Biz bu dokumanda kullandığımız kendilinuxunuyap hesabını açtık.



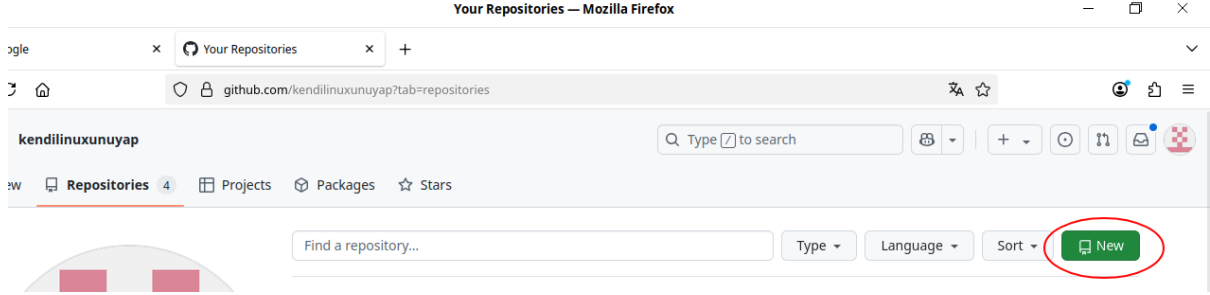
github Üzerinden Hesaba Giriş Yapılır

github hesabı açılır(kendilinuxunuyap) ve sağ tarafta bulunan menüden **Your repostrores** seçilir.

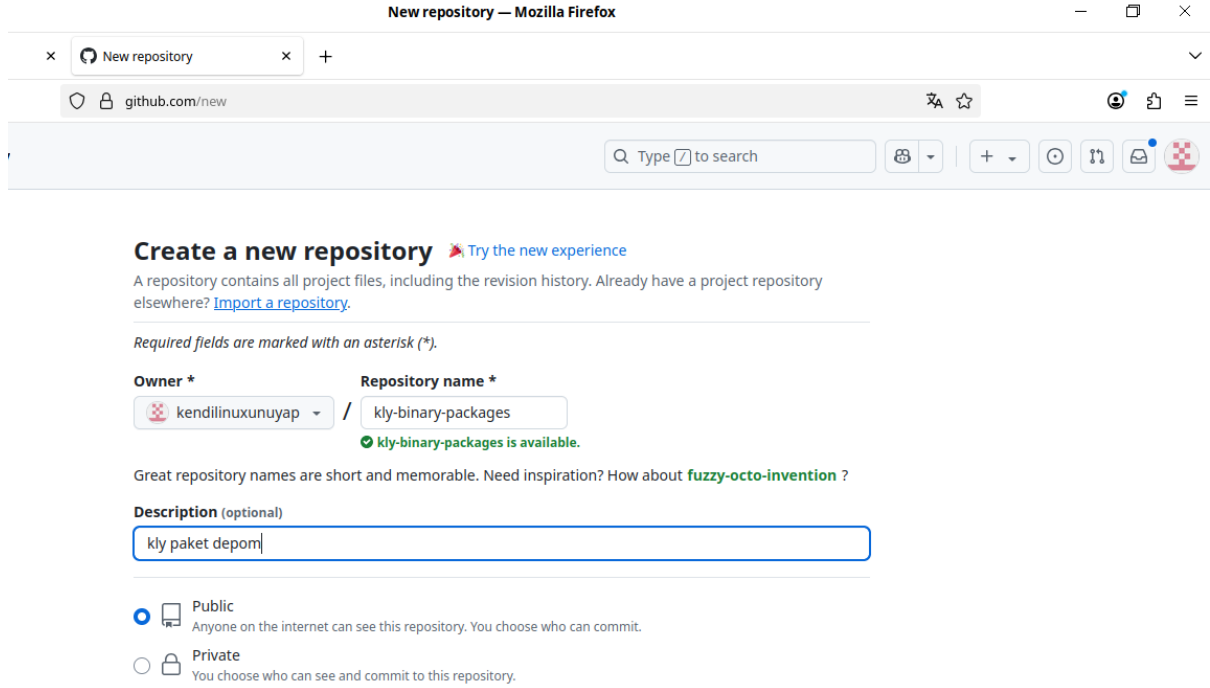


Yeni Depo Alanı Oluşturulur

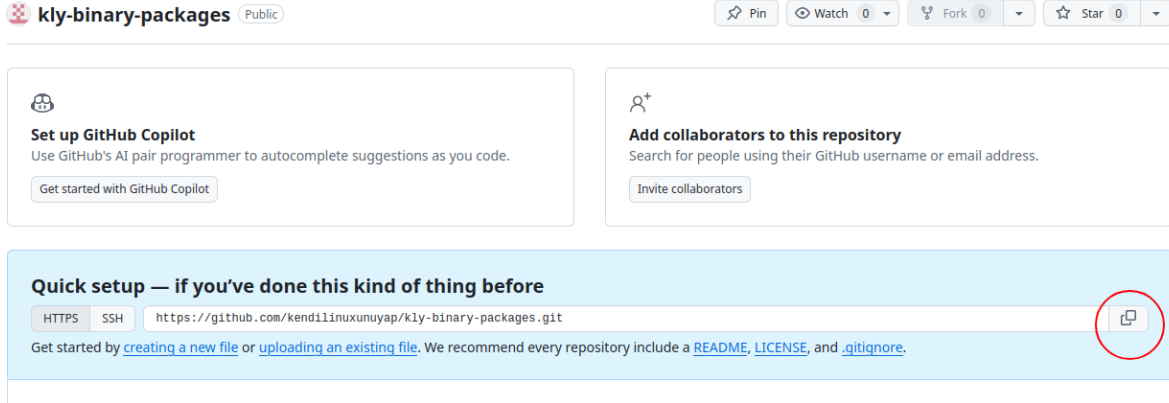
Karşımıza gelen ekrandan **New** Seçeneği seçilir.



github repository oluşturulur(kly-binary-packages)



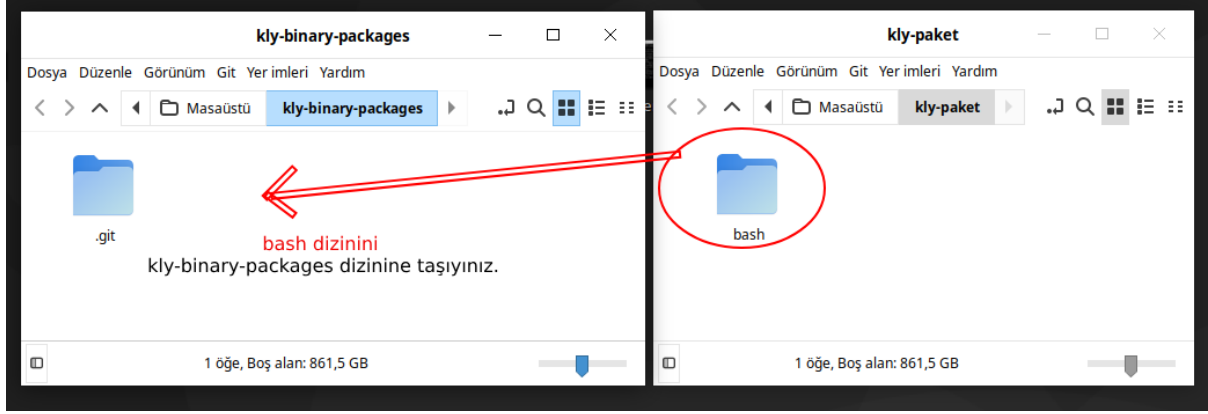
kly-binary-packages Depomuz Yerele(Bilgisayara) Kopyalanır(Clone) kly-binary-packages adresi kopayanır.



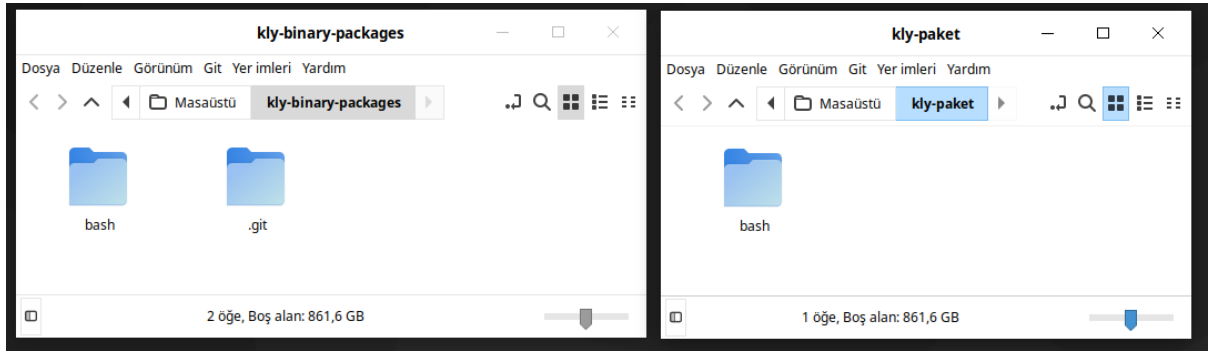
Yerelde(bilgisayarda) istediğiniz yere(masaüstünü tercih ettim) indirilir(klonlanır/download).



Paketimiz kly-binary-packages Dizinine Kopyalanır

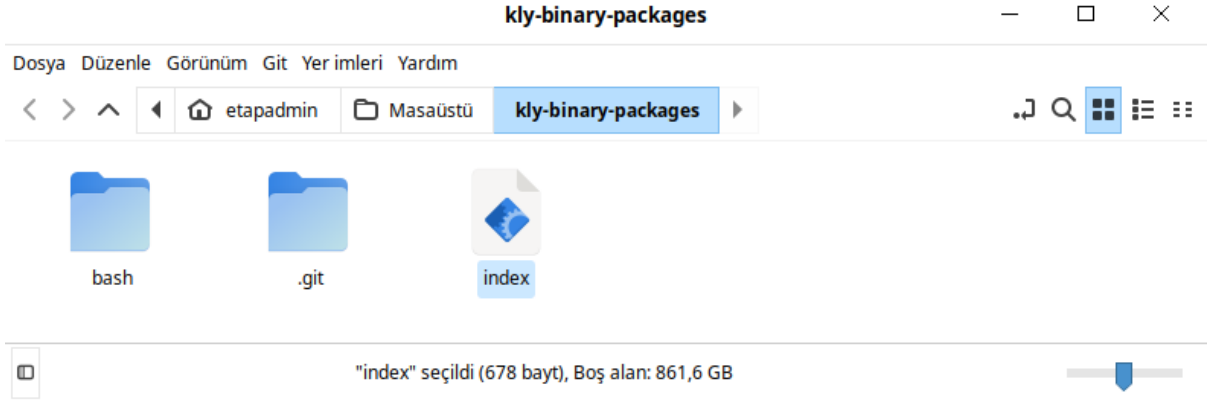


bash paketi aşağıdaki gibi taşınır.



index Dosyası Oluşturulur

Aşağıdaki script kly paket dosyalarımızı olduğu dizinde tek tek açarak içerisinde **klybuild** dosyalarını çıkartır. Paketle ilgili bilgileri alıp **index.lst** dosyası oluşturmaktadır. İstersek paketler local ortamdada index oluşturabiliriz. Bu dokümanda github üzerinde oluşturacak şekilde anlatılmıştır. Paket indeksi oluşturan **index.lst** dosyası aşağıdaki gibi olacaktır. Listede name, version ve depends(bağımlı olduğu paketler) bilgileri bulunmaktadır. Bilgilerin arasında | karakteri kullanılmıştır.



Yukarıda gösterildiği gibi **kly-binary-packages** dizininde aşağıda verilen **index** dosyasını oluşturunuz. Aşağıdaki script kodlarını **index** dosyasının içerisine ekleyin.

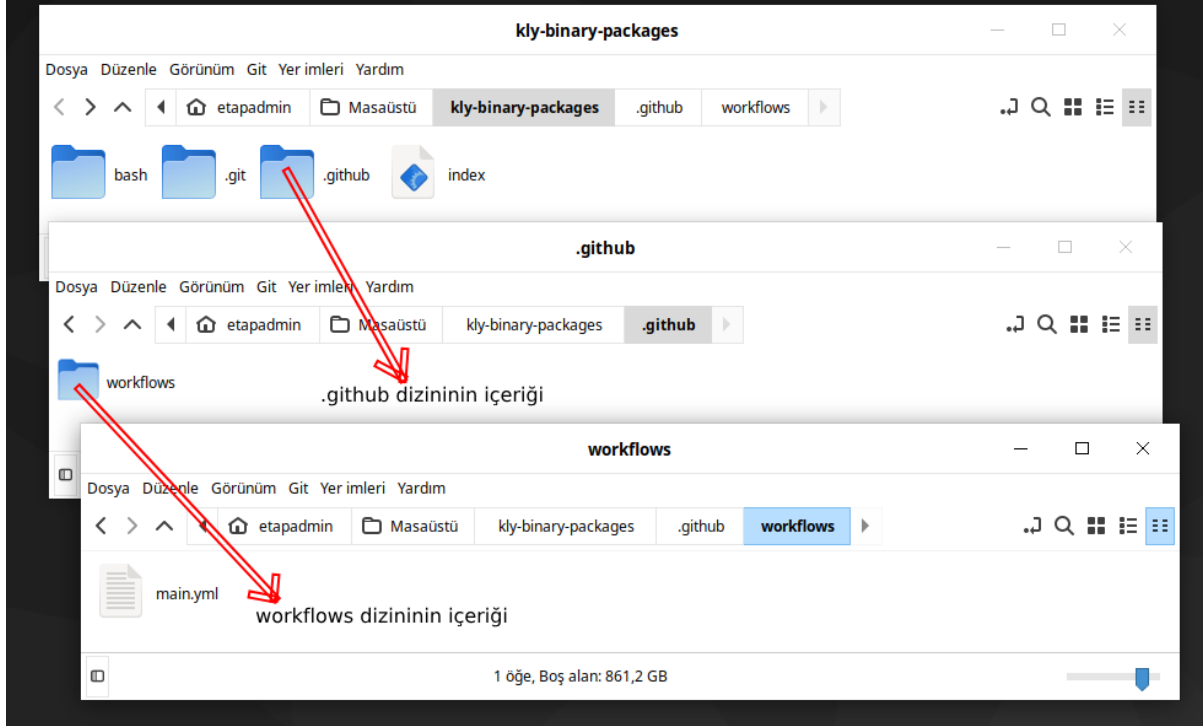
```
#-----
#!/bin/sh
#set -ex
mkdir /output -p
mkdir -p /klysource
>index.lst
find * -type f -name *.kly |
    while IFS= read file_name; do
        dosya="$(dirname $file_name)/klybuild"
        version=$(cat $dosya|grep version=)
        name=$(cat $dosya|grep name=)
        depends=$(cat $dosya|grep depends=)
        echo "$name|$version|$depends|$(dirname $file_name)">>index.lst
    done
cp -rf index.lst /output

# *****source files*****
cp -prfv ./ /klysource/

find /klysource/* -type f -name *.kly |
    while IFS= read file_name; do
        rm -rf "$file_name"
    done
tar -cf /output/klysourcepackage.tar /klysource/
rm -rf /klysource
```

main.yml Dosyası Oluşturulur

github'a dosya gönderdiğimizde **index** bash scriptimizi çalıştırması için aşağıda gösterilen şekilde **kly-binary-packages** dizinine **.github/workflows** dizinini oluşturun. **.github/workflows** dizini içine **main.yml** dosyasını oluşturunuz.



main.yml dosyasındaki **sh index** satırı **index** scriptimizi her githuba paket gönderdiğimizde(commit) çalışacak ve **index.lst** dosyasını oluşturacaktır. **main.yml** içeriğine aşağıdaki kodları ekleyiniz.

```
#-----
name: CI

on:
  push:
    branches: [ master ]
  schedule:
    - cron: "0 0 1 2 6"

jobs:
  compile:
    name: depoindex
    runs-on: ubuntu-latest
    steps:
      - name: Check out the repo
        uses: actions/checkout@v2
      - name: Run the build process with Docker
        uses: addnab/docker-run-action@v3
        with:
          image: debian:testing
          options: -v ${ github.workspace }:/root -v /output:/output
          run: |
            cd /root
            sh index
      - uses: "marvinpinto/action-automatic-releases@latest"
        with:
          repo_token: "${ secrets.GITHUB_TOKEN }"
          automatic_release_tag: "current"
          prerelease: false
          title: "Latest release"
          files: |
            /output/*
```

github Dosya oluşturma izni Verin

İnternet üzerinden **kly-binary-packages** reposunda settings->action->general->Workflow permissions->Read and write permissions işaretlenmelidir.

Approval for running fork pull request workflows from contributors

Choose which subset of users will require approval before running workflows on their pull requests. Both the pull request author and the actor of the pull request event triggering the workflow will be checked to determine if approval is required. If approval is required, a user with write access to the repository must [approve the pull request workflow to be run](#).

☐ Require approval for first-time contributors who are new to GitHub

Only users who are both new on GitHub and who have never had a commit or pull request merged into this repository will require approval to run workflows.

☒ Require approval for first-time contributors

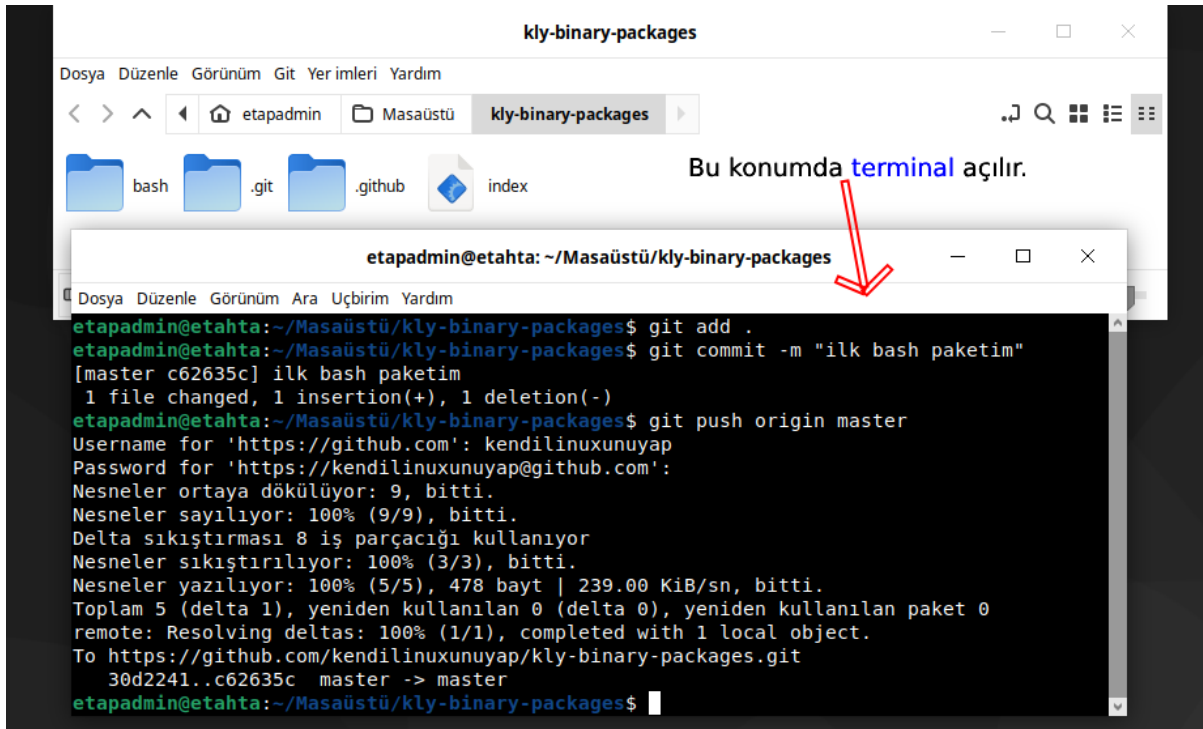
Only users who have never had a commit or pull request merged into this repository will require approval to run workflows.

☐ Require approval for all external contributors

All users that are not a member or owner of this repository will require approval to run workflows.

github'a kly-binary-packages Dizinini Yükleyelim(Uplod/Commit)

Yerelde **kly-binary-packages** dizini içeriğini github üzerine aşağıdaki gibi gönderilir.



The screenshot shows a terminal window titled "kly-binary-packages" with the following content:

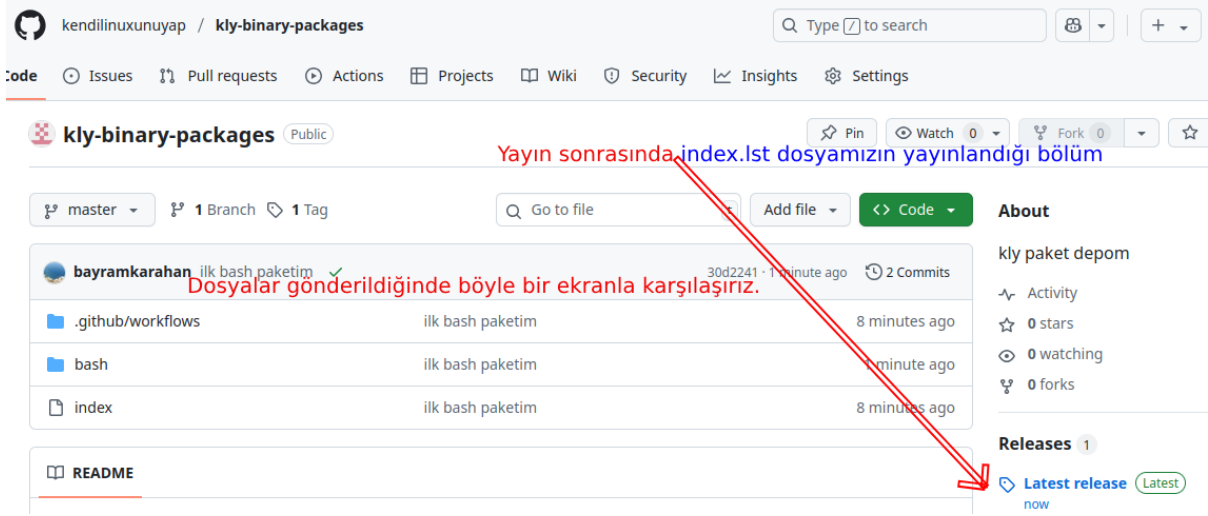
```
Dosya Düzenle Görünüm Git Yer imleri Yardım
< > ^ etapadmin Masaüstü kly-binary-packages
bash .git .github index

Bu konumda terminal açılır.

etapadmin@etahta: ~/Masaüstü/kly-binary-packages
Dosya Düzenle Görünüm Ara Uçbirim Yardım
etapadmin@etahta:~/Masaüstü/kly-binary-packages$ git add .
etapadmin@etahta:~/Masaüstü/kly-binary-packages$ git commit -m "ilk bash paketim"
[master c62635c] ilk bash paketim
1 file changed, 1 insertion(+), 1 deletion(-)
etapadmin@etahta:~/Masaüstü/kly-binary-packages$ git push origin master
Username for 'https://github.com': kendilinuxunuyap
Password for 'https://kendilinuxunuyap@github.com':
Nesneler ortaya dökülüyor: 9, bitti.
Nesneler sayılıyor: 100% (9/9), bitti.
Delta sıkıştırması 8 iş parçacığı kullanıyor
Nesneler sıkıştırılıyor: 100% (3/3), bitti.
Nesneler yazılıyor: 100% (5/5), 478 bayt | 239.00 KiB/sn, bitti.
Toplam 5 (delta 1), yeniden kullanılan 0 (delta 0), yeniden kullanılan paket 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/kendilinuxunuyap/kly-binary-packages.git
30d2241..c62635c master -> master
etapadmin@etahta:~/Masaüstü/kly-binary-packages$
```

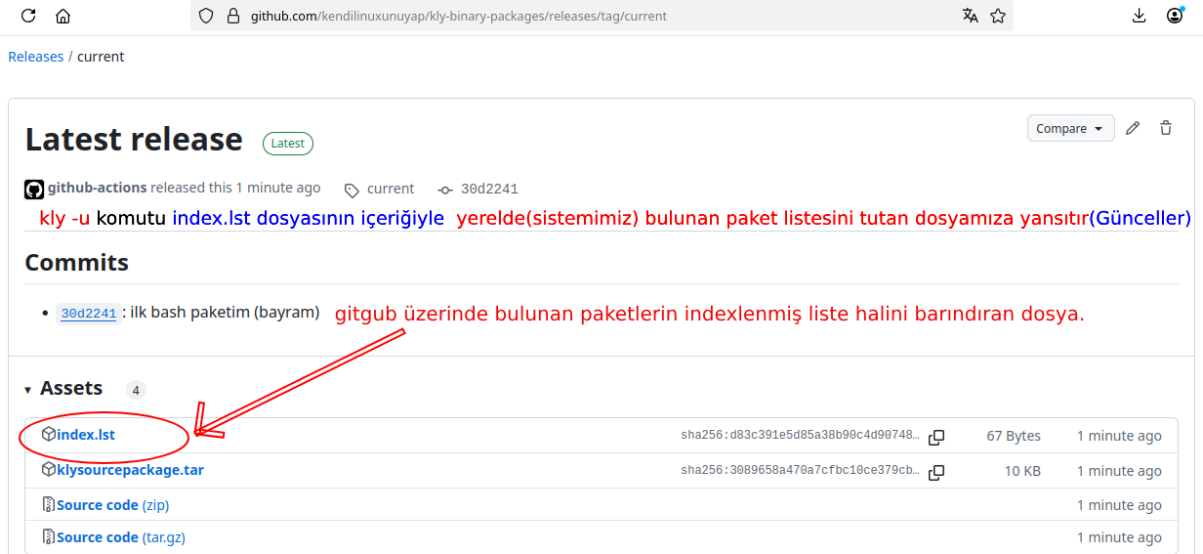
github kly-binary-packages Depo Kontrolü

Depomuza gönderdikten sonra aşağıdaki gibi gözükecektir.



index.lst içeriği

<https://github.com/kendilinuxunuyap/kly-binary-packages/releases/download/current/index.lst> adresinde bulunan dosya aşağıdaki gibi liste oluşturacaktır.



index.lst içeriği aşağıdaki gibidir. Tek paket olduğu için(sadece bash) bu şekilde gözükyor.

```
name="bash" | version="5.2.21" | depends="glibc, readline, ncurses" | bash
```

Birden fazla paketin olması durumunda aşağıdaki gibi gözükecektir.

```
name="acl" | version="2.3.1" | depends="attr" | acl
name="attr" | version="2.5.1" | depends="" | attr
name="audit" | version="3.1.1" | depends="" | audit
name="bash" | version="5.2.21" | depends="glibc, readline, ncurses" | bash
```

kly ile Paket Listelerini Güncelleme

kly Paket Sistemi yerelde **.kly** paketlerini **kly -pi paket.kly** şeklinde kurulabilir. Ama bütün paketlerin yerelde olması bu sistemi kullanacak kişi sayısını sınırlandıracak ve birkaç kişiyi geçmeyecektir. Paketleri internet ortamında tutmak sistemi kullanacak kişilerin sayısını artıracak ve istenilen zaman ve konumda paketlere erişim imkanı sunacaktır.

kly paketleri github üzerinde tutulmaktadır. Bu paketlerin listesini tutan index dosyası github ortamında <https://github.com/kendilinuxunuyap/kly-binary-packages/releases/download/current/index.lst> adresinde tutulmaktadır. Bu adresi sisteme kaydetmeliyizki güncelleme sırasında bu adreslerden güncel **index.lst** dosyasını yerele indirebilmeli. Adreslerin listesini tutan dosyamız **/etc/kly/sources.list** konumunda tutulmaktadır. Debianda da benzer(/etc/apt/sources.list) durum bulunmaktadır.

```
kly [Çalışıyor] - Oracle VM VirtualBox
Dosya Makine Görünüm Giriş Aygıtlar Yardım
root@kly:~# cat /etc/kly/sources.list
kendilinuxunuyap/kly-binary-packages
root@kly:~#
```

Yerelde **(Temel Sistem)** ise **/var/lib/kly/index.lst** dosyamızda paketlerin listesi tutulur. Mevcut sisteme yeni bir paket yaptık ve github'a yüklediğimizi varsayalım. Bu durumda yerelde **(Temel Sistem)** ise **/var/lib/kly/index.lst** konumundaki dosyanın <https://github.com/kendilinuxunuyap/kly-binary-packages/releases/download/current/index.lst> adresindeki dosya ile aynı olması gerekir. Bu senaryo tüm linux dağıtımlarında aynıdır. Bu sebeplerden dolayı bir paket kurulum öncesi genelde **güncelleme** işlemi yapılır. Aşağıda güncelleme işleminin yapıldığını göstermektedir.

```
kly [Çalışıyor] - Oracle VM VirtualBox
Dosya Makine Görünüm Giriş Aygıtlar Yardım
root@kly:~# cat /etc/kly/sources.list
kendilinuxunuyap/kly-binary-packages
root@kly:~# kly -u
***** - Paket Listesi Güncelleniyor *****
***** - Paket Listesi Güncellendi *****
root@kly:~#
```

Aşağıda **/var/lib/kly/index.lst** konumundaki dosyanın içeriğini görmekteyiz.

```
kly [Çalışıyor] - Oracle VM VirtualBox
Dosya Makine Görünüm Giriş Aygıtlar Yardım
root@kly:~# cat /var/lib/kly/index.lst
name="bash":version="5.2.21":depends="glibc,readline,ncurses"
root@kly:~#
```


GNU Araçlarıyla xorg ve x11 Derleme

Ön Hazırlık

Paket derleme işlemi öncesi aşağıdaki konuları bilmemiz gerekmektedir. Bunlar;

1. Derleme(Dinamik/Static)
2. chroot Kullanımı
3. İso Oluşturma
4. ssh Kullanımı
5. sftp Kullanımı
6. scp Kullanımı
7. VirtualBox Kullanımı
8. cfdisk Kullanımı

Burada liste halinde verilen konu başlıkları bu dokümanın **Yardımcı Konular** bölümünde anlatılmaktadır.

Bundan sonraki adımlarda kendi dağıtımımızın **xorg ve x11** pencere sistemini derleyerek **Temel Sistem** üzerinde çalıştıracağız!

GNU Araçlarıyla **xorg ve x11** derleme işlemini **kly Paket Sistemi** kullanılarak derleyeceğiz. Derleme işlemini **kly -c** komutuyla yapacağız. Derlenen paketleri **scp** ve **sftp** kullanarak **Temel Sistem** üzerine kopyalayacağız. **kly -i** komutumuzla kopyaladığımız paketi **Temel Sistem** üzerine kuracağız. Oluşturduğumuz paketleri istersek github'a yükleyip. github üzerinden kurabiliriz.

xorg ve x11'in Çalışması İçin Gerekli Paketler

0- Ön Hazırlık	25- libX11	50- libinput
1- xorg-server	26- libICE	51- mtdev
2- pixman	27- libXrender	52- libevdev
3- libpciaccess	28- libxcb	53- libwacom
4- libXau	29- libSM	54- libgudev
5- libXdmcp	30- xf86-input-libinput	55- libffi
6- libXfont2	31- xf86-input-vmmouse	56- xinit
7- libxshmfence	32- xf86-video-amdgpu	57- xcalc
8- libdrm	33- xf86-video-ast	58- libXi
9- libxcvt	34- xf86-video-ati	59- openbox
10- libfontenc	35- xf86-video-dummy	60- libXcursor
11- freetype	36- xf86-video-fbdev	61- libXfixes
12- libpng	37- xf86-video-intel	62- pango
13- harfbuzz	38- xf86-video-mga	63- libXrandr
14- glib	39- xf86-video-nouveau	64- fribidi
15- xterm	40- xf86-video-r128	65- xcb-util
16- libXft	41- xf86-video-siliconmotion	66- libthai
17- fontconfig	42- xf86-video-vboxvideo	67- libdatrie
18- dejavu	43- xf86-video-vesa	68-
19- libXext	44- xf86-video-vmware	69-
20- libXaw	45- xkbcomp	70-
21- libXmu	46- libxkbfile	71-
22- libXinerama	47- libglvnd	72-
23- libXpm	48- startup-notification	73-
24- libXt	49- xkeyboard-config	74-

Bağımlılık Zinciri

Linux paketinin sorunsuz çalışabilmesi için bağımlı olduğu tüm paketlerin önceden derlenmiş olması gerekir. **x11**'in en temel paketleri **xorg-server**, **mesa**, **llvm**, **cairo** paketleridir. Tüm paketleri derlese bile **xorg-server**, **mesa**, **llvm**, **cairo** paketleri düzgün ve uyumlu versiyonları olmadığı zaman x penceremiz açılmayacaktır. Buradaki tüm paketler ve bağımlılıkları derlendikten sonra **Xorg:0** şeklinde x pencere sistemimiz çalışacaktır.

Derleme Öncesi Hazırlık!

Paket derleme işlemine başlamadan önce, aşağıdaki temel araçları sisteminize kurmalısınız.

```
sudo apt update
sudo apt-get install debootstrap xorriso mtools make squashfs-tools gcc wget unzip xz-utils tar zstd fakeroot \
autoconf automake autotools-dev make meson cmake ninja-build pkgconf patch libtool grub-pc grub-pc-bin
```

xorg-server

Linux ve Unix-benzeri sistemlerde grafik kullanıcı arayüzünün çalışmasını sağlayan bir **görüntü sunucusudur**.

"**X.Org Foundation**" tarafından geliştirilen **X Window System**'in (X11) bir parçasıdır.

Bir uygulama, X protokolü üzerinden bir pencere açmak istediğinde, bu pencereyi ekrana **xorg-server** çizer.

Paketi Derleme :

Debian'da paketi derlemek için aşağıdaki paketlerin kurulu olması gerekir.

```
sudo apt install build-essential autoconf automake libtool pkg-config libexpat1-dev python3
```

```
#!/usr/bin/env bash
name="xorg-server"
version="21.1.6"
description="X.Org X servers"
source="https://www.x.org/releases/individual/xserver/xorg-server-$version.tar.xz"
depends="libmd,libbsd,libpox,libglvnd,libunwind,libfontenc,libxkbfile,xauth, \
xkbcomp,setxkbmap,\ freetype,libXfont2,libxcvt,mesa,libX11,libdrm,pixman, \
font-util,xcv-util-renderutil,xcv-util, xcv-util-image,xcv-util-wm,xcv-util-keysyms"
group="x11.base"

setup(){
    cd $SOURCEDIR
    meson setup $BUILDDIR --prefix=/usr \
        --libdir=/usr/lib64/ \
        -Dipv6=true \
        -Dxvfb=true \
        -Dxnest=true \
        -Dxcsecurity=true \
        -Dxorg=true \
        -Dxephyr=true \
        -Dglamor=true \
        -Dudev=true \
        -Ddtrace=false \
        -Dsystemd_logind=false \
        -Dsuid_wrapper=true \
        -Dxkb_dir=/usr/share/X11/xkb \
        -Dxkb_output_dir=/var/lib/xkb
}

build(){
    ninja -C $BUILDDIR
}

package(){
    DESTDIR=$DESTDIR ninja -C $BUILDDIR install
}
```

Not: Burada verilen derleme talimatımızı(script) ve bundan sonra gelen derleme talimatlarını **kly Paket Yapma(kly -c pakettizini)** konusu altında anlatılan şekilde derlemeli ve **.kly** paketlerini **kly Paket Sistemi** ile kullanabilirsiniz.

pixman

Pixman, grafik işlemleri için kullanılan düşük seviyeli bir kütüphanedir. Temel amacı, pikseller üzerinde doğrudan işlemler yapmaktır.

Paketi Derleme :

Debian'da paketi derlemek için aşağıdaki paketlerin kurulu olması gerekir.

```
sudo apt install build-essential autoconf automake libtool pkg-config libexpat1-dev python3
```

```
#!/usr/bin/env bash
name="pixman"
version="0.42.2"
description="Low-level pixel manipulation routines"
source="https://www.x.org/archive/individual/lib/pixman-$version.tar.gz"
depends=""
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libpciaccess

PCI (Peripheral Component Interconnect) aygıtlarına erişim sağlayan düşük seviyeli bir kütüphanedir. Genellikle Linux ve Unix benzeri sistemlerde, özellikle X.Org (grafik sunucusu) projelerinde ve bazı sürücülerde kullanılır.

Paketi Derleme :

Debian'da paketi derlemek için aşağıdaki paketlerin kurulu olması gerekir.

```
sudo apt install build-essential autoconf automake libtool pkg-config libexpat1-dev python3
```

```
#!/usr/bin/env bash
name="libpciaccess"
version="0.17"
description="Library providing generic access to the PCI bus and devices"
source="https://www.x.org/archive/individual/lib/libpciaccess-$version.tar.xz"
depends=""
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libXau

X11 (X Window System) ile birlikte kullanılan bir yetkilendirme (authentication) kütüphanesidir. X istemcileri ile X sunucusu arasında bağlantı kurulurken güvenlik doğrulaması sağlar.

Paketi Derleme :

Debian'da paketi derlemek için aşağıdaki paketlerin kurulu olması gerekir.

```
sudo apt install build-essential autoconf automake libtool pkg-config libexpat1-dev python3
```

```
#!/usr/bin/env bash
name="libXau"
version="1.0.11"
description="X.Org X authorization library"
source="https://www.x.org/releases/individual/lib/libXau-${version}.tar.xz"
depends="xorgproto"
builddepend=""
group="x11.libs"

setup(){
    export PKG_CONFIG_PATH=/usr/lib/pkgconfig
    cd $SOURCEDIR
    ./configure --prefix=/usr \
                --sysconfdir=/etc \
                --without-xproto
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libXdmcp

X Display Manager Control Protocol Library, X11 grafik sistemi kapsamında kullanılan ve uzak X istemcileriyle oturum yönetimi yapmaya yarayan bir kütüphanedir.

```
#!/usr/bin/env bash
name="libXdmcp"
version="1.1.5"
description="X.Org X Display Manager Control Protocol library"
source="https://www.x.org/archive/individual/lib/libXdmcp-$version.tar.xz"
depends=""
builddepend="xorgproto util-macros xmlto"
group="x11.libs"

setup(){
    cd $SOURCEDIR
    export PKG_CONFIG_PATH=/usr/lib/pkgconfig
    ./configure --prefix=/usr \
                --sysconfdir=/etc
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libXfont2

X sunucusu tarafından kullanılan bir yazı tipi erişim ve yönetim kütüphanesidir. X sunucusunun çeşitli font kaynaklarına erişmesini sağlar.

```
#!/usr/bin/env bash
name="libXfont2"
version="2.0.6"
description="X.Org Xfont library"
source="https://www.x.org/archive/individual/lib/libXfont2-$version.tar.xz"
depends=""
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```


libxshmfence

X11 grafik sistemi için geliştirilmiş küçük ve özel amaçlı bir kütüphanedir. Temel görevi, paylaşılan bellek üzerinden eşzamanlama sağlamaktır.

```
#!/usr/bin/env bash
name="libxshmfence"
version="1.3.2"
description="Shared memory fences using futexes"
source="https://www.x.org/archive/individual/lib/libxshmfence-$version.tar.xz"
depends="xorgproto"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libdrm

Linux'ta kullanıcı düzeyindeki uygulamaların çekirdekteki DRM (Direct Rendering Manager) altyapısıyla iletişim kurmasını sağlayan bir ara katman kütüphanesidir. Grafik donanımına doğrudan, güvenli ve verimli erişim için kullanılır. Özellikle Mesa (OpenGL), Wayland, X.Org, ve GPU sürücülerini tarafından kullanılır.

```
#!/usr/bin/env bash
name="libdrm"
version="2.4.120"
description="X.Org libdrm library"
source="https://dri.freedesktop.org/libdrm/libdrm-$version.tar.xz"
depends="libpciaccess"
group="x11.libs"

setup(){
    cd $SOURCEDIR
    meson setup $BUILDDIR --prefix=/usr \
        -D default_library=both \
        -D udev=false \
        -D etnaviv=disabled \
        -D freedreno=disabled \
        -D vc4=disabled \
        -D valgrind=disabled \
        -D install-test-programs=true
}

build(){
    ninja -C $BUILDDIR
}

package(){
    DESTDIR=$DESTDIR ninja -C $BUILDDIR install
}
```

libxcvt

VESA CVT standardına uygun olarak ekran çözünürlüğü ve zamanlama bilgileri hesaplayan bir kütüphanedir. Yeni ekran çözünürlükleri oluşturmak için kullanılan hesaplama kütüphanesidir.

```
#!/usr/bin/env bash
name="libxcvt"
version="0.1.2"
description="X.Org xcvt library and cvt program"
source="https://gitlab.freedesktop.org/xorg/lib/libxcvt/-\
/archive/libxcvt-$version/libxcvt-libxcvt-$version.tar.gz"
depends=""
group="x11.libs"

setup(){
cd $SOURCEDIR
    meson setup $BUILDDIR --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    ninja -C $BUILDDIR
}

package(){
    DESTDIR=$DESTDIR ninja -C $BUILDDIR install
}
```

libfontenc

X11 sistemlerinde kullanılan yazı tipi kodlama bilgilerini işleyen bir kütüphanedir. Bu kütüphane genellikle bitmap fontları işlerken ve yazı tipi dönüştürmeleri yapılırken kullanılır.

```
#!/usr/bin/env bash
name="libfontenc"
version="1.1.7"
description="PX.Org fontenc library"
source="https://www.x.org/archive/individual/lib/libfontenc-$version.tar.xz"
depends=""
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

freetype

Açık kaynaklı bir yazı tipi motorudur. Çeşitli yazı tipi dosyalarını çözümleyip ekranda yüksek kaliteli metinlerin piksel piksel çizilmesini sağlar.

```
#!/usr/bin/env bash
name="freetype"
version="2.13.1"
description="Package freetype"
source="https://download.savannah.gnu.org/releases/freetype/freetype-$version.tar.gz"
depends="zlib,bzip2,glib,libpng,harfbuzz"
group="media.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/ \
        --with-harfbuzz=yes
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libpng

Portable Network Graphics (PNG) formatındaki resim dosyalarını okumak ve yazmak için kullanılan açık kaynaklı bir kütüphanedir.

```
#!/usr/bin/env bash
name="libpng"
version="1.6.39"
description="Portable Network Graphics library"
source="https://salsa.debian.org/debian/libpng1.6/-\
/archive/debian/$version-2/libpng1.6-debian-$version-2.tar.gz"
depends="zlib"
group="media.libs"

setup(){
    cd $SOURCEDIR
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

harfbuzz

Metin şekillendirme (text shaping) motorudur. Yazı tipi karakterlerinin, dilin ve yazım kurallarının özelliklerine göre doğru ve estetik şekilde yerleşimini sağlar.

```
#!/usr/bin/env bash
name="harfbuzz"
version="8.3.0"
description="HarfBuzz text shaping engine"
source="https://github.com/harfbuzz/harfbuzz/archive/refs/tags/$version.tar.gz"
depends="cairo,glib"
group="media.libs"
#libgirepository1.0-dev
#libcairo2-dev
#libchafa-dev
setup(){
    cd $SOURCEDIR
    meson setup $BUILDDIR --prefix=/usr \
        --libdir=/usr/lib64/ \
        -Dglib=enabled \
        -Dgobject=enabled \
        -Dicu=enabled \
        -Dfreetype=enabled \
        -Dtests=disabled \
        -Dcairo=enabled \
        -Ddocs=enabled
}

build(){
    ninja -C $BUILDDIR
}

package(){
    DESTDIR=$DESTDIR ninja -C $BUILDDIR install
}
```

glib

GNOME projesi tarafından geliştirilen, C programlama dili için temel yardımcı kütüphanedir. C programlama dilinde sıkça ihtiyaç duyulan veri yapıları, dize işlemleri, bellek yönetimi, iş parçacığı yönetimi, olay döngüsü, sinyal sistemi gibi birçok temel fonksiyonu sağlar.

```
#!/usr/bin/env bash
name="glib"
version="2.78.0"
description="Low-level core library that forms the basis for projects\
  such as GTK+ and GNOME."
source="https://download.gnome.org/sources/glib/2.78/glib-${version}.tar.xz"
depends="bzip2,gettext,python,py3-packaging"
builddepend="bison,flex,libffi,meson,pcre2,py3-setuptools,py3-docutils,util-linux"
group="dev.libs"

setup(){
cd $SOURCEDIR
  cp $PACKAGEDIR/files/* $SOURCEDIR
  meson setup $BUILDDIR --prefix=/usr \
    --libdir=/usr/lib64 \
    --default-library both \
    -D glib_debug=disabled \
    -D selinux=disabled \
    -D sysprof=disabled
}

build(){
  ninja -C $BUILDDIR
}

package(){
  DESTDIR=$DESTDIR ninja -C $BUILDDIR install
}
```

Ek dosyaları indirmek için [tıklayınız](#).

xterm

Unix ve Linux sistemlerde kullanılan klasik, sade ve hafif bir terminal emülatörüdür.

```
#!/usr/bin/env bash
name="xterm"
version="388"
description="Terminal Emulator for X Windows"
depends="libX11,ncurses,util-linux,libXt,libXau,libXinerama,libXaw,libXpm,libXft"
source="https://invisible-island.net/archives/xterm/xterm-$version.tgz"
group="x11.terms"

setup(){
    cp -prfv $PACKAGEDIR/files/* $SOURCEDIR

    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/ \
        --exec-prefix=/usr \
        --with-app-defaults=/etc/X11/app-defaults \
        --with-icondir=/usr/share/icons \
        --with-icon-theme=yes \
        --with-tty-group=tty \
        --enable-warnings \
        --enable-logging \
        --enable-wide-chars \
        --enable-luit \
        --enable-256-color \
        --disable-imake \
        --enable-narrowproto \
        --enable-exec-xterm \
        --enable-dabbrev \
        --enable-backarrow-is-erase \
        --enable-sixel-graphics \
        --with-utempter \
        --with-desktop-category=System,TerminalEmulator
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
    make install-ti $jobs DESTDIR=${DESTDIR}
    mkdir -p "${DESTDIR}"/usr/share/applications
    mkdir -p "${DESTDIR}"/usr/share/xgreeters
    install $SOURCEDIR/{xterm,uxterm}.desktop "${DESTDIR}"/usr/share/applications/
    install $SOURCEDIR/xterm.desktop "${DESTDIR}"/usr/share/xgreeters/
}
```

Ek dosyaları indirmek için [tıklayınız](#).

libXft

X11 grafik sisteminde FreeType yazı tipi motorunu kullanarak daha güzel, pürüzsüz (antialiased) metinler göstermek için kullanılan bir kütüphanedir.

```
#!/usr/bin/env bash
name="libXft"
version="2.3.7"
description="XFreeType-based font drawing library for X"
source="https://www.x.org/archive/individual/lib/libXft-$version.tar.xz"
depends="fontconfig,libXrender"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

fontconfig

Linux ve Unix benzeri sistemlerde kullanılan, yazı tiplerini keşfetme, yapılandırma ve yönetme işlevini sağlayan bir kütüphane ve araç setidir. Sistem üzerindeki tüm yazı tiplerini tarar, organize eder ve uygulamaların kolayca kullanabilmesi için standart bir API sunar.

```
#!/usr/bin/env bash
name="fontconfig"
version="2.15.0"
description="Fontconfig is a library for configuring and customizing font access."
source="https://www.freedesktop.org/software/fontconfig/release/fontconfig-$version.tar.gz"
depends="freetype,expat"
group="media.libs"

setup(){
cd $SOURCEDIR
  cp -prfv $PACKAGEDIR/files/* $SOURCEDIR
  meson setup $BUILDDIR --prefix=/usr \
    --libdir=/usr/lib64/
}

build(){
  ninja -C $BUILDDIR
}

package(){
  DESTDIR=$DESTDIR ninja -C $BUILDDIR install
  # fix symlinks
  for link in $(ls ${DESTDIR}/etc/fonts/conf.d/); do
    if [[ -L ${DESTDIR}/etc/fonts/conf.d/$link ]]; then
      rm -f ${DESTDIR}/etc/fonts/conf.d/$link
      ln -s ../../usr/share/fontconfig/conf.avail/$link ${DESTDIR}/etc/fonts/conf.d/$link
    fi
  done
}
```

dejavu

Açık kaynaklı ve geniş kapsamlı bir yazı tipi ailesidir. Çok sayıda dil ve karakter setini kapsayan, özgür ve kaliteli fontlar sağlar.

```
#!/usr/bin/env bash
name="dejavu"
version="2.37"
description="DejaVu fonts, bitstream vera with ISO-8859-2 characters"
source="https://github.com/dejavu-fonts/dejavu-fonts/releases/download/\
version_${version}/_}/dejavu-lgc-fonts-ttf-$version.zip"
depends=""
group="media.fonts"
setup()
{
    /bin/busybox wget "https://github.com/dejavu-fonts/dejavu-fonts/releases/\
download/version_${version}/_}/dejavu-fonts-ttf-$version.zip"

    /bin/busybox unzip dejavu-fonts-ttf-$version.zip
    /bin/busybox rm dejavu-fonts-ttf-$version.zip
    /bin/busybox cp -prfv $SOURCEDIR ./
}
build()
{
    echo ""
}
package(){
    mkdir -p ${DESTDIR}/usr/share/fonts
    for font in dejavu dejavu-fonts-ttf; do
        cp -prfv $BUILDDIR/$font-$version/ttf ${DESTDIR}/usr/share/fonts/$font
    done
}
```

libXext

X sunucusundaki ek özelliklerin (uzantıların) istemciler tarafından kolayca kullanılmasını sağlayan bir kütüphanedir.

```
#!/usr/bin/env bash
name="libXext"
version="1.3.5"
description="X.Org Xext library"
source="https://www.x.org/archive/individual/lib/libXext-$version.tar.xz"
depends="libX11,xorgproto"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libXaw

X11 için geliştirilmiş klasik bir grafik kullanıcı arayüzü (GUI) widget kütüphanesidir. X Window System üzerinde temel GUI bileşenleri (düğmeler, listeler, menüler, metin kutuları vb.) sağlar.

```
#!/usr/bin/env bash
name="libXaw"
version="1.0.14"
description="Package libXaw"
source="https://www.x.org/archive/individual/lib/libXaw-$version.tar.gz"
depends="libXext,libXt,libXmu,libXpm"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libXmu

X11 pencere sistemi için çeşitli yardımcı işlevler sağlayan bir kütüphanedir. X uygulamaları tarafından sık kullanılan, ancak temel X kütüphanelerinde bulunmayan bazı ek fonksiyonları içerir.

X11 uygulamalarında pencere işlemleri, iletişim kutuları ve pencere özniteliklerine erişim gibi yardımcı işlevler için kullanılır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libXmu"
version="1.1.4"
description="X.Org Xmu library"
source="https://www.x.org/archive/individual/lib/libXmu-$version.tar.xz"
depends="libXt,libXext,libX11"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libXinerama

Xinerama, birden fazla fiziksel monitörü tek bir sanal ekran gibi kullanmayı sağlayan bir X11 uzantısıdır. **libXinerama** ise bu uzantıya erişim sağlayan istemci taraflı bir kütüphanedir.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libXinerama"
version="1.1.5"
description="X.Org Xinerama library"
source="https://www.x.org/archive/individual/lib/libXinerama-$version.tar.xz"
depends=""
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```


libXpm

X11 pencere sistemi için **XPM (X PixMap)** formatındaki görüntü dosyalarını işlemek üzere kullanılan bir kütüphanedir.

XPM, özellikle X11 uygulamalarında simge, buton ve diğer grafik öğeleri için kullanılan metin tabanlı bir bitmap görüntü formatıdır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libXpm"
version="3.5.14"
description="X.Org Xpm library"
source="https://www.x.org/archive/individual/lib/libXpm-$version.tar.xz"
depends="libX11"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libXt

X11 pencere sistemi için widget tabanlı GUI (grafik kullanıcı arayüzü) uygulamaları geliştirmeyi kolaylaştıran temel bir araç takımı altyapısı sağlayan bir kütüphanedir.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libXt"
version="1.3.0"
description="X.Org X Toolkit Intrinsics library"
source="https://www.x.org/archive/individual/lib/libXt-$version.tar.gz"
depends="libSM,libX11"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libX11

X11 pencere sistemi ile çalışan grafik uygulamaların X sunucusuyla iletişim kurmasını sağlayan temel istemci kütüphanesidir.

- X protokolünü kullanan uygulamalar ile X sunucusu arasında bağlantı kurar.
- X sunucusuna pencere oluşturma, olay alma (fare/klavye), çizim yapma gibi isteklerin gönderilmesini sağlar.
- X11 istemcilerinin çalışması için en temel ve gerekli kütüphanedir.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libX11"
version="1.8"
description="X.Org X11 library"
source="https://www.x.org/archive/individual/lib/libX11-$version.tar.xz"
depends="libxcb,libXau,libXdmcp,xtrans"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libICE

X11 pencere sistemi altında çalışan istemciler (uygulamalar) arasında protokol tabanlı iletişim kurmayı sağlayan bir kütüphanedir.

- libICE, X istemcilerinin birbirleriyle veri alışverişi yapmasını sağlar.
- ICE protokolü üzerinden bağlantı yönetimi, oturumlar arası veri paylaşımı ve kontrol sağlar.
- Genellikle libSM (Session Management) gibi üst düzey kütüphaneler tarafından kullanılır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libICE"
version="1.1.1"
description="X.Org Inter-Client Exchange library"
source="https://www.x.org/archive/individual/lib/libICE-$version.tar.xz"
depends="xorgproto"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libXrender

X11 pencere sistemi için 2D grafiklerin daha gelişmiş ve estetik şekilde çizilmesini sağlayan bir kütüphanedir. X sunucusundaki Render (Rendere) uzantısını kullanan istemci tarafı arayüzüdür.

- X11 Render uzantısına erişim sağlar ve yarı saydamlık, alfa kanalı, yumuşatma gibi gelişmiş grafik özelliklerini destekler.
- libX11 üzerine inşa edilmiştir; onun sunduğu temel çizim işlevlerini görüntü olarak zenginleştirir.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libXrender"
version="0.9.11"
description="X.Org Xrender library"
source="https://www.x.org/archive/individual/lib/libXrender-$version.tar.xz"
depends=""
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libxcb

X11 protokolüne erişim sağlamak için geliştirilmiş, düşük seviyeli, modern ve verimli bir istemci kütüphanesidir. libX11 kütüphanesinin daha hızlı ve daha modüler bir alternatifi veya tamamlayıcısı olarak tasarlanmıştır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libxcb"
version="1.17.0"
description="X C-language Bindings library"
#source="https://www.x.org/releases/individual/lib/libxcb-$version.tar.xz"
source="https://gitlab.freedesktop.org/xorg/lib/libxcb/-\
/archive/libxcb-$version/libxcb-libxcb-$version.tar.gz"
depends="libXau,libXdmcp,xcb-proto"
builddepend="libxslt,python3,util-macros"
group="x11.libs"

setup(){
    cd $SOURCEDIR
    #export PKG_CONFIG_PATH=/usr/lib/pkgconfig
    autoreconf -fvi
    ./configure --prefix=/usr \
                --libdir=/usr/lib64 \
                --enable-xinput \
                --enable-xkb \
                --disable-static
}

build(){
    #sed -i -e 's/ -shared / -Wl,-01,--as-needed\0/g' libtool
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libSM

X11 pencere sisteminde oturum yönetimi (session management) işlevlerini sağlayan bir kütüphanedir. Uygulamaların, oturum kapanmadan önce kayıt yapmasını, sonraki oturumda kaldığı yerden devam etmesini mümkün kılar.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libSM"
version="1.2.4"
description="libSM X.Org Session Management library"
source="https://www.x.org/archive/individual/lib/libSM-$version.tar.xz"
depends="xorgproto,libICE"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xf86-input-libinput

X11 için modern giriş aygıtı sürücüsü sağlayan bir kütüphanedir. Bu sürücü, özellikle dokunmatik ekranlar, fareler, klavyeler ve çoklu dokunmatik yüzeyler gibi giriş aygıtlarını yönetir.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-input-libinput"
version="1.4.0"
description="X.org input driver based on libinput"
source="https://gitlab.freedesktop.org/xorg/driver/xf86-input-libinput/-/archive/\
xf86-input-libinput-$version/xf86-input-libinput-xf86-input-libinput-$version.tar.gz"
depends="libinput"
group="x11.drivers"

setup(){
    cd $SOURCEDIR
    ./autogen.sh
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```


xf86-input-vmmouse

Sanallaştırılmış ortamlar için geliştirilen bir giriş aygıtı sürücüsüdür. Sanal makinelerde çalışan X11 sistemlerinde fareyi doğru şekilde yönetmek için kullanılır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-input-vmmouse"
version="13.2.0"
description="VMWare mouse input driver"
source="https://gitlab.freedesktop.org/xorg/driver/xf86-input-vmmouse/-/archive/\
xf86-input-vmmouse-$version/xf86-input-vmmouse-xf86-input-vmmouse-$version.tar.gz"
depends=""
group="x11.drivers"

setup(){
    cd $SOURCEDIR
    ./autogen.sh
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xf86-video-amdgpu

AMD grafik kartları için X11 video sürücüsüdür ve **AMD Radeon** ve **AMD Vega** serisi grafik kartlarıyla uyumlu çalışır. Bu sürücü, **amdgpu** adlı açık kaynaklı AMD GPU sürücüsünü kullanarak, X.Org Server'da donanım hızlandırması ve yüksek performanslı grafik işlevi sağlar.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-video-amdgpu"
version="23.0.0"
description="Accelerated Open Source driver for AMDGPU cards"
source="https://gitlab.freedesktop.org/xorg/driver/xf86-video-amdgpu/-/archive/\
xf86-video-amdgpu-$version/xf86-video-amdgpu-xf86-video-amdgpu-$version.tar.gz"
depends=""
group="x11.drivers"

setup(){
    cd $SOURCEDIR
    ./autogen.sh
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xf86-video-ast

AST (Advanced Systems Technology) grafik kartları için geliştirilmiş bir X11 video sürücüsüdür. AST kartları genellikle gömülü sistemler, iş istasyonları ve sunucu uygulamaları gibi alanlarda kullanılır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-video-ast"
version="1.1.6"
description="X.Org driver for ASpeedTech cards"
source="https://gitlab.freedesktop.org/xorg/driver/xf86-video-ast/-/archive/\
xf86-video-ast-$version/xf86-video-ast-xf86-video-ast-$version.tar.gz"
depends=""
group="x11.drivers"

setup(){
    cd $SOURCEDIR
    ./autogen.sh
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xf86-video-ati

AMD/ATI grafik kartları için X11 video sürücüsüdür. Bu sürücü, eski ATI Radeon grafik kartları için temel 2D grafik hızlandırması sağlar ve X.Org Server'da çalışır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-video-ati"
version="22.0.0"
description="ATI video driver"
source="https://gitlab.freedesktop.org/xorg/driver/xf86-video-ati/-/archive/\
xf86-video-ati-$version/xf86-video-ati-xf86-video-ati-$version.tar.gz"
depends=""
group="x11.drivers"

setup(){
    cp -prfv $PACKAGEDIR/files/* $SOURCEDIR
    cd $SOURCEDIR
    patch -Np1 -i $SOURCEDIR/patches/xf86-video-ati-19.1.0-upstream_fixes-1.patch
    ./autogen.sh
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

Ek dosyaları indirmek için [tıklayınız](#).

xf86-video-dummy

X11 için sanal (dummy) bir video sürücüsüdür. Bu sürücü, herhangi bir gerçek grafik donanımı kullanmayan, yalnızca yazılım temelli ortamlar veya test amaçları için tasarlanmıştır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-video-dummy"
version="0.4.0"
description="X.Org driver for dummy cards"
source="https://gitlab.freedesktop.org/xorg/driver/xf86-video-dummy/-/archive/\
xf86-video-dummy-$version/xf86-video-dummy-xf86-video-dummy-$version.tar.gz"
depends=""
group="x11.drivers"

setup(){
    cd $SOURCEDIR
    ./autogen.sh
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xf86-video-fbdev

framebuffer (fbdev) tabanlı grafik kartları için X11 video sürücüsüdür. Framebuffer, ekranın bir bellek alanında (genellikle bir RAM tamponu) saklandığı bir yöntemdir, bu da daha düşük seviyede ve donanım bağımsız bir ekran çıkışı sağlar.

fbdev, düşük seviyede grafik işleme sağlar, ancak donanım hızlandırma veya 3D grafik desteği sunmaz.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-video-fbdev"
version="0.5.0"
description="video driver for framebuffer devic"
source="https://gitlab.freedesktop.org/xorg/driver/xf86-video-fbdev/-/archive/\
xf86-video-fbdev-$version/xf86-video-fbdev-xf86-video-fbdev-$version.tar.gz"
depends=""
group="x11.drivers"

setup(){
    cd $SOURCEDIR
    ./autogen.sh
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xf86-video-intel

Intel grafik kartları için X11 video sürücüsüdür ve Intel HD Graphics ve Intel Iris Graphics gibi entegre grafik çözümleri için optimize edilmiştir. Intel'in entegre GPU'ları için X.Org Server altında yüksek performanslı 2D ve 3D grafik işleme sağlar.

Intel'in eski grafik donanımları için daha fazla uyum ve performans sağlarken, yeni Intel GPU'ları için artık yerini i915 ve mesa gibi yeni sürücülere bırakmıştır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-video-intel"
version="2.99.917"
description="X.Org driver for Intel cards"
source="https://salsa.debian.org/xorg-team/driver/xserver-xorg-video-intel/-/\
archive/xserver-xorg-video-intel-2_${version}+git20210115-1/\
xserver-xorg-video-intel-xserver-xorg-video-intel-2_${version}+git20210115-1.tar.gz"
depends=""
group=x11.drivers

setup(){
    cd $SOURCEDIR
    ./autogen.sh
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/ \
        --with-default-dri=3
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xf86-video-mga

Matrox grafik kartları için geliştirilmiş bir X11 video sürücüsüdür. Matrox'un MGA (Matrox Graphics Architecture) tabanlı grafik kartlarını destekler ve genellikle eski Matrox grafik donanımlarıyla uyumludur.

Matrox kartları, özellikle 3D hızlandırma ve çift monitör desteği gibi özelliklerle tanınır. Matrox'un eski grafik kartları için 2D ve bazı eski 3D grafik işlevlerini destekler.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-video-mga"
version="2.0.1"
description="Matrox video driver"
source="https://gitlab.freedesktop.org/xorg/driver/xf86-video-mga/-/archive/\
xf86-video-mga-$version/xf86-video-mga-xf86-video-mga-$version.tar.gz"
depends=""
group="x11.drivers"

setup(){
    cd $SOURCEDIR
    ./autogen.sh
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```


xf86-video-nouveau

NVIDIA grafik kartları için geliştirilmiş bir açık kaynaklı X11 video sürücüsüdür. Nouveau sürücüsü, NVIDIA'nın kapalı kaynaklı sürücülerine alternatif olarak geliştirilmiş olup, Linux ve BSD tabanlı sistemlerde NVIDIA grafik kartlarıyla uyumlu çalışır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-video-nouveau"
version="1.0.17"
description="Accelerated Open Source driver for nVidia cards"
source="https://gitlab.freedesktop.org/xorg/driver/xf86-video-nouveau/-/archive/\
xf86-video-nouveau-$version/xf86-video-nouveau-xf86-video-nouveau-$version.tar.gz"
depends=""
group="x11.drivers"

setup(){
    cp -prfv $PACKAGEDIR/files/* $SOURCEDIR
    cd $SOURCEDIR
    patch -Np1 < $SOURCEDIR/patches/xorg-server-21.1.diff
    autoreconf -fvi
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

Ek dosyaları indirmek için [tıklayınız](#).

xf86-video-r128

ATI R128 serisi grafik kartları için geliştirilmiş bir X11 video sürücüsüdür. Bu sürücü, ATI R128 (Rage 128) tabanlı grafik kartları için 2D grafik hızlandırma ve ekran yönetimi sağlar.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-video-r128"
version="6.12.1"
description="ATI Rage128 video drive"
source="https://gitlab.freedesktop.org/xorg/driver/xf86-video-r128/-/archive/\
xf86-video-r128-$version/xf86-video-r128-xf86-video-r128-$version.tar.gz"
depends=""
group="x11.drivers"

setup(){
    cd $SOURCEDIR
    ./autogen.sh
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xf86-video-siliconmotion

Silicon Motion grafik kartları için geliştirilmiş bir X11 video sürücüsüdür. Silicon Motion, genellikle gömülü sistemler, taşınabilir cihazlar ve eski bilgisayarlar için grafik çözümleri üreten bir üreticidir. Silicon Motion'un SM720, SM740, SM750 gibi eski grafik kartlarıyla uyumlu çalışır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-video-siliconmotion"
version="1.7.9"
description="Silicon Motion video driver"
source="https://gitlab.freedesktop.org/xorg/driver/xf86-video-siliconmotion/-\
/archive/xf86-video-siliconmotion-$version/\
xf86-video-siliconmotion-xf86-video-siliconmotion-$version.tar.gz"
depends=""
group="x11.drivers"

setup(){
    cd $SOURCEDIR
    ./autogen.sh
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xf86-video-vboxvideo

VirtualBox sanal makinesi için geliştirilmiş bir X11 video sürücüsüdür. VirtualBox üzerinde çalışan sanal makinelerde grafik hızlandırma sağlar ve sanal ekran çıkışı sunar.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-video-vboxvideo"
version="1.0.0"
description="VirtualBox guest video driver"
source="https://gitlab.freedesktop.org/xorg/driver/xf86-video-vbox/-/archive/\
xf86-video-vboxvideo-$version/\
xf86-video-vbox-xf86-video-vboxvideo-$version.tar.gz"
depends=""
group="x11.drivers"

setup(){
    cd $SOURCEDIR
    ./autogen.sh
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xf86-video-vesa

X11 için geliştirilmiş, genel amaçlı bir video sürücüsüdür. **VESA** (Video Electronics Standards Association) standartlarına dayanan grafik kartlarıyla çalışır ve donanım bağımsız olarak ekran çıkışı sağlar.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-video-vesa"
version="2.6.0"
description="Generic VESA video driver"
source="https://gitlab.freedesktop.org/xorg/driver/xf86-video-vesa/-/archive/\
xf86-video-vesa-$version/xf86-video-vesa-xf86-video-vesa-$version.tar.gz"
depends=""
group="x11.drivers"

setup(){
    cd $SOURCEDIR
    ./autogen.sh
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xf86-video-vmware

VMware sanal makineleri için geliştirilmiş bir X11 video sürücüsüdür. VMware sanal makinelerinde çalışan Linux ve BSD sistemlerine grafik hızlandırması ve sanal ekran yönetimi sağlar.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xf86-video-vmware"
version="13.4.0"
description="VMware SVGA video driver"
source="https://gitlab.freedesktop.org/xorg/driver/xf86-video-vmware/-/archive/\
xf86-video-vmware-$version/xf86-video-vmware-xf86-video-vmware-$version.tar.gz"
depends=""
group="x11.drivers"

setup(){
    cd $SOURCEDIR
    ./autogen.sh
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xkbcomp

X Keyboard Extension (XKB) ile ilgili yapılandırma dosyalarını oluşturmak ve düzenlemek için kullanılan bir komut satırı aracıdır. XKB, X11 sistemlerinde klavye düzenleri ve klavye özelliklerini yönetmek için kullanılan bir mekanizmadır. Özellikle XKB yapılandırma dosyalarını X11 sunucusuna yüklemek için kullanılır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xkbcomp"
version="1.4.6"
description="XKB keyboard description compiler"
source="https://gitlab.freedesktop.org/xorg/app/xkbcomp/-/archive/\
xkbcomp-$version/xkbcomp-xkbcomp-$version.tar.gz"
depends="libxkbfile,libX11"
group="x11.apps"

setup(){
    cd $SOURCEDIR
    autoreconf -fvi
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libxkbfile

X Keyboard Extension (XKB) ile ilgili yapılandırma dosyalarını işlemek için kullanılan bir C kütüphanesidir. Bu kütüphane, X11 sistemlerinde klavye düzenleri ve klavye yapılandırmalarını okuma ve yazma işlemleri yapmak için kullanılır. libxkbfile, XKB yapılandırma dosyaları ile etkileşimde bulunmak için gereken işlevleri sağlar.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libxkbfile"
version="1.1.2"
description="X.Org xkbfile library"
source="https://www.x.org/archive/individual/lib/libxkbfile-$version.tar.gz"
depends="libX11,xorgproto"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```


libglvnd

OpenGL (GL), OpenGL ES ve Vulkan gibi grafik API'leri için grafik sürücü yönetimini standartlaştıran ve modernize eden bir kütüphanedir. libglvnd, özellikle bir sistemde birden fazla grafik sürücüsünün çalışmasına olanak tanır ve farklı sürücüler arasında dinamik yükleme ve bağımlılık yönetimi sağlar.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libglvnd"
version="1.7.0"
description="The GL Vendor-Neutral Dispatch library"
source="https://gitlab.freedesktop.org/glvnd/libglvnd/-/archive/\
v$version/libglvnd-v$version.tar.gz"
depends="libXext"
group="media.libs"

setup(){
    cd $SOURCEDIR
    meson setup $BUILDDIR --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    ninja -C $BUILDDIR
}

package(){
    DESTDIR=$DESTDIR ninja -C $BUILDDIR install
}
```

startup-notification

Bir uygulamanın başlatılma süreci sırasında kullanıcıya görüntülü bildirimde bulunma amacıyla kullanılan bir X11 uzantısıdır. Bu uzantı, uygulama başlatıldığında, özellikle yavaş başlatan uygulamalarda kullanıcıyı bilgilendirir ve etkileşimli bir geri bildirim sağlar.

Paketi Derleme :

```
#!/usr/bin/env bash
name="startup-notification"
version="0.12"
description="Application startup notification and feedback library "
source="http://www.freedesktop.org/software/startup-notification/\
releases/startup-notification-${version}.tar.gz"
depends="libX11, xcb-util"
group="x11.libs"

cd startup-notification-$version

setup(){
    autoreconf -fvi
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xkeyboard-config

X11 sistemlerinde klavye düzenlerini (layout) ve klavye seçeneklerini yapılandırmak için kullanılan bir konfigürasyon dosyası paketidir. Bu paket, X Keyboard Extension (XKB) kullanarak, farklı klavye düzenlerinin, tuş atamalarının ve klavye seçeneklerinin yönetilmesini sağlar.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xkeyboard-config"
version="2.40"
description="X keyboard configuration database"
source="https://gitlab.freedesktop.org/xkeyboard-config/xkeyboard-config/-/\
archive/xkeyboard-config-$version/xkeyboard-config-xkeyboard-config-$version.tar.gz"
depends=""
group="x11.misc"

setup(){
    cd $SOURCEDIR
    meson setup $BUILDDIR --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    ninja -C $BUILDDIR
}

package(){
    DESTDIR=$DESTDIR ninja -C $BUILDDIR install
}
```

libinput

Giriş aygıtlarını (klavye, fare, dokunmatik ekran, vb.) yönetmek için kullanılan, açık kaynaklı bir kütüphanedir. Özellikle Linux ve Wayland tabanlı sistemlerde, giriş aygıtlarının doğru şekilde çalışmasını sağlamak amacıyla geliştirilmiştir. Modern ekran sunucuları ve giriş yönetim sistemlerinde giriş cihazlarının doğru bir şekilde işlev göstermesini sağlar.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libinput"
version="1.25.0"
url="https://gitlab.freedesktop.org/libinput/libinput"
description="Input device management and event handling library"
source="https://gitlab.freedesktop.org/libinput/libinput/-/archive/\
$version/libinput-$version.tar.gz"
depends="eudev,mtdev,libevdev"

setup(){
    cd $SOURCEDIR
    meson setup $BUILDDIR --prefix=/usr \
        --libdir=/usr/lib64/ \
        -Dudev-dir=/lib64/udev \
        -Dlibwacom=true \
        -Ddebug-gui=false \
        -Dtests=false
}

build(){
    ninja -C $BUILDDIR
}

package(){
    DESTDIR=$DESTDIR  ninja -C  $BUILDDIR install
}
```

mtdev

multitouch (çoklu dokunma) aygıtlarını yönetmek için kullanılan bir Linux kütüphanesi ve giriş aygıtı sürücüsüdür. Özellikle çoklu dokunmatik yüzeyler (multitouch touchpads, dokunmatik ekranlar vb.) ile çalışır ve çoklu dokunma olaylarını işler.

Paketi Derleme :

```
#!/usr/bin/env bash
name="mtdev"
version="1.1.6"
url="https://bitmath.org/code/mtdev/"
description="Multitouch Protocol Translation Library"
source="https://bitmath.org/code/mtdev/mtdev-$version.tar.gz"
group="dev.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make $jobs
}

package(){
    make install DESTDIR=$DESTDIR $jobs
}
```

libevdev

Linux tabanlı sistemlerde, evdev (event device) tabanlı giriş aygıtlarını (fareler, klavyeler, dokunmatik ekranlar vb.) yönetmek için kullanılan bir C kütüphanesidir. libevdev, giriş olaylarını okuma, yazma ve yapılandırma işlemleri sağlar ve bu sayede evdev aygıtları ile etkileşimi kolaylaştırır.

```
sudo apt install doxygen
```

Paketi Derleme :

```
#!/usr/bin/env bash
name="libevdev"
version="1.13.1"
description="Wrapper library for evdev devices"
source="https://gitlab.freedesktop.org/libevdev/libevdev/-/archive/\
libevdev-$version/libevdev-libevdev-$version.tar.gz"
depends=""
builddepend="doxygen,meson,python3"
group="sys.libs"
cd $name-$name-$version

setup(){
    cd $SOURCEDIR
    meson setup $BUILDDIR \
        --prefix=/usr \
        --libdir=/usr/lib \
        -Db_lto=true \
        -Dtests=disabled \
        -Ddocumentation=enabled \
        -Dcoverity=false
}

build(){
    ninja -C $BUILDDIR
}

package(){
    DESTDIR=$DESTDIR ninja -C $BUILDDIR install
}
```

libwacom

Linux sistemlerinde wacom tabletleri ve diğer dokunmatik grafik aygıtları için geliştirilmiş bir açık kaynaklı kütüphanedir. Wacom tabletleri ile etkileşimi sağlamak için tasarlanmıştır ve tablet yapılandırmasını, düğme ve kalem hassasiyetini yönetmek için kullanılır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libwacom"
version="2.10.0"
url="https://github.com/linuxwacom/libwacom/wiki"
description="Library to help implement Wacom tablet settings"
source="https://github.com/linuxwacom/libwacom/releases/download/\
libwacom-`${version}`/libwacom-`${version}`.tar.xz"
depends="eudev"
builddepend="meson"
group="dev.libs"

setup(){
    cd $SOURCEDIR
    meson setup $BUILDDIR \
        --prefix=/usr \
        --buildtype=release \
        -Dtests=disabled
}

build(){
    ninja -C $BUILDDIR $jobs
}

package(){
    DESTDIR=$DESTDIR ninja -C $BUILDDIR install $jobs
    install -D -m644 COPYING "${DESTDIR}/usr/share/licenses/${name}/LICENSE"
}
```

libgudev

Linux tabanlı sistemlerde donanım aygıtlarını yönetmek için kullanılan bir C kütüphanesidir. udev (Linux'un cihaz yöneticisi) ile uyumlu çalışarak, sistemdeki donanım aygıtları ile etkileşimi kolaylaştırır. libgudev, özellikle donanım aygıtlarının tanınması, bağlantı ve bağlantı olaylarının izlenmesi gibi işlevleri yönetir.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libgudev"
version="238"
url="https://example.org"
description="GObject bindings for libudev"
source="https://gitlab.gnome.org/GNOME/libgudev/-/archive/$version/\
libgudev-$version.tar.gz"
depends="glib,eudev"
builddepend=""
group="dev.libs"

setup(){
cd $SOURCEDIR
meson setup $BUILDDIR --prefix=/usr \
--libdir=/usr/lib64/ \
-Dintrospection=enabled \
-Dvapi=enabled \
-Dgtk_doc=false
}

build(){
meson compile -C $BUILDDIR
}

package(){
DESTDIR="$DESTDIR" meson install -C $BUILDDIR
}
```


libffi

C programlarının başka dillerde yazılmış (örneğin Python, Ruby, Lua gibi) fonksiyonları çalışma zamanında çağırabilmesini sağlayan bir düşük seviyeli programlama kütüphanesidir. Adı "foreign function interface" yani "yabancı fonksiyon arayüzü" anlamına gelir.

Paketi Derleme :

```
#!/bin/bash
name="libffi"
version="3.4.6"
description="A portable foreign-function interface library. "
source="https://github.com/libffi/libffi/releases/download/v${version}/\
libffi-${version}.tar.gz"
depends=""
builddepend=""
group="dev.libs"

setup(){
    cd $SOURCEDIR
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/ \
        --enable-pax_emutramp \
        --enable-portable-binary \
        --disable-exec-static-tramp
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xinit

X Pencere Sistemi (X11) için kullanılan bir başlatıcı komuttur. X sunucusunu başlatmak ve onunla birlikte bir istemci (örneğin bir pencere yöneticisi veya masaüstü ortamı) çalıştırmak için kullanılır.

xinit, doğrudan X sunucusunu başlatır ve ardından kullanıcı tarafından belirtilen bir istemci programı çalıştırır (örneğin: xterm, startkde, startxfce4, vs.).

Genellikle grafik arayüzü olmayan sistemlerde veya özel oturumlarda kullanılır.

~/xinitrc dosyası aracılığıyla hangi istemcilerin çalıştırılacağı belirlenebilir.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xinit"
version="1.4.2"
description="X Window System initializer xinit"
source="https://gitlab.freedesktop.org/xorg/app/xinit/-/archive/\
xinit-$version/xinit-xinit-1.4.2.tar.gz"
depends="util-macros,xorgproto,libX11,xorg-server,xterm,xhost,xrdb"
group="x11.apps"

setup(){
    cp -prfv ${dizin}/${paket}/files /tmp/bsp/build/files/
    cp -prfv $PACKAGEDIR/files $SOURCEDIR
    cd $SOURCEDIR

    autoreconf -vfi
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/ \
        --sysconfdir=/etc
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
    install -m755 -D $SOURCEDIR/files/Xwrapper.config "$DESTDIR"/etc/X11/Xwrapper.config
    install -m755 -D $SOURCEDIR/files/xinitrc "$DESTDIR"/etc/X11/xinit/xinitrc
    install -m755 -D $SOURCEDIR/files/Xsession "$DESTDIR"/etc/X11/xinit/Xsession
    install -m755 $SOURCEDIR/files/xserverrc "$DESTDIR"/etc/X11/xinit/xserverrc
    install -m755 -D $SOURCEDIR/files/xinit.sysconf "$DESTDIR"/etc/sysconf.d/xinit
    install -m755 -D $SOURCEDIR/files/xinit.initd "$DESTDIR"/etc/init.d/xinit
    install -m755 -D $SOURCEDIR/files/xinit.conf.d "$DESTDIR"/etc/conf.d/xinit
    install $SOURCEDIR/files/Xresources "$DESTDIR"/etc/X11/Xresources
    # allow local connections config
    mkdir -p "$DESTDIR"/etc/X11/xinit/xinitrc.d
    echo "#!/bin/sh" > "$DESTDIR"/etc/X11/xinit/xinitrc.d/10-local.sh
    echo "xhost +local:" >> "$DESTDIR"/etc/X11/xinit/xinitrc.d/10-local.sh
    chmod 754 "$DESTDIR"/etc/X11/xinit/xinitrc.d/10-local.sh
}
```

Ek dosyaları indirmek için [tıklayınız](#).

xcalc

X11 (X Pencere Sistemi) üzerinde çalışan basit bir grafiksel hesap makinesi uygulamasıdır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xcalc"
version="1.1.1"
description="scientific calculator for X"
source="https://gitlab.freedesktop.org/xorg/app/xcalc/-/archive/\
xcalc-$version/xcalc-xcalc-$version.tar.gz"
depends="libXaw,libICE,libSM,libXau,libXau,libXext,libXi,libXmu,libXrender,libXt,libxcb"
group="x11.apps"

setup(){
    cd $SOURCEDIR
    autoreconf -fvi
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libXi

X11 (X Pencere Sistemi) için X Input Extension (Giriş Uzantısı) işlevlerini sağlayan bir kütüphanedir. X istemcilerinin klavye, fare, dokunmatik ekran gibi giriş aygıtlarına erişmesini ve bu aygıtlarla etkileşime geçmesini sağlar.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libXi"
version="1.8"
description="X.Org Xi library"
source="https://www.x.org/archive/individual/lib/libXi-$version.tar.gz"
depends="libXfixes"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

openbox

X11 için geliştirilmiş, hafif, esnek ve yüksek derecede özelleştirilebilir bir pencere yöneticisidir.

- openbox, yalnızca pencere yönetiminden sorumludur, yani masaüstü ortamı (desktop environment) değildir.
- Uygulamalar arasında pencere geçişi, konumlandırma, boyutlandırma ve dekorasyon işlevlerini sağlar.
- Özellikle düşük sistem kaynaklı bilgisayarlarda tercih edilir.

Paketi Derleme :

```
#!/usr/bin/env bash
name="openbox"
version="3.6.1"
description="Highly configurable and lightweight X11 window manager"
source="http://openbox.org/dist/openbox/openbox-$version.tar.xz"
depends="libSM,libxml2,pango,startup-notification,libXrandr,librsvg,libXinerama"
group="x11.wm"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --sysconfdir=/etc \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libXcursor

X11 (X Pencere Sistemi) için fare imleci (cursor) yönetimini geliştiren bir kütüphanedir. Özellikle şeffaflık, ölçeklenebilirlik ve tema desteği gibi modern özellikleri sağlayarak klasik X imleç sistemini geliştirir.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libXcursor"
version="1.2.1"
description="X.Org Xcursor library"
source="https://www.x.org/archive/individual/lib/libXcursor-$version.tar.xz"
depends="libXrender,libXfixes,libX11"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libXfixes

X11 (X Pencere Sistemi) için geliştirilen Xfixes uzantısının (X Fixes Extension) istemci tarafı kütüphanesidir. Bu uzantı, X11 protokolüne çeşitli küçük ama önemli iyileştirmeler getirir ve bu iyileştirmeleri uygulamaların kullanabilmesini sağlar.

Sağladığı Başlıca Özellikler:

- İmleç gizleme ve değiştirme: Uygulamalar imleci gizleyebilir veya özel bir görünümle değiştirebilir.
- Bölge yönetimi (Region handling): Ekranın sadece belirli bölgelerinde yeniden çizim yapılmasını sağlar (örneğin pencere efektleri için).
- Selection notifikasyonu: Kopyala/yapıştır işlemlerinde seçim değişikliklerini takip etme.
- Damage tracking: Pencerenin hangi kısımlarının değiştiğini takip etmeye yardımcı olur (Compositor'lar için önemlidir).

Paketi Derleme :

```
#!/usr/bin/env bash
name="libXfixes"
version="6.0.0"
description="X.Org Xfixes library"
source="https://www.x.org/archive/individual/lib/libXfixes-$version.tar.gz"
depends=""
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

pango

Modern uygulamalar için tasarlanmış bir metin yerleşim ve işleme (text layout and rendering) kütüphanesidir. Özellikle çok dilli ve uluslararası metinleri doğru ve düzgün biçimde çizmek için kullanılır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="pango"
version="1.51.0"
description="Layout and render international text"
source="https://download.gnome.org/sources/pango/1.51/pango-$version.tar.xz"
depends="fribidi,libthai,harfbuzz,cairo,libXft"
makedepend="gobject-introspection"
group="x11.libs"

setup(){
cd $SOURCEDIR
meson setup $BUILDDIR --prefix=/usr \
--libdir=/usr/lib64/ \
-Dintrospection=enabled \
-Dxft=enabled \
-Dcairo=enabled \
-Dlibthai=enabled \
-Dfreetype=enabled \
-Dgtk_doc=false
}

build(){
ninja -C $BUILDDIR
}

package(){
DESTDIR=$DESTDIR ninja -C $BUILDDIR install
}
```


libXrandr

X11 sisteminde çalışan uygulamalara, ekran çözünürlüğü ve yönü gibi ekran yapılandırmalarını dinamik olarak değiştirme yeteneği kazandıran RandR (Resize and Rotate) uzantısının istemci kütüphanesidir.

- Çözünürlük değiştirme: Monitör çözünürlüğü uygulama içinden değiştirilebilir.
- Ekran yönü: Ekran döndürme (örneğin dikey moda geçme) yapılabilir.
- Çoklu monitör desteği: Aynı anda birden fazla ekranı yapılandırabilir; konumlarını, çözünürlüklerini ve hizalamalarını değiştirebilir.
- Olay takibi: Ekran bağlantı/ayırılma olaylarını dinleyebilir (örneğin dizüstü bilgisayara ikinci ekran bağlandığında algılama).

Paketi Derleme :

```
#!/usr/bin/env bash
name="libXrandr"
version="1.5.3"
description="X.Org Xrandr library"
source="https://www.x.org/archive/individual/lib/libXrandr-$version.tar.xz"
depends="libXrender"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

fribidi

Tam adıyla **GNU FriBidi**, Unicode metinlerde sağdan sola (RTL - Right-To-Left) yazım desteği sağlayan bir bi-directional (bidi) metin işleme kütüphanesidir. Özellikle Arapça, İbranice, Farsça gibi sağdan sola yazılan dillerin düzgün görüntülenmesi için kullanılır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="fribidi"
version="1.0.12"
description="A free implementation of the unicode bidirectional algorithm"
source="https://github.com/fribidi/fribidi/releases/download/\
v$version/fribidi-$version.tar.xz"
depends=""
builddepend="meson"
group="dev.libs"

setup(){
    cd $SOURCEDIR
    ./configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

xcb-util

X11 için geliştirilmiş bir genel yardımcı araçlar kütüphanesidir ve XCB (X C Binding) API'si ile uyumlu çalışan çeşitli araçlar sunar. xcb-util kütüphanesi, X11 uygulamalarının daha esnek ve fonksiyonel olmasını sağlayan birçok küçük ama önemli işlevi içerir.

- Yüksek seviyeli X11 işlevleri: xcb-util, libxcb'nin sunduğu temel işlevlikten daha yüksek seviyede işlevler sağlar.
- Pencere yöneticisi eklentileri: Örneğin, pencere yöneticileri için kullanılan bazı ekstra özellikler (xprop gibi) sağlar.
- X11 protokolü genişletme: X11 sunucusunun bazı protokollerine daha kolay erişim sağlayan işlevler sunar.
- Daha hızlı XCB kullanımı: XCB'nin daha düşük seviyeli işlevlerini daha hızlı ve kolay bir şekilde kullanma olanağı tanır.

Paketi Derleme :

```
#!/usr/bin/env bash
name="xcb-util"
version="0.4.0"
description="X C-language Bindings sample implementations"
source="https://www.x.org/releases/individual/xcb/xcb-util-${version}.tar.gz"
depends="libxcb,util-macros,xorgproto"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64 \
        --enable-shared \
        --enable-static \
        --disable-devel-docs \
        --without-doxygen
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libthai

Tayland yazı sistemini (Thai script) işlemek ve düzenlemek için kullanılan bir C kütüphanesidir. Özellikle Tayland alfabesinde yazılmış metinleri işlemek için geliştirilmiştir ve Unicode destekli metin işleme sağlar.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libthai"
version="0.1.29"
description="Package libthai"
source="https://github.com/tlwg/libthai/releases/download/\
v$version/libthai-$version.tar.xz"
depends="libdatrie"
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

libdatrie

Trie (prefix tree) veri yapısını kullanan bir C kütüphanesidir. Hızlı arama, önek eşleşmesi (prefix matching) ve sıralama gibi işlemleri gerçekleştiren veritabanlarında ve metin işleme uygulamalarında kullanılır.

- Prefix arama: Trie yapısı sayesinde verilen bir önek (prefix) ile başlayan kelimeleri hızlı bir şekilde bulabilirsiniz.
- Sıralama: Trie yapısındaki veriler doğal sıralıdır, bu yüzden veri sıralaması sağlanabilir.
- Hızlı arama: Trie, özellikle çok büyük veri kümesinde yapılan aramaları çok hızlı hale getirir.
- Bellek verimliliği: Trie yapısında depolanan veriler sıkıştırılabilir, bu da bellek kullanımını optimize eder.

Paketi Derleme :

```
#!/usr/bin/env bash
name="libdatrie"
version="0.2.13"
description="Implementation of double-array structure for\
    representing trie, as proposed by Junichi Aoe."
source="https://github.com/tlwg/libdatrie/releases/download/\
v$version/libdatrie-$version.tar.xz"
depends=""
group="x11.libs"

setup(){
    $SOURCEDIR/configure --prefix=/usr \
        --libdir=/usr/lib64/
}

build(){
    make
}

package(){
    make install DESTDIR=$DESTDIR
}
```

ISO Hazırlama

Dağıtım özgü ISO oluşturmak için o dağıtımın paket sistemi kullanılır. Bu dokümanda anlatılan **kly** paket sistemi, önceki bölümde detaylıca açıklanmıştır.

Bu bölümde, **kly** paket sistemini kullanarak ISO oluşturma adımları basit ve anlaşılır şekilde verilecektir. Aşağıda paylaşılan script ile hazırlanan iso **/tmp/distro/distro.iso** konumunda olacaktır. Sistemi oluşturan paketleri **github.com/kendilinuxunuyap/kly-binary-packages** konumundan indirerek oluşturmaktadır.

Not: Anlatılan yöntem genel Linux dağıtımlarının ISO oluşturma mantığına dayanır. Araçlar, dizinler ve komutlarda küçük farklılıklar olabilir, ancak iş akışı aynıdır.

kly Paket Sistemiyle ISO Oluşturma Scripti

```
#-----
#!/bin/bash
rootfs="/tmp/distro/rootfs"
distro="/tmp/distro"
rm -rf $rootfs
mkdir -p $rootfs -p $rootfs/bin $rootfs/etc
##Kurulum scripti
cp -prf files/bin/* $rootfs/bin/
cp -prf files/etc/* $rootfs/etc/

# temel izinler ve dosyalar oluşturuluyor
cd $rootfs/
mkdir -p bin dev etc home lib64 proc root run sbin sys usr var etc/kly \
tmp tmp/kly/kur var/log var/tmp usr/lib64/x86_64-linux-gnu \
usr/lib64/pkgconfig usr/local/{bin,etc,games,include,lib,sbin,share,src}
ln -s lib64 lib
cd var&&ln -s ../run run&&cd -
cd usr&&ln -s lib64 lib&&cd -
cd usr/lib64/x86_64-linux-gnu&&ln -s ../pkgconfig pkgconfig&&cd -

echo -e "/bin/sh\n/bin/bash\n/bin/rbash\n/bin/dash" >> "$rootfs/etc/shell"
echo "tmpfs /tmp tmpfs rw,nodev,nosuid 0 0" >> "$rootfs/etc/fstab"
echo "127.0.0.1 kly" >> "$rootfs/etc/hosts"
echo "kly" > "$rootfs/etc/hostname"
echo "nameserver 8.8.8.8" > "$rootfs/etc/resolv.conf"

# paket adresi ekleniyor
echo "kendilinuxunuyap/kly-binary-packages">$rootfs/etc/kly/sources.list

### installing kly create_package in rootfs
echo root:x:0:0:root:/root:/bin/sh > $rootfs/etc/passwd
chmod 755 $rootfs/etc/passwd

### system chroot bind/mount
for dir in dev dev/pts proc sys; do mount -o bind /$dir $rootfs/$dir; done
## paket listesi güncelleniyor
$rootfs/bin/klyupdate $rootfs

## paketler kuruluyor
for paket in glibc readline ncurses \bash openssl acl attr libcap libpcr2 gmp \
coreutils util-linux \grep \sed mpfr \gawk findutils libgcc libcap-ng sqlite \
\gzip xz-utils zstd \bzip2 \elfutils libselinux \tar \zlib brotli curl shadow \
\file eudev cpio libsepol kmod audit libxcrypt libnsl pam libtirpc e2fsprogs \
dosfstools initramfs-tools libxml2 expat libmd libaio lvm2 popt icu iproute2 \
net-tools dhcp openrc rsync kbd busybox kernel kernel-headers live-boot \
live-config parted nano grub dialog efibootmgr efivar libssh openssl
do
chroot $rootfs /bin/klykur $paket;
done

#### system chroot umount
for dir in dev dev/pts proc sys ; do while umount -lf -R $rootfs/$dir \
2>/dev/null ; do true; done done
exit
```

```

#-----
# scriptin devamı
chroot $rootfs useradd live -m -s /bin/sh -d /home/live
chroot $rootfs echo -e "live\nlive\n"|chroot $rootfs passwd live

for grp in users tty wheel cdrom audio dip video plugdev netdev; do
    chroot $rootfs usermod -aG $grp live || true
done

sed -i "/agetty_options/d" $rootfs/etc/conf.d/agetty
echo -e "\nagetty_options=\"-l /usr/bin/login\"" >> $rootfs/etc/conf.d/agetty

### update-initrd
fname=$(basename $rootfs/boot/config*)
kversion=${fname:7}
mv $rootfs/boot/config* $rootfs/boot/config-$kversion
cp $rootfs/boot/config-$kversion $rootfs/etc/kernel-config

chroot $rootfs update-initramfs -u -k $kversion
#### system chroot umount
for dir in dev dev/pts proc sys ;
do while umount -lf -R $rootfs/$dir 2>/dev/null ; do true; done done

mkdir -p "$distro/iso" "$distro/iso/boot" "$distro/iso/boot/grub"
mkdir -p "$distro/iso/live"
#### Copy kernel and initramfs
cp -pf $rootfs/boot/initrd.img-* $distro/iso/boot/initrd.img
cp -pf $rootfs/boot/vmlinuz-* $distro/iso/boot/vmlinuz
rm -rf $rootfs/boot

#### Create squashfs
mksquashfs $rootfs $distro/filesystem.squashfs -comp xz -wildcards
mv $distro/filesystem.squashfs $distro/iso/live/filesystem.squashfs

#### Write grub.cfg
cat << EOF > "$distro/iso/boot/grub/grub.cfg"
set timeout=3 # Timeout for menu
set default=1 # Default boot entry

# Menu Colours
set menu_color_normal=white/black
set menu_color_highlight=white/blue
insmod all_video
terminal_output console
terminal_input console
menuentry "Canli(live) GNU/Linux 64-bit" --class liveiso {
    linux /boot/vmlinuz boot=live init=/sbin/openrc-init net.ifnames=0 \
    biosdevname=0
    initrd /boot/initrd.img
}
menuentry "Kur GNU/Linux 64-bit" --class liveiso {
    linux /boot/vmlinuz boot=live init=/bin/kur quiet
    initrd /boot/initrd.img
}
EOF
grub-mkrescue $distro/iso/ -o $distro/distro.iso

```

Kaynaklar:

- <https://www.subrat.info/build-kernel-and-userspace/> - 08/07/2025
- <https://medium.com/@chienhaotan/compiling-and-running-a-minimal-kernel-with-busybox-bfc45a991017> - 08/07/2025
- [https://wiki.archlinux.org/title/GRUB_\(T%C3%BCrk%C3%A7e\)](https://wiki.archlinux.org/title/GRUB_(T%C3%BCrk%C3%A7e)) - 08/07/2025
- <https://chatgpt.com/share/6875084c-6050-8012-9229-a37b47351aa2> - 14/07/2025

Sistem Kurulumu

Sistem Kurma

Hazırlanan ISO ile birlikte, farklı kurulum araçları da gelebilir. Bu araçlar, çeşitli kurulum yöntemlerini destekleyebilir. En sık kullanılan kurulum yöntemleri şunlardır:

1. Tek bölüme sistem kurulumu
2. UEFI sistem kurulumu (boot + sistem)

Bu bölümde, tek bölüm kurulum ve boot + sistem şeklinde iki farklı kurulum yöntemi sırayla anlatılacaktır. Anlatılan yöntemler, farklı kullanıcı senaryolarına cevap verebilecek şekilde tasarlanmıştır.

Ancak dikkat edilmesi gereken önemli bir nokta şudur: Her yöntemi ayrı ayrı son kullanıcıya seçenek olarak sunmak, özellikle tecrübesiz kullanıcılar için kafa karıştırıcı olabilir. Örneğin:

- Kullanıcı, kurulum sırasında **EFI** seçeneğini işaretlediğinde fakat sistem aslında **Legacy BIOS** ise, ya da tam tersi durumda, yanlış seçim yapabilir.
- Ayrıca, **sda** ve **nvme** diskler arasında farklı kurulum senaryoları gerekebilir.

Bu nedenle hazırlanan kurulum sistemi, aşağıdaki tüm olası senaryolara otomatik cevap verecek şekilde tasarlanmıştır:

- **Legacy BIOS** → **sd*** disk tipi
- **Legacy BIOS** → **nvme*** disk tipi (**desteklenmemektedir**)
- **UEFI** → **sd*** disk tipi
- **UEFI** → **nvme*** disk tipi

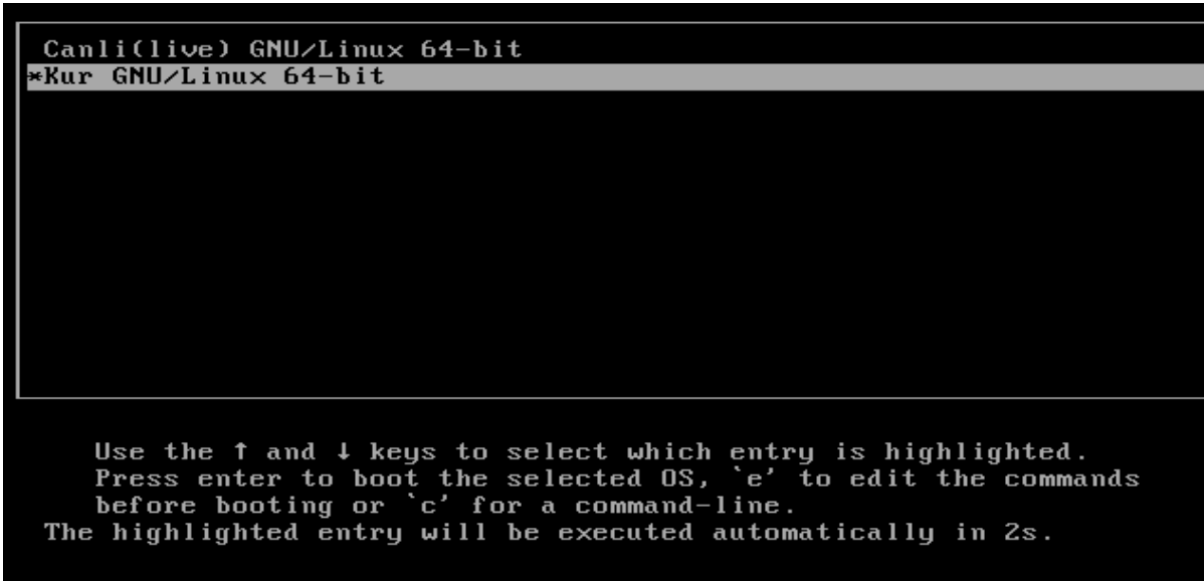
Not

Bu senaryolara göre hazırlanan kurulum scripti, **dialog** aracı kullanılarak oluşturulmuştur. İlgili kurulum scriptleri, **base-file** paketinin içindeki **files.rar** arşivinde yer almaktadır.

Bu bölümde yalnızca **Legacy BIOS** ve **UEFI** sistem kurulumunun genel adımları anlatılacaktır. Bu yöntemleri kendi ihtiyaçlarınıza göre düzenleyerek kullanabilirsiniz.

Sistem Kurulumu

Sistemin kurulumu için resimlerde görünen sıraya göre seçimler yapmalıyız.



Kurulum menüsünde kullanıcı adları ve parolaları, klavye varsayılan olarak;

- Kullanıcı: root Parola: 1
- Kullanıcı: user1 Parola: 1
- Dil : tr_TR
- Klavye : trq

menüden değişiklik yapabilirsiniz. Değişiklik yapmadan sadece kurulum diskini ve disk bölümünü seçip Install(Yükle) işlemi yapabilirsiniz.



```

kurulumu geçildi.....
legacy kurulum .....
Kurulum Yapılacak Disk sda
mount: /kaynak: WARNING: source write-protected, mounted read-only.
*****disk bölümleri biçimlendiriliyor *****
e2fsck 1.47.0 (5-Feb-2023)
e2fsck: need terminal for interactive repairs
tune2fs 1.47.0 (5-Feb-2023)
Recovering journal.

This operation requires a freshly checked filesystem.
Please run e2fsck -f on the filesystem.

mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 2096896 4k blocks and 524288 inodes
Filesystem UUID: 9ea082d0-a034-4dab-8e2f-564e68c99c9c
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

Information: You may need to update /etc/fstab.

***** kurulum başladı*****
Sistem yüklenmeye başlandı. Tahmini 3-5 dakika sürecektir.. Lütfen bekleyiniz.....
.....

```

Kaynak:

- [https://wiki.archlinux.org/title/GRUB_\(T%C3%BCrk%C3%A7e\)#UEFI_sistemler](https://wiki.archlinux.org/title/GRUB_(T%C3%BCrk%C3%A7e)#UEFI_sistemler) - 08/07/2025
- https://tldp.org/HOWTO/html_single/SquashFS-HOWTO/ - 09/07/2025
- <https://askubuntu.com/questions/437880/extract-a-squashfs-to-an-existing-directory> - 11/07/2025
- https://grok.com/share/c2hhcmQtMw%3D%3D_2ad2ac6b-d067-40b0-9b55-46db0c2c98dc - 10/07/2025

X Pencere Sistemi

Xorg Çalıştırma

Xorg Yapılandırma Dosyası

Xorg, genellikle /etc/X11/xorg.conf dosyasını kullanır. Eğer bu dosya yoksa, Xorg varsayılan ayarlarla çalışacaktır. Ancak, özel ayarlar yapmak istiyorsanız, bu dosyayı oluşturmanız gerekebilir. Aşağıdaki komut ile bir yapılandırma dosyası oluşturabilirsiniz:

```
sudo X -configure
```

Bu komut, xorg.conf.new adında bir dosya oluşturacaktır. Bu dosyayı /etc/X11/xorg.conf olarak kopyalayabilirsiniz:

```
sudo cp ~/xorg.conf.new /etc/X11/xorg.conf
```

Xwrapper.config

```
echo needs_root_rights=yes>/etc/X11/Xwrapper.config  
echo allowed_users=anybody>>/etc/X11/Xwrapper.config
```

Kullanıcı Ayarı

Kullanıcının tty ve wheel grubunda olması lazım ayrıca **/dev/tty*** dosyalarının grub ve izinleri ayarlanmalıdır. ... code-block:: shell

```
chmod 620 /dev/tty* chgrp tty /dev/tty* usermod -a -G tty by addgroup wheel usermod -a -G wheel by
```

Xorg'u Elle Başlatma

Xorg'u elle başlatmak için terminalde aşağıdaki komutu kullanabilirsiniz:

```
export DISPLAY=:0  
Xorg:0
```

Bu şekilde çalıştırmak yerine;

```
startx
```

Bu komut, varsayılan kullanıcı arayüzünü başlatacaktır. Eğer belirli bir pencere yöneticisi veya masaüstü ortamı ile başlatmak istiyorsanız, ~/.xinitrc dosyasını düzenlemeniz gerekebilir. Örneğin, openbox kullanmak istiyorsanız, dosyanın içeriği şöyle olmalıdır:

```
exec openbox-session
```

Xorg'u Durdurma

Xorg'u durdurmak için, terminalde Ctrl + Alt + Backspace tuş kombinasyonunu kullanabilirsiniz. Bu, X sunucusunu kapatacaktır.

Xorg Hata ayıklama

Hata Ayıklama

Eğer Xorg başlatılamıyorsa, hata ayıklamak için aşağıdaki komutu kullanarak log dosyalarını kontrol edebilirsiniz:

```
cat /var/log/Xorg.0.log
```

Bu dosya, Xorg'un başlatılması sırasında oluşan hataları ve uyarıları içerecektir. Hataları inceleyerek sorunu çözmeye çalışabilirsiniz.

Yardımcı Konular

ssh ile Sanal Makinaya Bağlanma

Bir sanal makineye (**VM**) SSH üzerinden bağlanmak, uzak sistem yönetimi için yaygın ve güvenli bir yöntemdir. Aşağıdaki adımları takip ederek sanal makinana kolayca bağlanabilirsin.

1. SSH Sunucusunun Kurulu ve Çalışır Olmalıdır

Temel Sistem:

```
klykur openssh  
rc-update add sshd  
rc-service sshd start
```

2. Sanal Makina IP Adresi

SanalMakina içerisinde aşağıdaki komutlardan biriyle IP adresini öğrenilir:

```
ip a
```

veya

```
hostname -I
```

Örnek IP adresi:

```
192.168.1.101
```

3. sshd_config Dosyasını Düzenle

/etc/ssh/sshd_config config dosyasını nano ile açıp,

```
nano /etc/ssh/sshd_config
```

#PermitRootLogin prohibit-password satırını

PermitRootLogin yes olarak değiştirilir.

4. SSH ile Bağlantı Kur

Ana sistemden terminal açarak aşağıdaki komut ile SSH bağlantısı yapılır:

```
ssh kullanıcı_adı@ip_adresi
```

Örnek:

```
ssh kly@192.168.1.101
```

İlk bağlantıda şu uyarıyı verebilir:

```
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

yes yazıp enter'a basılıp, şifreyi girildiğinde **SSH** bağlantısı hazır olur.

sftp ile Sanal Makinaya Bağlanma

SFTP (Secure File Transfer Protocol), SSH protokolü üzerinden güvenli dosya transferi yapmayı sağlar. Bir sanal makineye bağlanarak dosya alışverişi yapmak için aşağıdaki adımları takip edebilirsiniz.

1. SSH Sunucusunun Kurulu ve Çalışır Olmalıdır

SFTP, SSH servisi üzerinden çalışır. Sanal makinede SSH sunucusunun yüklü ve aktif olması gerekir.

```
sudo rc-status sshd
```

Eğer kurulu değilse:

```
klykur openssh
```

2. Sanal Makinanın IP Adresini Öğren

Sanal makine içinde aşağıdaki komutlardan biriyle IP adresini öğrenilir:

```
ip a
```

veya

```
hostname -I
```

Örnek IP adresi:

```
192.168.1.101
```

3. SFTP ile Bağlantı Kur

Ana makineden terminal açarak aşağıdaki komut ile SFTP bağlantısı başlatılır:

```
sftp kullanıcı_adı@ip_adresi
```

Örnek:

```
sftp kly@192.168.1.101
```

Bağlandıktan sonra şu ekranı görürsünüz:

```
sftp>
```

Aşağıdaki komutları kullanabilirsiniz:

- **ls** : Uzak sunucudaki dizin içeriğini listeler.
- **lcd /yerel/yol** : Yerel sistemdeki dizini değiştirir.
- **cd /uzak/yol** : Uzak sunucudaki dizini değiştirir.
- **get dosya.txt** : Uzak dosyayı indirir.
- **put dosya.txt** : Yerel dosyayı sunucuya yükler.

- **exit** : Bağlantıyı sonlandırır.

4. Grafik Arayüz SFTP Alternatifleri

Linux ortamlarında, dosya yöneticileri ve FTP programları ile de SFTP bağlantısı kurulabilir.

GNOME Nautilus / Cinnamon Nemo ile SFTP bağlantısı için:

Adres çubuğuna bağlantı bilgisi şu formatta yazılır:

```
sftp://kly@192.168.1.101
```

FileZilla gibi bir ftp programı ile aşağıdaki ayarlar kullanılarak SFTP bağlantısı yapılır:

- Host: *sftp://192.168.1.101*
- Username: *kly*
- Password: şifre
- Port: 22

Not: Sanal makinenizin ağ ayarlarının (NAT veya Bridged) ve güvenlik duvarı yapılandırmasının doğru olması gerekir.

scp ile Sanal Makinaya Dosya ve Dizin Kopyalama

SCP (Secure Copy Protocol), SSH üzerinden güvenli şekilde dosya ve izin kopyalamak için kullanılan bir komut satırı aracıdır.

Sanal makinaya dosya veya izin göndermek için aşağıdaki yöntemler kullanılabilir.

1. Dosya Kopyalama

Yerel makinedeki bir dosyayı sanal makinaya kopyalamak için:

```
scp /yerel/yol/dosya.txt kullanıcı_adı@ip_adresi:/uzak/yol/
```

Örnek:

```
scp ~/Belgeler/rapor.pdf kly@192.168.1.101:/home/kly/
```

2. Dizin Kopyalama

Bir dizini ve içeriğini kopyalamak için `-r` (recursive) seçeneği kullanılır:

```
scp -r /yerel/yol/dizin kullanıcı_adı@ip_adresi:/uzak/yol/
```

Örnek:

```
scp -r ~/Projeler/myproject kly@192.168.1.101:/home/kly/
```

3. Sanal Makinadan Dosya veya Dizin Kopyalama

Ters yönde, sanal makinadan yerel makineye dosya veya izin kopyalamak için de benzer komut kullanılır:

```
scp kullanıcı_adı@ip_adresi:/uzak/yol/dosya.txt /yerel/yol/
```

```
scp kly@192.168.1.101:/home/kly/rapor.pdf ~/Belgeler/
```

veya izin için:

```
scp -r kullanıcı_adı@ip_adresi:/uzak/yol/dizin /yerel/yol/
```

```
scp -r kly@192.168.1.101:/home/kly/ ~/Projeler/myproject
```

4. Önemli Notlar

- Sanal makinenin IP adresi ve kullanıcı adı doğru olmalıdır.
- SSH servisi sanal makinada çalışıyor olmalıdır.
- Güvenlik duvarı ve ağ ayarları dosya transferine izin vermelidir.
- Kopyalama sırasında parola istenebilir veya SSH anahtarları kullanılabilir.

apt Paket Sistemi

apt paket sistemi debian tabanlı sistemlerde paket yönetimi için kullanılan bir uygulamadır. Temel işlemler şunlardır.

Yerel Paket İndexini Güncelleme

```
sudo apt update
```

Paket Yükleme

```
sudo apt install paket_adi
```

Paket Silme

```
sudo apt remove paket_adi
```

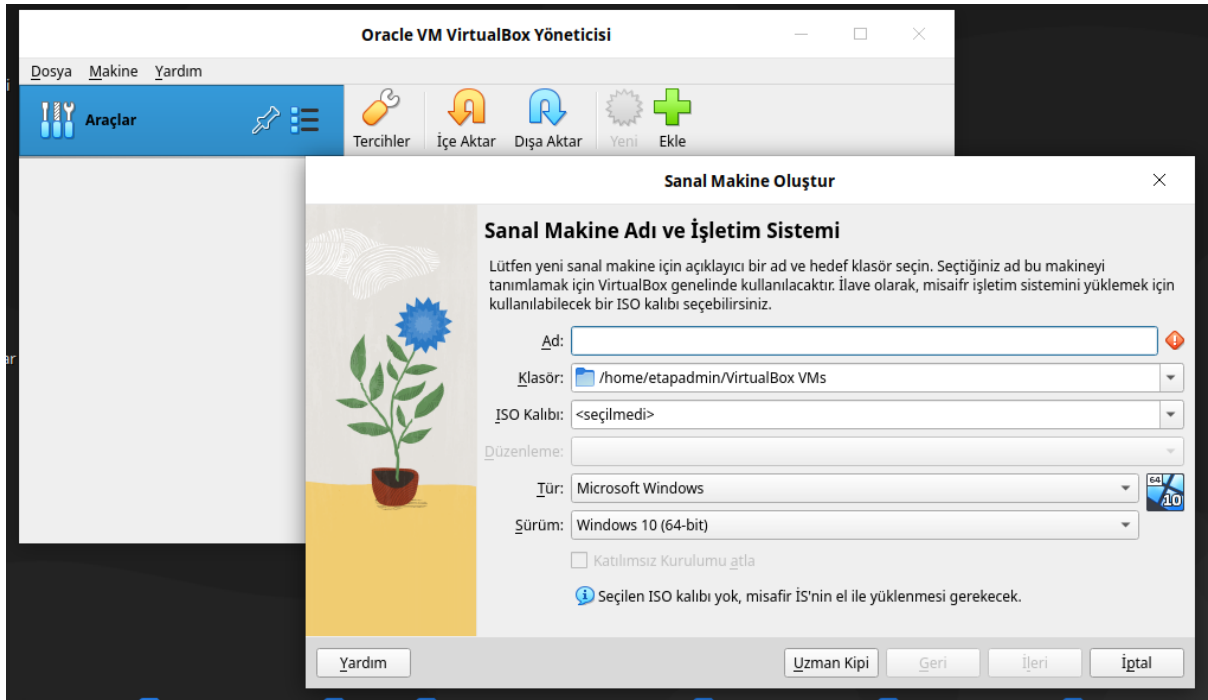
VirtualBox

VirtualBox, Oracle tarafından geliştirilen bir açık kaynaklı sanallaştırma yazılımıdır. Bilgisayarınızda çalışan bir işletim sisteminin (örneğin Windows, Linux, macOS) içinde başka bir işletim sistemi çalıştırmanıza izin verir. Bu çalıştırdığınız ikinci işletim sistemi (konuk/guest) kendi sanal donanımına sahipmiş gibi davranır.

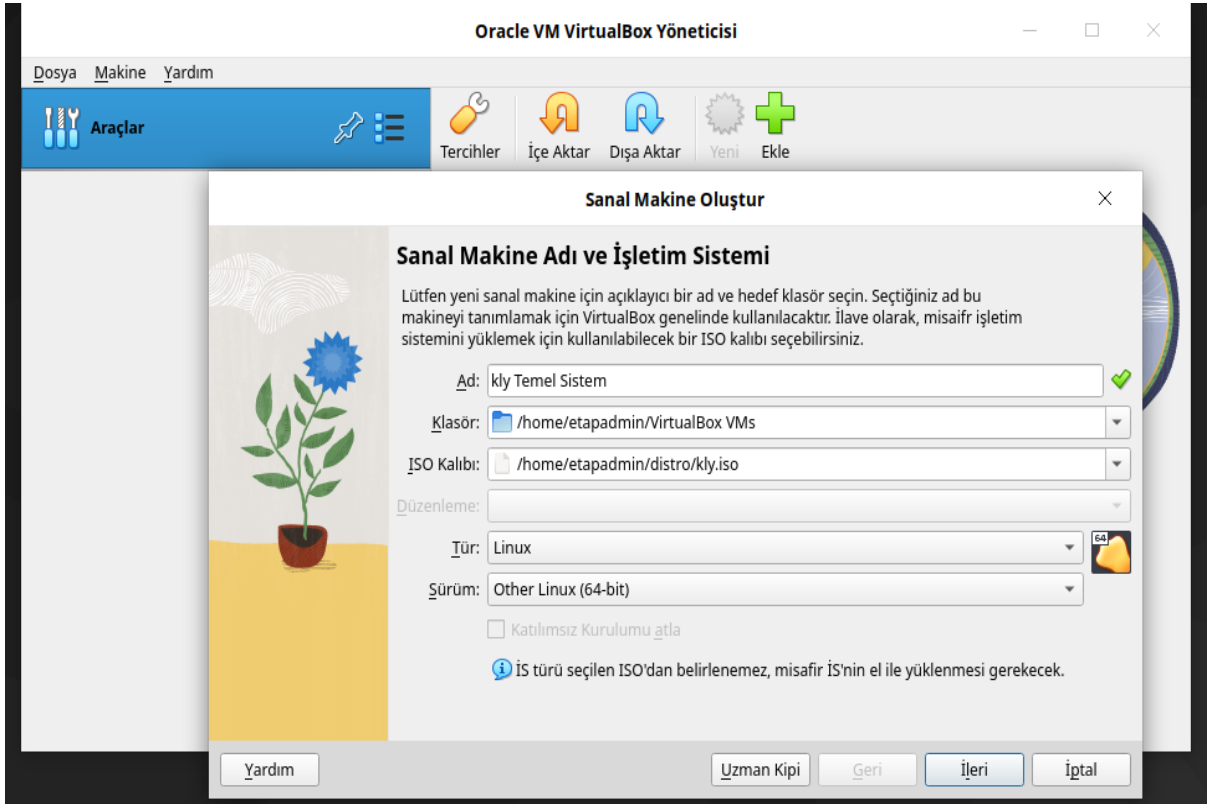
```
sudo apt update # index güncellenir
sudo apt install virtualbox-7.0 # Sistem virtualbox kurulur
```



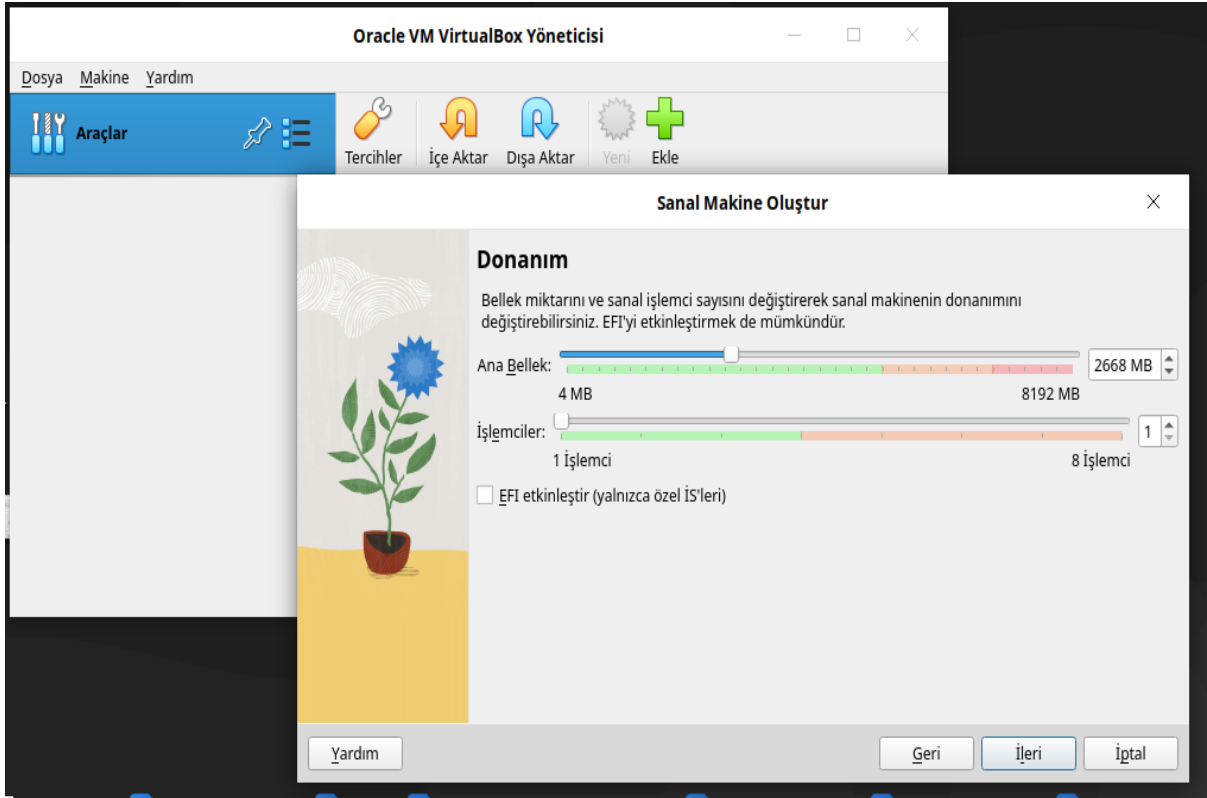
Ekle seçeneğini seçin. Aşağıdaki gibi bir ekran gelecektir.



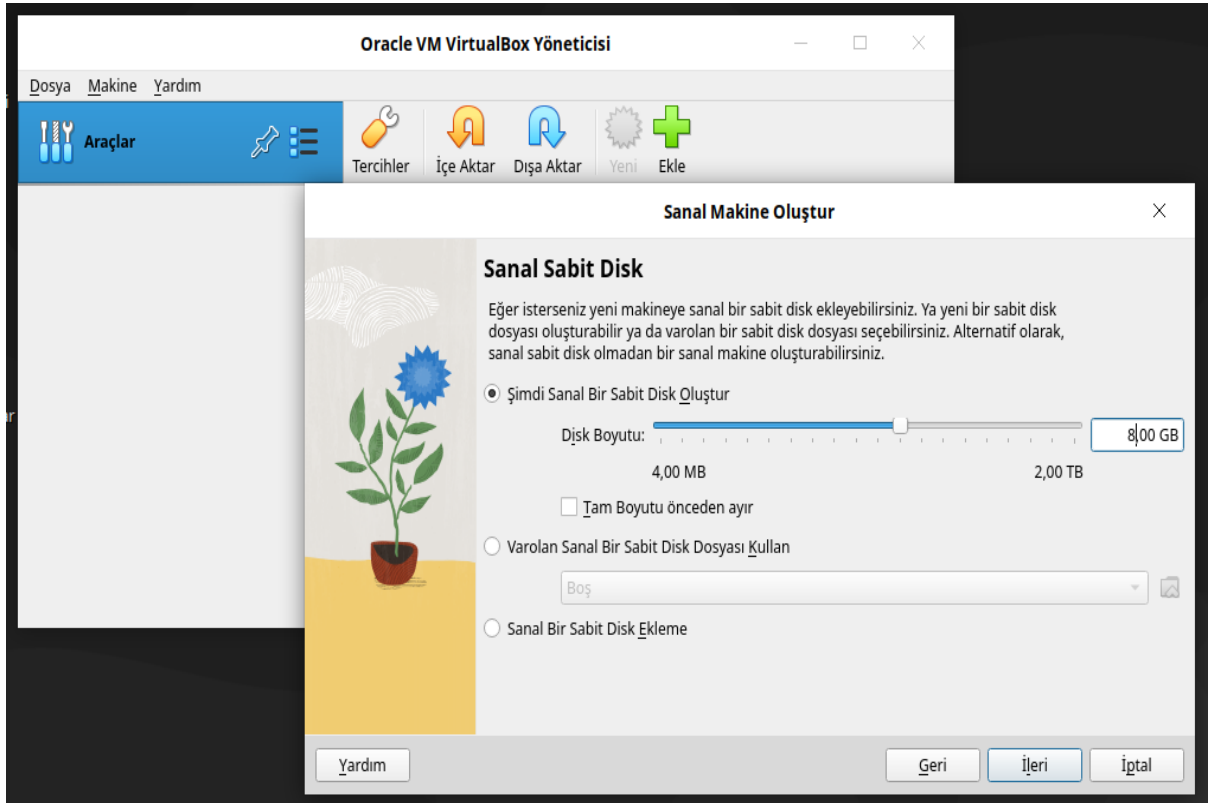
Aşağıdaki gibi seçenekleri doldurunuz.



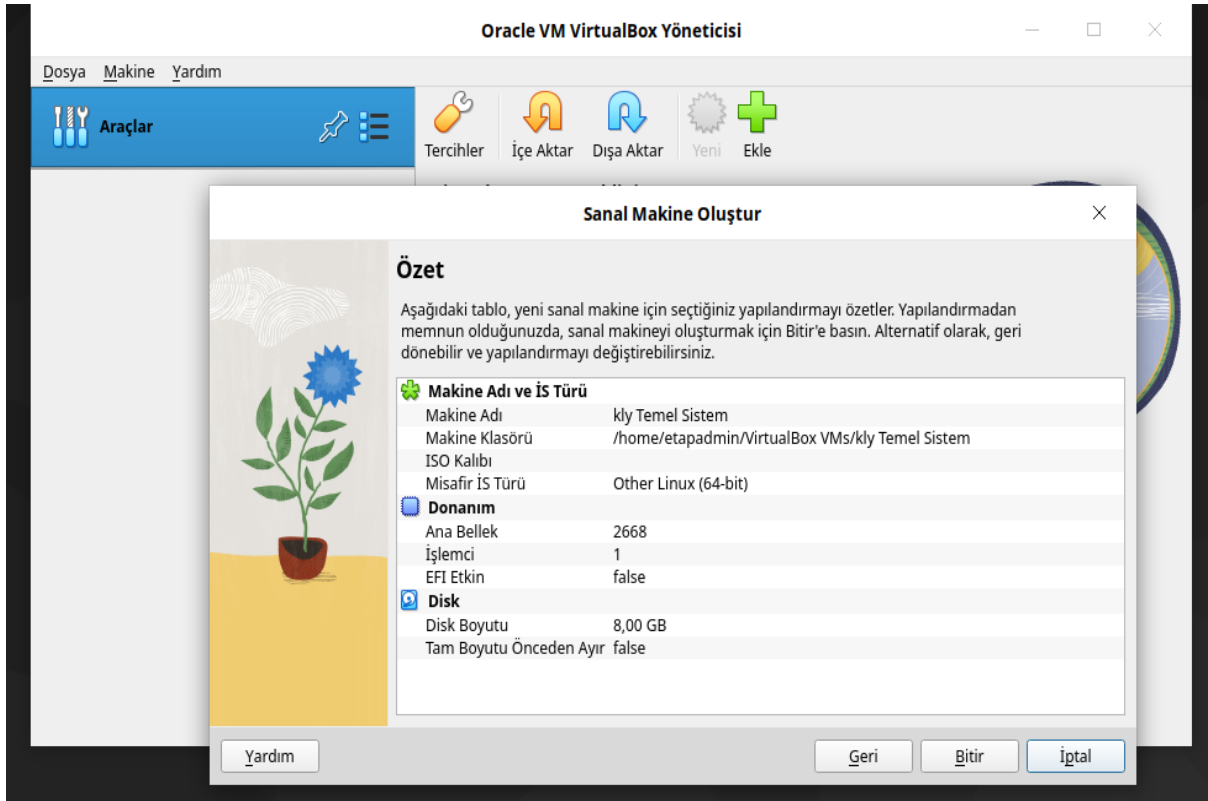
İleri seçeneği seçilir ve aşağıdaki gibi ekrana gelirsiniz. Bellek miktarını aşağıdaki gibi belirleyiniz. **İleri** seçeneği seçilir.



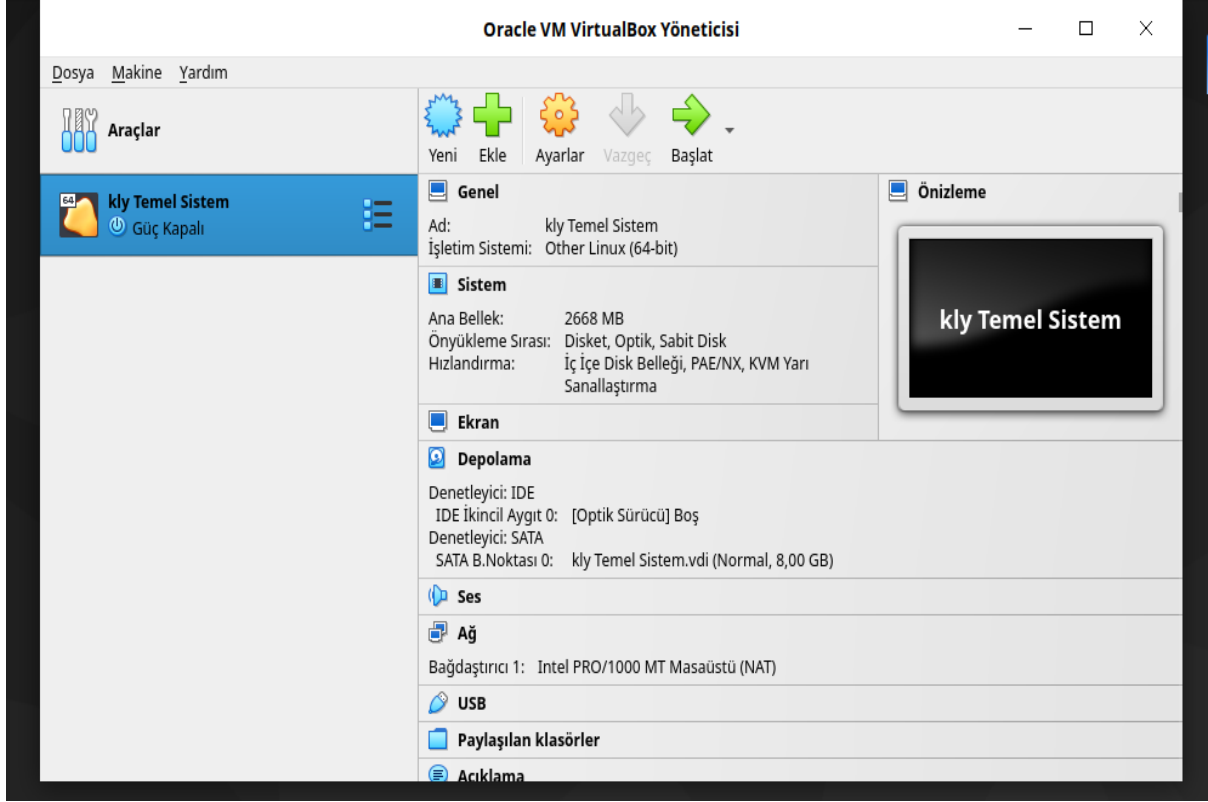
Disk boyutu belirlenir. 8GB bu sistem için yeterli. **İleri** seçeneği seçilir.



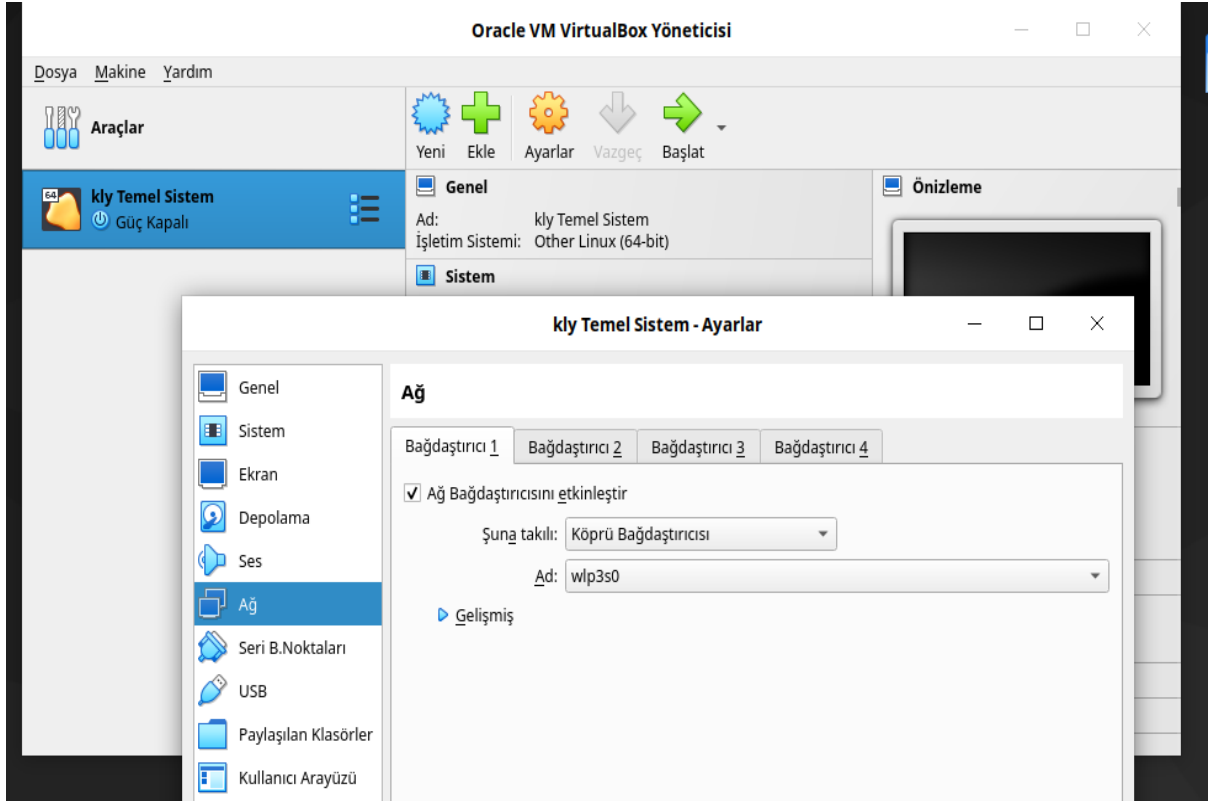
Tüm işlemler bitince aşağıdaki gibi pencere gelir. Bitir'i seçerek işlem tamamlanır.



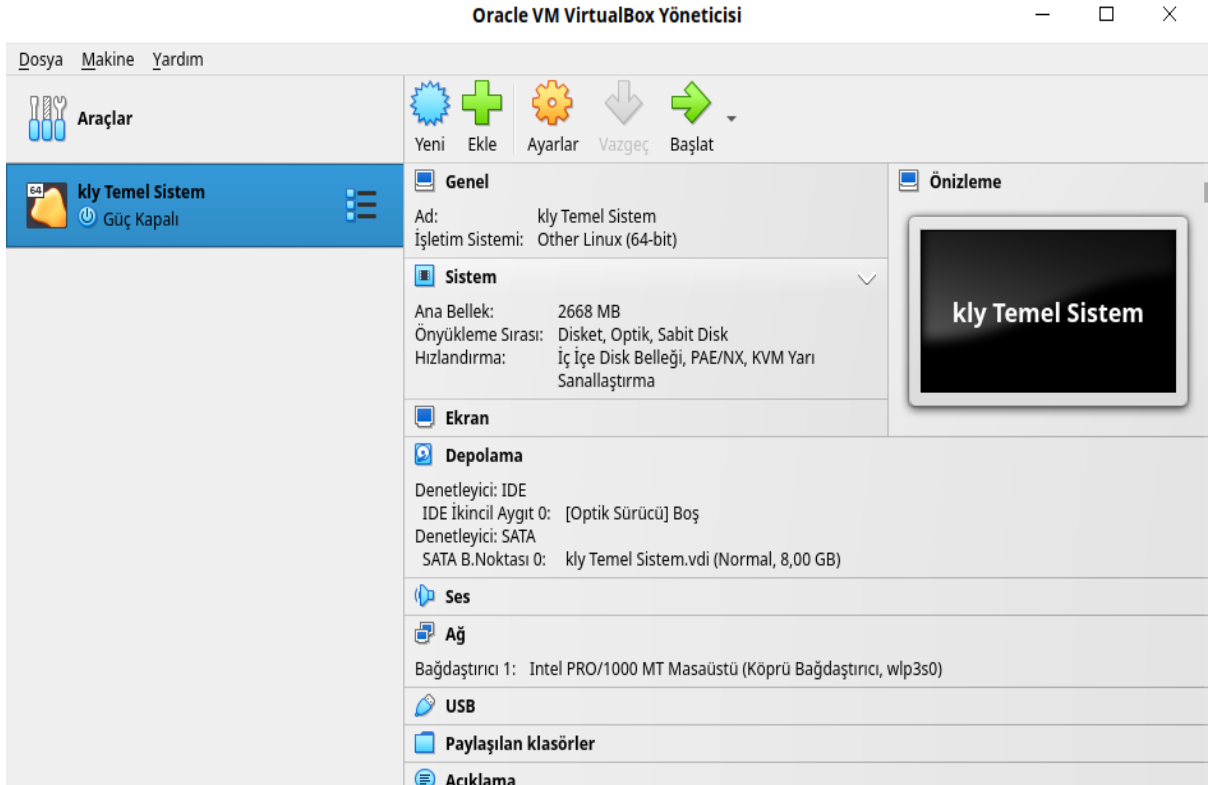
Bitir işlemi seçildikten sonra aşağıdaki gibi görünecektir.



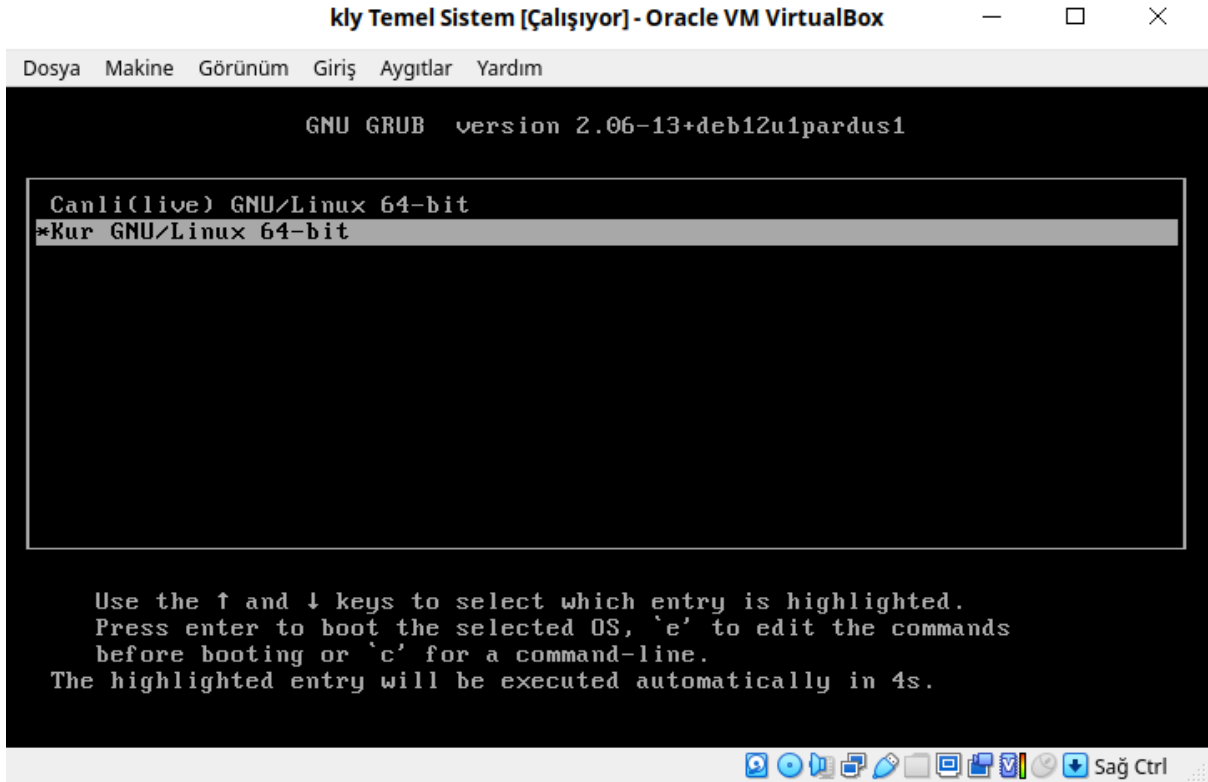
Ayarlar bölümünde aşağıdaki gibi **Ağ** ayarında değişiklik yapılır.



Başlat seçilerek sistem çalıştırılır.



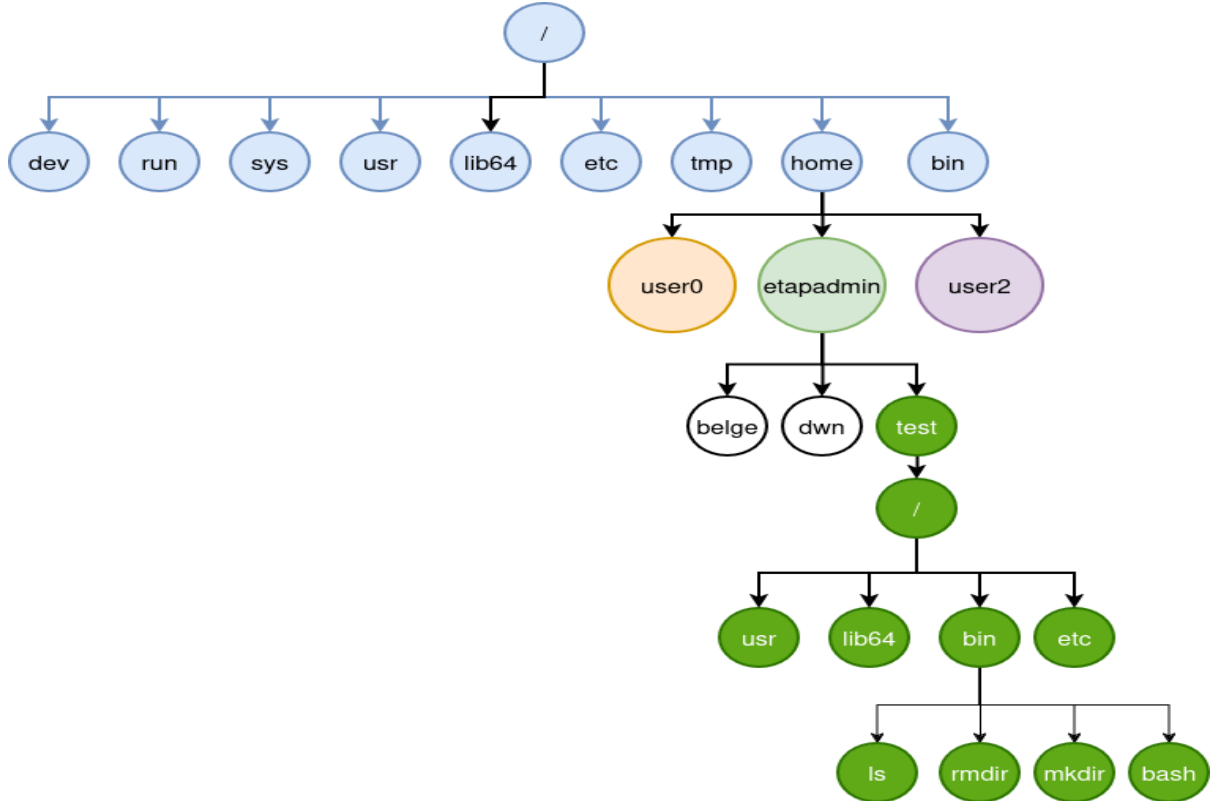
İso açıldığında aşağıdaki gibi ekran gelecektir. Burada seçili olan iso **Temel Sistem** isosudur.



Chroot Nedir?

chroot komutu çalışan sistem üzerinde belirli bir klasöre root yetkisi verip sadece o klasörü sanki linux sistemi gibi çalıştıran bir komuttur. Sağladığı avantajlar çok fazladır. Bunlar;

- Sistem tasarlama
- Sistem üzerinde yeni dağıtımlara müdahale etme ve sorun çözme
- Kullanıcı kendine özel geliştirme ortamı oluşturabilir.
- Yazılım bağımlıkları sorunlarına çözüm olabilir.
- Kullanıcıya sadece kendisine verilen alanda sınırsız yetki verme vb.



Yukarıdaki resimde user1 altında wrk dizini altına yeni bir sistem kurulmuş gibi yapılandırmayı gerçekleştirmiş.

/home/etapadmin/test dizinindeki sistem üzerinde sisteme erişmek için;

```
# sisteme erişim yapıldı.  
sudo chroot /home/etapadmin/test
```

/home/etapadmin/test dizinindeki sistem üzerinde sistemi silmek için;

```
# sistem silindi  
sudo rm -rf /home/etapadmin/test
```


Yeni sistem tasarlamak ve erişmek için temel komutları ve komut yorumlayıcının olması gerekmektedir. Bunun için bize gerekli olan komutları bu yapının içine koymamız gerekmektedir. Örneğin ls komutu için doğrudan çalışıp çalışmadığını ldd komutu ile kontrol edelim.

```
etapadmin@etahta: ~  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
etapadmin@etahta:~$ ldd /bin/ls  
linux-vdso.so.1 (0x00007ffc68471000)  
libgtk3-nocsd.so.0 => /usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0 (0x00007ff3fa9db000)  
libselinux.so.1 => /lib/x86_64-linux-gnu/libselinux.so.1 (0x00007ff3fa9ad000)  
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007ff3fa7cc000)  
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x00007ff3fa7c7000)  
libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x00007ff3fa7c2000)  
libpcre2-8.so.0 => /usr/lib/x86_64-linux-gnu/libpcre2-8.so.0 (0x00007ff3fa726000)  
/lib64/ld-linux-x86-64.so.2 (0x00007ff3faa2a000)  
etapadmin@etahta:~$
```

Görüldüğü gibi ls komutunun çalışması için bağımlı olduğu kütüphane dosyaları bulunmaktadır. Bağımlı olduğu dosyaları yeni oluşturduğumuz sistem dizinine aynı dizin yapısında kopyalamamız gerekmektedir. Bu dosyalar eksiksiz olursa ls komutu çalışacaktır. Fakat bu işlemi tek tek yapmamız çok zahmetli bir işlemdir. Bu işi yapacak script dosyası aşağıda verilmiştir.

Bağımlılık Scripti

Iddscript.sh

```
#!/bin/bash  
  
if [ $# != 2 ]  
then  
    echo "usage $0 PATH_TO_BINARY target_folder"  
    exit 1  
fi  
path_to_binary="$1"  
target_folder="$2"  
  
# if we cannot find the the binary we have to abort  
if [ ! -f "${path_to_binary}" ]  
then  
    echo "The file '${path_to_binary}' was not found. Aborting!"  
    exit 1  
fi  
  
echo "---> copy binary itself" # copy the binary itself  
cp --parents -v "${path_to_binary}" "${target_folder}"  
  
echo "---> copy libraries" # copy the library dependencies  
ldd "${path_to_binary}" | awk -F'[> ]' '{print $(NF-1)}' | while read -r lib  
do  
    [ -f "$lib" ] && cp -v --parents "$lib" "${target_folder}"  
done
```

Basit Sistem Oluşturma

Bu örnekte kullanıcının(etapadmin) ev dizinine(/home/etapadmin) test dizini oluşturuldu ve işlemler yapıldı. ls, rmdir, mkdir ve bash komutlarından oluşan sistem hazırlama.

Sistem Dizinin Oluşturulması

```
# ev dizinine test dizini oluşturuldu.  
mkdir /home/etapadmin/test/
```

/home/etapadmin/ dizinine **Bağımlılık Scripti** kodunu **lddscrip.sh** oluşturalım.

ls Komutu

```
# ls komutu ve bağımlılığı kopyalandı.  
bash lddscrip.sh /bin/ls /home/etapadmin/test/
```

```
etapadmin@etahta: ~  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
etapadmin@etahta:~$ bash lddscrip.sh /bin/ls /home/etapadmin/test/  
---> copy binary itself  
'/bin -> /home/etapadmin/test/bin  
'/bin/ls' -> '/home/etapadmin/test/bin/ls'  
---> copy libraries  
'usr -> /home/etapadmin/test/usr  
'usr/lib -> /home/etapadmin/test/usr/lib  
'usr/lib/x86_64-linux-gnu -> /home/etapadmin/test/usr/lib/x86_64-linux-gnu  
'usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0' -> '/home/etapadmin/test/usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0'  
'lib -> /home/etapadmin/test/lib  
'lib/x86_64-linux-gnu -> /home/etapadmin/test/lib/x86_64-linux-gnu  
'lib/x86_64-linux-gnu/libselinux.so.1' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libselinux.so.1'  
'lib/x86_64-linux-gnu/libc.so.6' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libc.so.6'  
'lib/x86_64-linux-gnu/libdl.so.2' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libdl.so.2'  
'lib/x86_64-linux-gnu/libpthread.so.0' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libpthread.so.0'  
'usr/lib/x86_64-linux-gnu/libpcre2-8.so.0' -> '/home/etapadmin/test/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0'  
'lib64 -> /home/etapadmin/test/lib64  
'lib64/ld-linux-x86-64.so.2' -> '/home/etapadmin/test/lib64/ld-linux-x86-64.so.2'  
etapadmin@etahta:~$
```

Bu işlemi diğer komutlar içinde sırasıyla yapmamız gerekmektedir.

rmdir Komutu

```
# rmdir komutu ve bağımlılığı kopyalandı.  
bash lddscrip.sh /bin/rmdir /home/etapadmin/test/
```

```
etapadmin@etahta: ~  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
etapadmin@etahta:~$ bash lddscrip.sh /bin/rmdir /home/etapadmin/test/  
---> copy binary itself  
'/bin/rmdir' -> '/home/etapadmin/test/bin/rmdir'  
---> copy libraries  
'usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0' -> '/home/etapadmin/test/usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0'  
'lib/x86_64-linux-gnu/libc.so.6' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libc.so.6'  
'lib/x86_64-linux-gnu/libdl.so.2' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libdl.so.2'  
'lib/x86_64-linux-gnu/libpthread.so.0' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libpthread.so.0'  
'lib64/ld-linux-x86-64.so.2' -> '/home/etapadmin/test/lib64/ld-linux-x86-64.so.2'  
etapadmin@etahta:~$
```

mkdir Komutu

```
# mkdir komutu ve bağımlılığını kopyalandı.  
bash lddscripts.sh /bin/mkdir /home/etapadmin/test/
```

```
etapadmin@etahta: ~  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
etapadmin@etahta:~$ bash lddscripts.sh /bin/mkdir /home/etapadmin/test/  
--> copy binary itself  
'/bin/mkdir' -> '/home/etapadmin/test/bin/mkdir'  
--> copy libraries  
'/usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0' -> '/home/etapadmin/test/usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0'  
'/lib/x86_64-linux-gnu/libselinux.so.1' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libselinux.so.1'  
'/lib/x86_64-linux-gnu/libc.so.6' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libc.so.6'  
'/lib/x86_64-linux-gnu/libdl.so.2' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libdl.so.2'  
'/lib/x86_64-linux-gnu/libpthread.so.0' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libpthread.so.0'  
'/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0' -> '/home/etapadmin/test/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0'  
'/lib64/ld-linux-x86-64.so.2' -> '/home/etapadmin/test/lib64/ld-linux-x86-64.so.2'  
etapadmin@etahta:~$
```

bash Komutu

```
# bash komutu ve bağımlılığını kopyalandı.  
bash lddscripts.sh /bin/bash /home/etapadmin/test/
```

```
etapadmin@etahta: ~  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
etapadmin@etahta:~$ bash lddscripts.sh /bin/bash /home/etapadmin/test/  
--> copy binary itself  
'/bin/bash' -> '/home/etapadmin/test/bin/bash'  
--> copy libraries  
'/usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0' -> '/home/etapadmin/test/usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0'  
'/lib/x86_64-linux-gnu/libtinfo.so.6' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libtinfo.so.6'  
'/lib/x86_64-linux-gnu/libc.so.6' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libc.so.6'  
'/lib/x86_64-linux-gnu/libdl.so.2' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libdl.so.2'  
'/lib/x86_64-linux-gnu/libpthread.so.0' -> '/home/etapadmin/test/lib/x86_64-linux-gnu/libpthread.so.0'  
'/lib64/ld-linux-x86-64.so.2' -> '/home/etapadmin/test/lib64/ld-linux-x86-64.so.2'  
etapadmin@etahta:~$
```

chroot Sistemde Çalışma

```
sudo chroot /home/etapadmin/test komutunu kullanmalıyız.
```

```
etapadmin@etahta: ~  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
etapadmin@etahta:~$ sudo chroot /home/etapadmin/test  
[sudo] password for etapadmin:  
bash-5.2# ls  
bin lib lib64 usr  
bash-5.2# mkdir abc  
bash-5.2# ls  
abc bin lib lib64 usr  
bash-5.2# rmdir abc  
bash-5.2# ls  
bin lib lib64 usr  
bash-5.2# pwd  
/  
bash-5.2# ldd  
bash: ldd: command not found  
bash-5.2# exit  
exit  
etapadmin@etahta:~$
```

- **abc** dizini oluşturuldu, **abc** dizini silindi, **pwd** komutuyla konum öğrenildi, **ldd** komutu sistemimizde olmadığından hata verdi.
- Çıkış için ise **exit** komutu kullanılarak sistemden çıkıldı.

Kaynak:

<https://stackoverflow.com/questions/64838052/how-to-delete-n-characters-appended-to-ldd-list>
<https://app.diagrams.net/>

Kaynak Kod Derleme

Bir uygulamanın kodları genellikle çalışmaz(python benzeri kodlar istisna). Bu kodlardan sistemlerin çalışması için çalışabilir dosyalar üretilir(linuxta ikili dosya, elf, windowsta exe, com vb.). Bu çalışabilir dosyaları koddan oluştururken iki farklı şekilde oluşturabiliriz.

1- **Paylaşımlı Derleme(dynamic)**: Kendine lazım olan kütüphaneleri sistem üzerindeki başka uygulamalarla ortak kullanır. 2- **Paylaşımsız, gömülü(static)**: Kendisine lazım olan kütüphaneleri kendi içinde barındırır(portable uygulama gibi).

Şimdi aşağıdaki kaynak kodumuzu iki farklı yöntemle derleyelim.

```
//main.c dosyamız
#include <stdio.h>
void main(){
    printf("Merhaba\n");
}
```

2-Paylaşımlı Derleme(dynamic):

Derlenen uygulama sistemde bulunan kütüphaneleri kullanacak şekilde derlenmesidir. Uygulama boyutu küçüktür, taşınabilirliği sınırlanabilir. Aşağıdaki gibi derlenir.

```
gcc -o main main.c
```

main.c kodumuzu **main** adında ikili çalışabilir dosyaya dönüştürdük. **ldd** komutuyla **main** dosyasının kullandığı kütüphaneler öğrenilir.

```
unset LD_PRELOAD
ldd ./main
    linux-vdso.so.1 (0x00007ffdb3bb9000)
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f2c53fe0000)
    /lib64/ld-linux-x86-64.so.2 (0x00007f2c541e7000)
```

Burada **libc.so.6** ve **ld-linux-x86_64.so.2** dosyaları **glibc** tarafından sağlanır. Derlenmiş dosyanın çalışması için tüm bağımlılıklarının sistemde bulunması gereklidir. Sadece gerekli olan kütüphaneleri görmek için **readelf -d** komutu kullanılabilir. Aşağıda gerekli olan kütüphaneler listeleniyor.

```
readelf -d ./main | grep -i needed
0x0000000000000001 (NEEDED)                               Paylaşımlı kitaplık: [libc.so.6]
```

ldconfig

Sistemdeki kütüphanelerin konumlarını **/etc/ld.so.conf** dosyasına bakarak belirler.

```
#/etc/ld.so.conf dosyası
/usr/lib64
/usr/lib
/lib64
/lib
```

Kütüphanelerde değişiklik yapılmışsa ve hemen bu değişikliği sistemin görmesini istersek **ldconfig** komutu kullanılmalıdır.

2-Paylaşımsız, gömülü(static):

Derlenen uygulama sistemde bulunan ve çalışması için gerekli olan kütüphaneleri uygulama içine dahil eden bir derleme yöntemidir. Uygulamamızı static derlemek için **-static** parametresi ekleyerek derlenir.

```
gcc -o main main.c -static
```

Paylaşımlı(dynamic) derleme işleminde bağımlı olduğu dosyaları **ldd** komutunu kullanarak öğrenmiştik. Şimdi paylaşımsız derlediğimiz **main** dosyasında bağımlı olduğu kütüphaneleri kontrol ediyoruz.

```
ldd main  
not a dynamic executable
```

Bağımlı kütüphaneler yerine **not a dynamic executable** mesajı gördük. Bunun anlamı çalışması için hiçbir kütüphaneye ihtiyaç duymaz. Bu bir avantaj ve taşınabilirliği artırır. Deventajı ise boyutu büyük olur. İhtiyaca göre paylaşımlı veya paylaşımsız derleme tercih edilir.

Derleme Araçları

Linux sistemlerinde kodlarımızı derlemek kullanılan uygulamalara derleme araçları denir. Birden fazla araç olsada en temel derleme araçları **make**, **cmake**, **meson**, **python**.

1-make

make, yazılım geliştirme süreçlerinde sıkça kullanılan bir araçtır. Özellikle C ve C++ gibi dillerde projelerin derlenmesi ve yönetilmesi için kullanılır.

Aşağıdaki C kodumuzu(merhaba.c dosyası) make ile nasıl derleyeceğimizi anlatalım;

```
#include <stdio.h>
int main(){
    puts("Merhaba");
    return 0;
}
```

Makefile Nedir?

Makefile, make aracının nasıl çalışacağını belirten bir dosyadır. İçinde hedefler, bağımlılıklar ve komutlar bulunur. Örneğin, bir C programını derlemek için aşağıdaki gibi basit bir Makefile oluşturabilirsiniz. Makefile ve merhaba.c dosyası aynı konumda olmalıdır.

```
CC=gcc
CFLAGS=-I.
all: program
program: merhaba.o
    $(CC) -o program merhaba.o
merhaba.o: merhaba.c
    $(CC) -c merhaba.c $(CFLAGS)
clean:
    rm -f *.o program
```

merhaba dosyasından **program** adında bir ikili dosya oluşturmak için aşağıdaki komutlar Makefile ve merhaba.c dosyasının olduğu konumda çalıştırılır.

```
make
make install
```

Genellikle Makefile dosyası olmaz. Onun yerine **configure.ac** dosyası olur. **configure.ac** dosyasından **Makefile** dosyası elde etmek için, öncelikle **autoconf** ve **automake** araçlarının sisteminizde kurulu olması gerekmektedir.

autoreconf -fvi komutu çalıştırılarak configure dosyamızı üretir. Bu araçlar, yapılandırma dosyalarınızı işleyerek gerekli **Makefile** dosyalarını oluşturmanıza yardımcı olur.

configure komutunun devamında **--prefix** dışında başka parametreleride olabilir. Bu parametreleri "Read.me" dosyası içerisinde bulunabilir veya **configure --help** komutu kaynak kodların olduğu konumda kullanılarak görülebilir. Bu kaynak kod aşağıdaki gibi derlenir.

```
$ autoreconf -fvi
$ ./configure --prefix=/usr
$ make
$ make install
```

2-cmake

CMake, projelerin derlenmesi ve yapılandırılması için kullanılan bir araçtır. Bir paketi CMake ile derlemek için genellikle aşağıdaki adımları izleriz:

İlk olarak, projenin kök dizininde bir CMakeLists.txt dosyası oluşturun. Ardından, CMake komutunu kullanarak projeyi yapılandırın ve derleyin. Örneğin:

```
mkdir compile_packages
cd compile_packages
cmake ..
make
```

Bu adımları takip ederek, CMake ile paketinizi başarıyla derleyebilirsiniz.

CMake'in esnekliği ve kullanım kolaylığı sayesinde paketlerin derlenmesi ve yapılandırılması oldukça kolay hale gelir.

Bu kaynak kod aşağıdaki gibi derlenir:

```
mkdir compile_packages
cd compile_packages
cmake ..
make
make install
```

3-meson

Meson, modern bir yapılandırma betik dili ve Ninja ise hızlı bir derleyici araçtır. Bir paketi derlemek için öncelikle Meson ile yapılandırma dosyalarını oluşturmanız gerekir. Ardından, Ninja derleyici aracını kullanarak bu yapılandırmayı derleyebilirsiniz.

İlk olarak, projenizin dizinine gidin ve Meson ile yapılandırma dosyalarını oluşturun:

```
meson setup builddir --prefix=/usr
```

Sonra, oluşturulan builddir dizinine geçin ve Ninja ile derlemeyi başlatın:

```
ninja -C builddir
```

Bu adımları takip ederek Meson ve Ninja kullanarak paketinizi başarılı bir şekilde derleyebilirsiniz.

Bu kaynak kod aşağıdaki gibi derlenir:

```
# paketin yükleneceği konum
INSTALL_ROOT=/
meson setup builddir --prefix=/usr
ninja -C builddir
INSTALL_ROOT=$INSTALL_ROOT ninja -C builddir install
```

4-python

Python ile paket derlemek için genellikle **setuptools** ve **distutils** gibi kütüphaneler kullanılır. Öncelikle, projenizin kök dizininde bir **setup.py** dosyası oluşturmanız gerekir. Bu dosya, paketin yapılandırmasını ve gereksinimlerini tanımlar.

Örnek bir setup.py dosyası aşağıdaki gibi olabilir:

```
from setuptools import setup

setup(
    name='paket_adi',
    version='1.0',
    packages=['paket'],
    install_requires=[
        'bağımlılık_paketi',
    ],
)
```

Ardından, terminalde projenizin bulunduğu dizine giderek aşağıdaki komutu çalıştırarak paketi derleyebilirsiniz:

Bu komut, paketi dist klasörüne derleyecektir. Artık paketiniz hazır ve dağıtımına uygun hale gelmiştir.

BusyBox

BusyBox, <https://busybox.net/about.html> adresteki bilgilere göre birçok temel Unix aracını tek bir ikili dosya içinde sunan, küçük ve esnek bir programdır. Çoğunlukla **initramfs** sistemlerinde ve gömülü Linux dağıtımlarında tercih edilir. BusyBox ile komutlar şu şekilde çalıştırılabilir:

```
busybox [komut]
```

Bir komutu doğrudan çalıştırabilmek için BusyBox'ı o komut adıyla sembolik bağlamak mümkündür. Örneğin, **tar** komutunu BusyBox üzerinden çalıştırmak için:

```
ln -s /bin/busybox ./tar
./tar
```

Burada busybox içindeki gömülü olan tar kullanmayı gördük. Aslında aklımıza gelen her komutun busybox içinde gömülü hali vardır. Ama bu tüm komutlar için ve tüm parametreleri için geçerli değildir. Busybox içindeki tüm komutların kısayolunu(sembolik bağı) eklemek için aşağıdaki komut kullanılır.

```
busybox --install -s /bin
```

Derleme

Gömülü sistem tasarımı veya **initramfs** içinde kullanılacak busybox **paylaşımsız(static)** derlenir. Bu şekilde derlendiğinde glibc kütüphanelerine ihtiyaç duymadan çalışacaktır.

Şimdi busybox derlemek için <https://busybox.net/> adresinden **stable** başlığı altındaki kaynak kodlarını indiriniz.

```
# kaynak kod indirilir
wget https://busybox.net/downloads/busybox-1.36.1.tar.bz2

# indirilen dosya açılır
tar -xvjf busybox-1.36.1.tar.bz2

# açılan kaynak dosyalarının bulunduğu dizine geçilir
cd busybox-1.36.1/
```

Şimdi kodlarımızı indirdik. derlemek için aşağıdaki komutları çalıştıralım.

```
# varsayılan yapılandırmayı uygula
make defconfig

# static derleme yapacaksak sed ile .config dosyasını düzenliyoruz.
sed -i "s|.*CONFIG_STATIC_LIBGCC .*|CONFIG_STATIC_LIBGCC=y|" .config
sed -i "s|.*CONFIG_STATIC .*|CONFIG_STATIC=y|" .config

# derliyoruz
make

# Static derleme sonucu aşağıdaki gibi görünür.
ldd /bin/busybox
özdevimli bir çalıştırılabilir değil
```

Derleme tamamlandığında, oluşturulan **busybox** ikili dosyası kaynak dizininde bulunur.

OpenRC

Openrc sistem açılışında çalışacak uygulamaları çalıştıran servis yöneticisidir.

Çalıştırılması

Openrc servis yönetiminin çalışması için boot parametrelerine yazılması gerekmektedir. **/boot/grub.cfg** içindeki **linux /vmlinuz init=/usr/sbin/openrc-init root=/dev/sdax** olan satırda **init=/usr/sbin/openrc-init** yazılması gerekmektedir. Artık sistem openrc servis yöneticisi tarafından uygulamalar çalıştırılacak ve sistem hazır hale getirilecek.

openrc Kullanımı

Servis etkinleştirip devre dışı hale getirmek için **rc-update** komutu kullanılır. Aşağıda **udhcpc** internet servisi örnek olarak gösterilmiştir. **/etc/init.d/** konumunda **udhcpc** dosyamızın olması gerekmektedir.

```
# servis etkinleştirmek için
rc-update add udhcpc boot
# servisi devre dışı yapmak için
rc-update del udhcpc boot
# Burada udhcpc servis adı, boot ise runlevel adıdır.
```

Elle **/etc/runlevels/** altında bulunan **boot, default, nonetwork, shutdown, sysinit** dizinlerine **/etc/init.d/** dizininin altındaki dosyaları **In** komutuyla kısayol yaparsak **rc-update add** komutunun yaptığı görevi yapmış oluruz. Kısayolu silerek **rc-update del** komutunun görevini yapmış oluruz.

/etc/runlevels/ altında bulunan **boot, default, nonetwork, shutdown, sysinit** dizinler servis dosyalarımızın hangi sırayla çalışacağını belirleyen dizinlerdir. Mesela **boot** dizini ilk açılış sırasında çalışacak olan servis dosyalarının konulacağı dizindir.

Servisleri başlatıp durdurmak için ise **rc-service** komutu kullanılır.

```
rc-service udhcpc start
# veya şu şekilde de çalıştırılabilir.
/etc/init.d/udhcpc start
```

Servislerin durumunu öğrenmek için **rc-status** komutu kullanılır. Ayrıca sistemdeki servislerin sonraki açılışta hangisinin başlatılacağını öğrenmek için parametresiz olarak **rc-update** kullanabilirsiniz.

```
# şu an hangi servislerin çalıştığını gösterir
rc-status
# sonraki açılışta hangi servislerin çalışacağını gösterir
rc-update
```

Sistemi kapatmak veya yeniden başlatmak için **openrc-shutdown** komutunu kullanabilirsiniz.

```
openrc-shutdown -p 0 # kapatmak için
openrc-shutdown -r 0 # yeniden başlatmak için
```

Servis Dosyası

Openrc servis dosyaları basit birer **bash** betiğidir. Bu betikler **openrc-run** komutu ile çalıştırılır ve çeşitli fonksiyonlardan oluşabilir. Servis dosyaları **/etc/init.d** içerisinde bulunur. Servisleri ayarlamak için ise **/etc/conf.d** içerisine aynı isimle ayar dosyası oluşturabiliriz.

Çalıştırılacak komut, komut parametreleri ve **pidfile** dosyamızı aşağıdaki gibi belirtebiliriz.

```
description="Ornek servis"
command=/usr/bin/ornek-servis
command_args=- -parametre
pidfile=/run/ornek-servis.pid
```

Bununla birlikte **start**, **stop**, **status**, **reload**, **start_pre**, **stop_pre** gibi fonksiyonlar da yazabiliriz.

```
start(){
    ebegin "Starting ${RC_SVCNAME}"
    start-stop-daemon --start --pidfile "/run/servis.pid" --exec /usr/bin/ornek-servis --parametre
}
```

Servis bağımlılıklarını belirtmek için ise **depend** fonksiyonu kullanılır.

```
depend() {
    need localmount
    after dbus
}
```

Qemu Kullanımı

qemu açık kaynaklı Virtual Box, Vmware benzeri sanallaştırma aracıdır.

Sisteme Kurulum

```
sudo apt update
sudo apt install qemu-system-x86 qemu-utils
```

qemu Kullanımı

```
# 30GB disk oluşturuldu.
qemu-img create disk.img 30G
qemu-system-x86_64 --enable-kvm -hda disk.img -m 2G -cdrom etahta.iso
# qemu-system-x86_64 --enable-kvm -hda disk.img -m 2G #sadece disk ile çalıştırılıyor
# qemu-system-x86_64 -m 2G -cdrom etahta.iso #sadece iso dosyası ile çalıştırma
```

Sistem Hızlandırılması

--enable-kvm eğer sistem disk ile çalıştırıldığında bu parametre eklenmezse yavaş çalışacaktır.

Boot Menu Açma

Sistemin diskten mi imajdan mı başlayacağını başlangıçta belirlemek için boot menu gelmesini istersek aşağıdaki gibi komut satırına seçenek eklemeliyiz.

```
qemu-system-x86_64 --enable-kvm -cdrom distro.iso -hda disk.img -m 4G -boot menu=on
```

Uefi kurulum için:

```
sudo apt-get install ovmf
```

```
qemu-system-x86_64 --enable-kvm -bios /usr/share/ovmf/OVMF.fd -cdrom distro.iso -hda disk.img -m 4G -boot menu=on
```

qemu Host Erişimi:

İstemci bilgisayar ip'si:10.0.2.15 ve ana bilgisayar 10.0.0.2 olarak ayarlıyord.

vmlinuz ve initrd

qemu ile vmlinuz ve initrd.img dosyaları iso olmadan test edilebilir.

```
qemu-system-x86_64 --enable-kvm -kernel /boot/vmlinuz-5.17 -initrd /home/deneme/initrd.img -append "quiet" -m 512m
```

qemu Terminal Yönlendirmesi

```
qemu-system-x86_64 --enable-kvm -kernel vmlinuz -initrd initrd.img -m 3G -serial stdio -append "console=ttyS0"
```

Diskteki Sistemin Açılışını Terminale Yönlendirme

```
qemu-system-x86_64 -nographic -kernel boot/vmlinuz -hda disk.img -append console=ttyS0
```

Kaynak: | <https://www.ubuntubuzz.com/2021/04/how-to-boot-uefi-on-qemu.html>

Live Sistem Oluşturma

Canlı sistem oluşturma veya RAM üzerinden çalışan sistem hazırlamak için SquashFS dosya sisteminde dağıtım sıkıştırılmalıdır. SquashFS, Linux işletim sistemlerinde sıkıştırılmış bir dosya sistemidir. Sistemimizi sıkıştırır ve ardından salt okunur bir dosya sistemine dönüştürür.

SquashFS Oluşturma

```
# mksquashfs input_source output/filesystem.squashfs -comp xz -wildcards
mksquashfs initrd $HOME/distro/filesystem.squashfs -comp xz -wildcards
```

Cdrom Erişimi

Cd veya Dvd aygıtı Linux sistemlerinde /dev/sr0 aygıt dosyası olarak erişilir. Cd içeriği üzerinde okuma yapmak için aşağıdaki komutu kullanabiliriz.

```
cat /dev/sr0
```

Cdrom Bağlama

```
mkdir cdrom
mount /dev/sr0 /cdrom
```

Bu işlem sonucunda cdrom bağlanmış olacaktır. ISO dosyamızın içerisine erişebiliriz.

Squashfs Dosyasını Bulma

Genellikle isoların içine squashfs dosyası oluşturulur. Bu sayede live yükleme yapılabilir. Örneğin, /live/filesystem.squashfs imaj dosyalarında konumudur.

Squashfs Bağlama

Squashfs dosyasını bağlamadan önce loop modülünün yüklü olması gerekmektedir. Eğer yüklemeyerseniz;

```
# loop modülü yüklenir.
modprobe loop
mkdir canli
mount -t squashfs -o loop cdrom/live/filesystem.squashfs /canli
```

Squashfs Sistemine Geçiş

Yukarıdaki adımlarda squashfs dosyamızı /canli adında dizine bağlamış olduk. Bu aşamadan sonra sistemimizin bir kopyası olan squashfs canlıdan erişilebilir veya sistemi buradan başlatabiliriz.

Squashfs dosya sistemimize bağlanmak için;

```
chroot canli /bin/bash
```

Bu işlemin yerine exec komutuyla bağlanırsak sistemimiz id "1" değeriyle çalıştıracaktır. Eğer sistemin bu dosya sistemiyle açılmasını istiyorsak exec ile çalıştırıp id=1 olmasına dikkat etmeliyiz.

kmod Nedir?

Linux çekirdeği ile donanım arasındaki haberleşmeyi sağlayan kod parçalarıdır. Bu kod parçalarını kernele eklediğimizde kerneli tekrardan derlememiz gerekmektedir. Her eklenen koddan sonra kernel derleme, kod çıkarttığımızda kernel derlemek ciddi bir iş yükü ve karmaşa yaratacaktır.

Bu sorunların çözümü için modul vardır. moduller kernele istediğimiz kod parçalarını ekleme ya da çıkartma yapabiliyoruz. Bu işlemleri yaparken kernel derleme işlemi yapmamıza gerek yok.

Kernele modul yükleme kaldırma için kmod aracı kullanılmaktadır. kmod aracı;

```
ln -s kmod /bin/depmod
ln -s kmod /bin/insmod
ln -s kmod /initrd/bin/lsmmod
ln -s kmod /bin/modinfo
ln -s kmod /bin/modprobe
ln -s kmod /bin/rmmod
```

şeklinde sembolik bağlarla yeni araçlar oluşturulmuştur.

lsmod : yüklü modulleri listeler

insmod: tek bir modul yükler

rmmod: tek bir modul siler

modinfo: modul hakkında bilgi alınır

modprobe: insmod komutunun aynısı fakat daha işlevseldir. module ait bağımlı olduğu modülleride yüklemektedir. modprobe modülü /lib/modules/ dizini altında aramaktadır.

depmod: /lib/modules dizinindeki modüllerin listesini günceller. Fakat başka bir dizinde ise basedir=konum şeklinde belirtmek gerekir. konum dizininde /lib/modules/** şeklinde kalsörler olmalıdır.

strip

strip komutu, derlenmiş program veya paylaşılan kütüphanelerin dosya boyutunu küçültmek için kullanılır. Derleme sırasında eklenen gereksiz sembol ve hata ayıklama bilgilerini kaldırarak dosyayı küçültür.

strip komutunu kullanmak için terminalde şu komutu yazabilirsiniz:

```
strip dosya_adı
```

Burada "dosya_adı", boyutu azaltılacak dosyanın adıdır. Örneğin:

```
strip program
```

Bu komut, özellikle Linux ve Unix sistemlerinde yaygın kullanılır. Dosya boyutunu küçültürken disk alanı tasarrufu sağlar ve programların dağıtımı için uygundur.

Ancak, hata ayıklama veya sembol bilgilerine ihtiyaç duyduğunuz durumlarda strip komutunu kullanmamanız gerekir.

Özetle, strip komutu derlenmiş dosyaların gereksiz bilgilerini temizleyerek boyutlarını azaltan basit ve etkili bir Linux aracıdır.

Kullanıcı Ekleme

linux sistemlerine kullanıcı eklemek için iki farklı komut bulunur. Bu komutlar adduser ve useradd komutlarıdır.

adduser:

Kullanıcı dostu bir arayüz sunar. Birçok aşamayı bizim adımıza yapar. Temelde useradd komutunu kullanarak yazılan bir script olarak düşünebiliriz.

```
# adduser komutuyla kullanıcı oluşturma
sudo adduser yeni_kullanici
```

useradd:

Kullanıcıdan tüm parametreleri girmesini ister. Bu sebepten çok tercih edilmemektedir. Fakat özel bir şekilde kullanıcı oluşturulmak istenildiğinde tercih edilir. Bu dokümanda kurulum aşamalarında useradd komutu kullanıldı.

```
# useradd komutuyla kullanıcı oluşturma
sudo useradd -m -s /bin/bash yeni_kullanici -d /home/yeni_kullanici
# -s/bin/bash : kullanıcının kullanacağı shell belirtiliyor.
# -d /home/yeni_kullanici : home dizinine oluşturulacak kullanıcı dizini belirtiyoruz
```


github

GitHub üzerinde yeni bir depo açmak oldukça basit bir işlemdir. Aşağıdaki adımları takip ederek hızlıca kendi deponuzu oluşturabilirsiniz:

- **GitHub Hesabınıza Giriş Yapın:** GitHub ana sayfasına gidin ve hesabınıza giriş yapın. Eğer bir hesabınız yoksa, öncelikle bir hesap oluşturmalsınız.
- **Yeni Depo Oluşturma:** Sağ üst köşede bulunan "+" simgesine tıklayın ve "New repository" seçeneğini seçin.
- **Depo Bilgilerini Girin:** Açılan sayfada, depo adını (repository name) ve isteğe bağlı olarak bir açıklama (description) girin. Depo özel (private) veya herkese açık (public) olarak ayarlanabilir.
- **İlk Dosyayı Oluşturma:** "Initialize this repository with a README" seçeneğini işaretleyerek, depo oluşturulduğunda otomatik olarak bir README dosyası oluşturabilirsiniz.
- **Depoyu Oluşturma:** "Create repository" butonuna tıklayarak deponuzu oluşturun.

Artık GitHub üzerinde yeni bir deponuz var! Bu depoyu yerel bilgisayarınıza klonlayarak projelerinizi yönetmeye başlayabilirsiniz. Örneğin, terminalde şu komutu kullanarak deponuzu klonlayabilirsiniz:

```
git clone https://github.com/kullaniciadi/depoadi.git
```

Bu adımları takip ederek GitHub üzerinde kolayca depo açabilirsiniz.

X11 Kullanıcısının Tespiti

Linux ortamında masaüstüdeyken sudo ile script çalıştırıldığında masaüstünü açtığımız kullanıcının tespiti aşağıdaki kodla yapılabilir.

```
#!/bin/bash

# Detect the name of the display in use
display=":$(ls /tmp/.X11-unix/* | sed 's#/tmp/.X11-unix/X##' | head -n 1)"

# Detect the user using such display
user=$(who | grep '('$display')' | awk '{print $1}')

# Detect the id of the user
uid=$(id -u $user)

echo $user $uid
```

Kaynaklar

- <https://tr.wikipedia.org/wiki/Linux> - 02/07/2025
- <https://www.subrat.info/build-kernel-and-userspace/> - 08/07/2025
- <https://medium.com/@chienhaotan/compiling-and-running-a-minimal-kernel-with-busybox-bfc45a991017> - 08/07/2025
- <https://stackoverflow.com/questions/64838052/how-to-delete-n-characters-appended-to-ldd-list> - 20/06/2025
- <https://gist.github.com/bluedragon1221/a58b0e1ed4492b44aa530f4db0ffef85> - 09/07/2025
- <https://app.diagrams.net/>
- <https://www.ubuntubuzz.com/2021/04/how-to-boot-uefi-on-qemu.html> - 20/06/2024
- <https://wiki.gentoo.org/wiki/OpenRC> - 30/06/2025
- <https://busybox.net/> - 01/07/2025
- <https://busybox.net/about.html> - 01/07/2025
- <https://sulincix.gitlab.io/distro-kitabi/> - 05/07/2025
- <https://gitlab.com/turkman/devel/doc/wiki/> - 05/07/2025
- <https://turkman.gitlab.io/devel/doc/wiki/> - 05/07/2025
- https://grok.com/share/c2hhcmQtMw%3D%3D_5c049e44-be57-4652-872f-55def669d917 - 09/07/2025
- <https://www.linuxfromscratch.org/lfs/>
- [https://wiki.archlinux.org/title/GRUB_\(T%C3%BCrk%C3%A7e\)#UEFI_sistemler](https://wiki.archlinux.org/title/GRUB_(T%C3%BCrk%C3%A7e)#UEFI_sistemler) - 08/07/2025
- https://tldp.org/HOWTO/html_single/SquashFS-HOWTO/ - 09/07/2025
- <https://askubuntu.com/questions/437880/extract-a-squashfs-to-an-existing-directory> - 11/07/2025
- https://grok.com/share/c2hhcmQtMw%3D%3D_2ad2ac6b-d067-40b0-9b55-46db0c2c98dc - 10/07/2025
- <https://chatgpt.com/>

Not: Metin düzenlemelerinde chatgpt kullanılmıştır.

Geliştiricilere Mesajımız

Gnu/Linux, açık kaynaklı bir işletim sistemidir. Kullanıcı ve geliştiricilere bir çok avantajlar sunmaktadır. Bunlar;

- Kodlarına erişilebilir(Açık Kaynak)
- Dağıtılabir
- Değişirilebilir
- Virüsten etilenme azdır
- Özelleştirilebilir
- Bağımlılığı azaltır
- Güvenlik en önemli şarttır
- Düşük donanımlarda iyi performan verir

Bu özellikleri açısından Gnu/Linux tercih etmek çok avantajlıdır. Geliştirme aşamalarına destek vermek, öğrenmek bize, toplumumuza ve ülkemize sayısız faydalar saylayacaktır.

Lisans Bilgileri

Bu proje iki farklı lisans altında dağıtılmaktadır:

1. Kaynak Kodlar (Source Code):

Kaynak kodların tamamı, aşağıdaki lisans altında dağıtılmaktadır:

Version 3, 29 June 2007

Copyright (C) <YIL> <İSİM / KURUM>

Bu program özgür bir yazılımdır: Free Software Foundation tarafından yayınlanan GNU Genel Kamu Lisansı'nın (GPL) 3. versiyonu veya (tercihinize bağlı olarak) daha sonraki bir versiyonu altında dağıtılabilir ve/veya değiştirilebilir.

Bu program, yararlı olacağı umuduyla dağıtılmaktadır, ancak **HERHANGİ BİR GARANTİ OLMAKSIZIN**; hatta **SATILABİLİRLİK** veya **BELİRLİ BİR AMACA UYGUNLUK** GARANTİSİ DAHİLİNDE BİLE OLMADAN dağıtılmaktadır. Ayrıntılar için GNU Genel Kamu Lisansı'na bakınız.

Lisansın bir kopyasını şu adresten edinebilirsiniz: <https://www.gnu.org/licenses/gpl-3.0.html>

2. Dokümantasyon ve Medya (Documentation and Media):

Proje içerisindeki tüm dokümantasyon, grafikler, görseller ve metinler aşağıdaki lisans altında dağıtılmaktadır:

Copyright (C) <2025> <İSİM / KURUM>

Bu lisans, aşağıdaki şartlar altında kullanıma izin verir:

- **Atıf (BY):** Bu çalışmayı uygun bir atıf vererek, lisans bağlantısını sağlayarak ve değişiklikleri belirterek paylaşmalısınız.
- **Aynı Lisansla Paylaş (SA):** Bu çalışmadan üretilen türev eserler aynı lisans altında lisanslanmalıdır.

Daha fazla bilgi ve tam lisans metni için şu bağlantıya bakınız:: <https://creativecommons.org/licenses/by-sa/4.0/>

Ek Bilgiler

Bu lisanslar hakkında daha fazla bilgi için:

- GNU GPLv3: <https://www.gnu.org/licenses/gpl-3.0.html>
- CC BY-SA 4.0: <https://creativecommons.org/licenses/by-sa/4.0/>